Artificial Neural Networks and Deep Learning

Complete Study Notes

Foundations

Machine Learning vs. Deep Learning · Biological & Artificial Neurons
Perceptron · Hebbian Learning · XOR Limitation
Feed Forward Neural Networks · Activation Functions · Universal Approximation

Training & Optimisation

Gradient Descent · Backpropagation · Chain Rule
MLE · Loss Functions · Overfitting · Regularisation · Dropout
Momentum · Adam · Batch Normalisation · Weight Initialisation

Computer Vision

Image Classification · Linear Classifiers · Challenges
Convolutional Neural Networks · Pooling · Receptive Fields
LeNet-5 · AlexNet · VGG · GoogLeNet · ResNet · MobileNet · EfficientNet
Data Augmentation · Transfer Learning · CAM · Grad-CAM

Dense Prediction & Detection

Semantic Segmentation · FCN · U-Net · Transpose Convolution
Localisation · Object Detection · R-CNN · Fast/Faster R-CNN · YOLO
Instance Segmentation · Mask R-CNN · Feature Pyramid Networks

Sequential Models

Recurrent Neural Networks · BPTT · Vanishing Gradients
LSTM · GRU · Seq2Seq · Beam Search

Unsupervised Representations

Autoencoders · Word Embeddings · Word2vec · GloVe

Contents

1 Machine Learning and Deep Learning

1.1 What is Machine Learning?

Machine Learning (ML) is a category of research and algorithms focused on finding patterns in data and using those patterns to make predictions. According to Tom Mitchell's classic 1997 definition: *a computer program is said to learn from experience E with respect to some class of task T and performance measure P, if its performance at tasks in T, as measured by P, improves with experience E.* In concrete terms: T is the task (regression, classification, ...), E is the data, and P is a loss or error measure.

ML falls within the broader artificial intelligence umbrella, intersecting with statistics, pattern recognition, databases, and knowledge discovery.

1.2 Machine Learning Paradigms

Given a dataset $D = \{x_1, x_2, \ldots, x_N\}$, there are three main paradigms depending on what information is available alongside the inputs.

Supervised learning is the paradigm studied most in this course. Alongside the inputs we are given desired outputs $t_1, t_2, \ldots, t_N$, and the goal is to learn a mapping that generalises correctly to new inputs. The two main flavours are *regression*, where the target is a continuous value, and *classification*, where the target is a discrete class label.

Unsupervised learning receives only the raw data D with no labels. The goal is to exploit regularities in the data to build a useful representation — for example, grouping similar examples together via *clustering*.

Reinforcement learning involves an agent that produces actions $a_1, \ldots, a_N$ which affect an environment, and in return receives scalar rewards $r_1, \ldots, r_N$. The agent learns to act so as to maximise cumulative reward over time.

1.3 The Limitation of Hand-crafted Features and the Rise of Deep Learning

Classical ML follows a two-stage pipeline: first a human expert *hand-crafts features* from raw data (e.g. SIFT/HoG for images, MFCC for audio), then a learned classifier or regressor operates on those features. This approach has two fundamental problems. First, if the chosen features do not capture the discriminative structure of the data, even the best classifier cannot succeed — in the worst case the classes are entirely inseparable in feature space. Second, designing good features requires deep domain expertise and is time-consuming.

Deep Learning replaces hand-crafted features with *learned features*: a hierarchy of representations is learned directly from raw data, optimised **end-to-end** for the task at hand. Each layer extracts increasingly abstract representations, and because the entire pipeline is trained jointly, the features are guaranteed to be useful for the final objective. Deep Learning is therefore best summarised as *learning data representations from data.*

End-to-End Training

A system is trained **end-to-end** when *every* component — from raw input to final output — is parameterised by learnable weights, and a *single* loss function drives gradient updates through the entire chain simultaneously.

Concretely, consider an image classifier built as:

$$\underbrace{\text{raw pixels}}_{\text{input}} \xrightarrow{\text{feature extractor } f_\theta} \underbrace{\mathbf{z}}_{\text{representation}} \xrightarrow{\text{classifier } g_\phi} \underbrace{\hat{y}}_{\text{prediction}} .$$

In a **classical ML pipeline**, f is hand-crafted (fixed, not learned) and only ϕ is optimised. In an **end-to-end deep learning pipeline**, *both* θ and ϕ are optimised together by back-propagating the task loss $\mathcal{L}(\hat{y}, y)$ all the way from the output back to the raw input.
Why it matters: when f is fixed, a suboptimal representation bottlenecks the classifier regardless of how well g is trained. End-to-end training removes this bottleneck: the network learns the *best possible* representation for the task at hand, not the best one a human could design.

ML vs Deep Learning: the key distinction

In classical ML the *features are fixed* and only the classifier is learned. In Deep Learning *both the features and the classifier are learned jointly*, end-to-end. This is what makes deep models so powerful on raw perceptual data (images, audio, text) where good features are hard to specify by hand.

The practical explosion of Deep Learning from 2012 onwards was enabled by two factors: the availability of *large labelled datasets* (Big Data) and *massive computational power* (GPU acceleration).

2 From Biological to Artificial Neurons

2.1 The Brain as Inspiration

In the 1940s, researchers were looking for a computational model beyond the sequential von Neumann architecture. Computers of the time were excellent at executing precise programs and fast arithmetic, but they could not interact robustly with noisy environments, were not fault-tolerant, and could not adapt. The human brain offered an inspiring alternative: roughly 10^{11} neurons, each connected to about 7,000 others via synapses, forming a massively parallel, distributed, and fault-tolerant system. These properties motivated the idea of building *artificial* neural networks.

2.2 Biological Neuron Operation

A biological neuron receives electrochemical signals through its **dendrites**. These signals are either *excitatory* (increasing the membrane potential) or *inhibitory* (decreasing it). The signals accumulate in the **cell body** (soma). When the cumulative potential exceeds a **threshold** at the **axon hillock**, the neuron *fires*: it emits an electrical spike that travels down the **axon** and is transmitted to other neurons via **synaptic terminals**, where *neurotransmitters* are released across the synapse to the dendrites of the next neuron.

2.3 The Artificial Neuron

The artificial neuron mimics this mechanism mathematically. It receives I inputs $x_1, \ldots, x_I$, each multiplied by a learned **weight** w_i (positive weights are excitatory, negative weights are inhibitory). The weighted sum is compared against a **bias** b (which plays the role of the firing threshold), and the result is passed through a nonlinear **activation function** h_j:

Artificial Neuron

$$h_j(\mathbf{x} \mid \mathbf{w}, b) = h_j\left(\sum_{i=1}^{I} w_i \cdot x_i - b\right) = h_j\left(\sum_{i=0}^{I} w_i \cdot x_i\right) = h_j(\mathbf{w}^T\mathbf{x})$$

where the bias is absorbed as an extra weight $w_0 = -b$ with a fixed input $x_0 = 1$.

The bias shift trick ($x_0 = 1$, $w_0 = -b$) is standard and simplifies notation: the neuron's full computation is just $h_j(\mathbf{w}^T\mathbf{x})$ with an augmented weight vector.

3 The Perceptron

3.1 Historical Background

Three milestones mark the early history of artificial neurons:

- **1943 — McCulloch & Pitts:** proposed the first formal neuron model, the **Threshold Logic Unit**. The activation function was the Heaviside step function: the neuron outputs 1 if its weighted input exceeds the threshold, 0 otherwise.
- **1957 — Frank Rosenblatt:** built the first **Perceptron** in hardware (the Mark I Perceptron). Weights were encoded in potentiometers and updated by electric motors during learning.
- **1960 — Bernard Widrow:** introduced the **ADALINE** (Adaptive Linear Neuron / Element), and the idea of representing the threshold as a learnable bias term w_0 — the convention still used today.

3.2 Perceptron as a Linear Classifier

The perceptron computes a weighted sum of its inputs (with bias absorbed) and applies the sign function:

Perceptron Output

$$h_j(\mathbf{x} \mid \mathbf{w}) = \text{Sign}\left(\sum_{i=0}^{I} w_i \cdot x_i\right) = \text{Sign}(w_0 + w_1x_1 + \cdots + w_Ix_I)$$

$$\text{Sign}(z) = \begin{cases} +1 & \text{if } z > 0 \\ 0 & \text{if } z = 0 \\ -1 & \text{if } z < 0 \end{cases}$$

The perceptron is a **linear classifier**: its decision boundary is the hyperplane

$$w_0 + w_1x_1 + \cdots + w_Ix_I = 0.$$

In two dimensions this reduces to a line. Solving for x_2:

$$x_2 = -\frac{w_0}{w_2} - \frac{w_1}{w_2}x_1,$$

which is just a straight line with slope $-w_1/w_2$ and intercept $-w_0/w_2$. The weight vector $\mathbf{w}$ is normal to this line, and the quantity $w_0 + \mathbf{w}^T\mathbf{x}$ is proportional to the *signed distance* of a point $\mathbf{x}$ from the boundary (see Section 3.4).

3.3 Perceptron as Logical Operator

A perceptron with two binary inputs $x_1, x_2 \in \{0, 1\}$ can implement Boolean functions.

Logical OR ($w_0 = -\frac{1}{2},\ w_1 = w_2 = 1$):

$$h_{\text{OR}}(x_1, x_2) = \begin{cases} 1 & \text{if } -\frac{1}{2} + x_1 + x_2 > 0 \\ 0 & \text{otherwise} \end{cases}$$

Logical AND ($w_0 = -2,\ w_1 = \frac{3}{2},\ w_2 = 1$):

$$h_{\text{AND}}(x_1, x_2) = \begin{cases} 1 & \text{if } -2 + \frac{3}{2}x_1 + x_2 > 0 \\ 0 & \text{otherwise} \end{cases}$$

The weights are not unique: any set of weights that places the decision boundary correctly will work. The important point is that the perceptron is *single trainable hardware* that implements any linearly separable Boolean function without being re-programmed.

3.4 Hyperplane Geometry and Signed Distance

Understanding the geometry of the decision boundary is important for understanding both the perceptron's classification rule and its learning algorithm.

Hyperplane Geometry

Let $L : w_0 + \mathbf{w}^T\mathbf{x} = 0$ be a hyperplane in $\mathbb{R}^n$. Then:

- For any two points $\mathbf{x}_1, \mathbf{x}_2$ on L: $\mathbf{w}^T(\mathbf{x}_1 - \mathbf{x}_2) = 0$, so $\mathbf{w}$ is *normal* to L.
- The unit normal is $\mathbf{w}^* = \mathbf{w}/\|\mathbf{w}\|$.
- For any point $\mathbf{x}_0$ on L: $\mathbf{w}^T\mathbf{x}_0 = -w_0$.
- The **signed distance** of any point $\mathbf{x}$ from L is:

$$\mathbf{w}^{*T}(\mathbf{x} - \mathbf{x}_0) = \frac{1}{\|\mathbf{w}\|}(w_0 + \mathbf{w}^T\mathbf{x})$$

The key insight is that the perceptron's net input ($w_0 + \mathbf{w}^T\mathbf{x}$) is directly proportional to the signed distance of $\mathbf{x}$ from the decision boundary. Points on the positive side have a positive net input, points on the negative side have a negative net input, and points exactly on the boundary have zero net input. The sign function applied to this quantity is therefore exactly the classification rule.

3.5 Hebbian Learning

Donald Hebb (1949) proposed the synaptic learning rule: *the strength of a synapse increases according to the simultaneous activation of the relative input and the desired target.* Formally, weights are updated online (one sample at a time) starting from a random initialisation, and only when a sample is misclassified:

Hebbian (Perceptron) Learning Rule

$$w_i^{k+1} = w_i^k + \Delta w_i^k, \qquad \Delta w_i^k = \eta \cdot x_i^k \cdot t^k$$

where $\eta > 0$ is the **learning rate**, x_i^k is the i-th input at step k, and $t^k \in \{-1, +1\}$ is the desired output.

Practical notes on Hebbian learning

The algorithm starts from random weights, cycles through the training set, and fixes one misclassified example at a time. It terminates when all examples are classified correctly. Two important questions arise: (1) *Does it always converge?* — Yes, if the data is linearly separable (Perceptron Convergence Theorem). (2) *Does it always converge to the same weights?* — No; the solution depends on the initialisation and the order in which examples are presented.

3.6 Perceptron Learning as Stochastic Gradient Descent

It is instructive to understand *what objective function* Hebbian learning is minimising, because this establishes the connection to gradient-based optimisation that will be central throughout the course.

Using the $+1/-1$ output coding, a misclassified point satisfies $t_i(\mathbf{w}^T\mathbf{x}_i + w_0) < 0$ (the net input has the wrong sign). The **perceptron criterion** is defined as the total (negative) signed distance of all misclassified points from the decision boundary:

$$D(\mathbf{w}, w_0) = -\sum_{i \in \mathcal{M}} t_i \left(\mathbf{w}^T\mathbf{x}_i + w_0\right)$$

where $\mathcal{M}$ is the set of currently misclassified points. This is non-negative (each term in the sum is negative, and the overall sign is flipped) and equals zero only when $\mathcal{M}$ is empty, i.e., when all points are correctly classified.

The gradients of D with respect to the model parameters are:

$$\frac{\partial D}{\partial \mathbf{w}} = -\sum_{i \in \mathcal{M}} t_i \mathbf{x}_i, \qquad \frac{\partial D}{\partial w_0} = -\sum_{i \in \mathcal{M}} t_i.$$

Applying stochastic gradient descent on a single misclassified point ($i \in \mathcal{M}$):

$$\begin{pmatrix} \mathbf{w}^{k+1} \\ w_0^{k+1} \end{pmatrix} = \begin{pmatrix} \mathbf{w}^k \\ w_0^k \end{pmatrix} + \eta \begin{pmatrix} t_i \mathbf{x}_i \\ t_i \end{pmatrix} = \begin{pmatrix} \mathbf{w}^k \\ w_0^k \end{pmatrix} + \eta \begin{pmatrix} t_i \mathbf{x}_i \\ t_i x_0 \end{pmatrix}$$

This is precisely the Hebbian update rule. Therefore:

Hebbian learning = SGD on the perceptron criterion

The Hebbian learning rule is not a heuristic — it is exactly Stochastic Gradient Descent applied to the perceptron criterion $D(\mathbf{w}, w_0)$, which measures the total distance of misclassified points from the decision boundary. Only misclassified points contribute to the gradient, which is why the rule only updates when a sample is wrong.

3.7 What the Perceptron Cannot Do: the XOR Problem

The perceptron is limited to *linearly separable* problems. The canonical counterexample is the XOR function: no single straight line can separate the $(-1, -1, -1)$, $(-1, +1, +1)$, $(+1, -1, +1)$, $(+1, +1, -1)$ pattern. Minsky and Papert formalised this limitation in their 1969 book *Perceptrons*.

More generally, the capabilities of a network depend on its topology:

- **Single layer (perceptron):** can only form half-spaces bounded by hyperplanes. Cannot solve XOR.
- **Two layers:** can form convex open or closed regions. Can solve XOR.
- **Three or more layers:** can form arbitrary regions (complexity limited by number of nodes). Can solve any classification problem given enough neurons.

Unfortunately, the Hebbian learning rule does not extend to multilayer networks — it only applies to the output layer where the target is known. For hidden layers there are no direct targets, so a different learning algorithm is needed. This motivates the development of backpropagation.

4 Feed Forward Neural Networks

4.1 Architecture

A **Feed Forward Neural Network** (FFNN), also called a **Multi-Layer Perceptron** (MLP), extends the single perceptron to a deep, layered architecture. Information flows in one direction only: from inputs to outputs.

Feed Forward Neural Network

A FFNN consists of:

- An **input layer** with I neurons (one per input feature, plus a bias unit).
- One or more **hidden layers** with $J_1, J_2, \ldots$ neurons each.
- An **output layer** with K neurons.

Layers are connected by weight matrices $W^{(l)} = \{w_{ji}^{(l)}\}$, where $w_{ji}^{(l)}$ is the weight from neuron i in layer $l-1$ to neuron j in layer l. The output of layer l depends only on the previous layer:

$$h^{(l)} = \{h_j^{(l)}(h^{(l-1)}, W^{(l)})\}$$

A FFNN is a *non-linear* parametric model characterised entirely by: the number and sizes of layers, the choice of activation functions, and the values of the weights. Crucially, activation functions must be *differentiable* to allow gradient-based training (backpropagation).

4.2 Activation Functions

The choice of activation function determines the expressiveness of each neuron and the behaviour of gradients during training. Three classical choices are:

Activation Functions

Name	Formula	Derivative	Range
Linear	$g(a) = a$	$g'(a) = 1$	$(-\infty, +\infty)$
Sigmoid	$g(a) = \dfrac{1}{1+e^{-a}}$	$g'(a) = g(a)(1-g(a))$	$(0, 1)$
Tanh	$g(a) = \dfrac{e^{a}-e^{-a}}{e^{a}+e^{-a}}$	$g'(a) = 1-g(a)^2$	$(-1, 1)$

The **sigmoid** and **tanh** are S-shaped (*squashing* functions): they map any real number to a bounded range. Both are smooth and differentiable everywhere. A useful property of the sigmoid is that its

derivative can be expressed in terms of the function itself: $g'(a) = g(a)(1 - g(a))$, which makes it cheap to compute during backpropagation.

Sigmoid vs Tanh

Tanh is a rescaled and shifted version of sigmoid: $\tanh(a) = 2\,\sigma(2a) - 1$. Tanh is zero-centred (its outputs range from -1 to $+1$), which in practice leads to better-behaved gradients in hidden layers. Sigmoid is preferred at the output layer for binary classification because its output can be interpreted as a probability.

4.3 Output Layer Design

The output layer activation depends on the task:

- **Regression:** the output spans all of $\mathbb{R}$, so use a *linear* activation. The network directly predicts a real-valued quantity.
- **Binary classification ($\{0, 1\}$ coding):** use a *sigmoid* output. The output can be interpreted as the posterior probability $p(t = 1 \mid \mathbf{x})$.
- **Binary classification ($\{-1, +1\}$ coding):** use a *tanh* output.
- **Multi-class classification (K classes):** use one output neuron per class with classes encoded as **one-hot vectors** (e.g. $\Omega_0 = [0, 0, 1]$, $\Omega_1 = [0, 1, 0]$, $\Omega_2 = [1, 0, 0]$). Apply the **softmax** activation:

Softmax Activation (Multi-class Output)

$$y_k = \frac{\exp(z_k)}{\sum_{k'=1}^{K} \exp(z_{k'})}, \qquad z_k = \sum_j w_{kj}\, h_j \left(\sum_i w_{ji}^{(1)} x_i \right)$$

The softmax guarantees that outputs are non-negative and sum to 1, making them interpretable as a probability distribution over classes.

Activation function rule of thumb

For *hidden layers*: always use sigmoid or tanh (or ReLU in modern networks). For *output layers*: match the activation to the coding and task — linear for regression, sigmoid for binary $\{0, 1\}$, tanh for binary $\{-1, +1\}$, softmax for multi-class.

4.4 Universal Approximation Theorem

Universal Approximation Theorem (Hornik, 1991)

A single hidden layer feedforward neural network with sigmoid (S-shaped) activation functions can approximate any measurable function to any desired degree of accuracy on a compact set.

This theorem is theoretically reassuring: one hidden layer is always sufficient in principle. However, it comes with important practical caveats:

- It guarantees *existence*, not that a learning algorithm will *find* the required weights.
- In the worst case, an exponential number of hidden units may be required.

- A single very wide layer may fail to generalise well. In practice, *deep* networks with multiple moderate-width layers tend to learn more efficiently and generalise better.

5 Optimisation and Learning

5.1 The Supervised Learning Objective

Given a training set $\mathcal{D} = \{(\mathbf{x}_1, t_1), \ldots, (\mathbf{x}_N, t_N)\}$, we want to find weights $\mathbf{w}$ such that the network output $g(\mathbf{x}_n \mid \mathbf{w})$ approximates the target t_n for new, unseen data. This is formalised as minimising an **error function** (also called a **loss function**) $E(\mathbf{w})$ over the training set.

5.2 Non-Linear Optimisation and Gradient Descent

The ideal approach would be to set the gradient to zero and solve analytically:

$$\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}} = 0.$$

However, because $g(\mathbf{x} \mid \mathbf{w})$ is a nonlinear function of $\mathbf{w}$ (due to the nonlinear activations), this system of equations has no closed-form solution in general. The standard approach is iterative **gradient descent**:

Gradient Descent Update

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \left.\frac{\partial E(\mathbf{w})}{\partial \mathbf{w}}\right|_{\mathbf{w}^k}$$

where $\eta > 0$ is the **learning rate**. Starting from a random initialisation, this procedure iteratively moves the weights in the direction of steepest descent of E.

Because the error surface of a neural network is non-convex and has many local minima, gradient descent may get stuck. A common mitigation is **momentum**: add a fraction α of the previous gradient step to the current one:

$$\mathbf{w}^{k+1} = \mathbf{w}^k - \eta \left.\frac{\partial E}{\partial \mathbf{w}}\right|_{\mathbf{w}^k} - \alpha \left.\frac{\partial E}{\partial \mathbf{w}}\right|_{\mathbf{w}^{k-1}}$$

Momentum gives the update inertia: it helps the optimiser continue through small local minima and speeds up progress along flat directions. Another practical strategy is to run optimisation from *multiple random restarts* and keep the solution with lowest training error.

5.3 Gradient Descent Variants

Computing the full gradient over the entire dataset at each step (**batch gradient descent**) is often computationally prohibitive for large N. Three variants trade off accuracy against cost:

Gradient Descent Variants

Variant	Gradient estimate	Bias	Variance
Batch GD	$\frac{1}{N}\sum_{n=1}^{N}\frac{\partial E(\mathbf{x}_n,\mathbf{w})}{\partial \mathbf{w}}$	Unbiased	Low
SGD	$\frac{\partial E(\mathbf{x}_n,\mathbf{w})}{\partial \mathbf{w}}$ (one sample)	Unbiased	High
Mini-batch GD	$\frac{1}{M}\sum_{n\in\text{mini-batch}}\frac{\partial E(\mathbf{x}_n,\mathbf{w})}{\partial \mathbf{w}}$	Unbiased	Medium

Mini-batch gradient descent ($M \ll N$) is the standard choice in practice: it provides a good trade-off between variance and computational cost, and enables hardware parallelism.

Why SGD works despite noisy gradients

Each SGD step uses only a single sample, so the gradient estimate is noisy. However, this noise is actually beneficial: it acts as a form of regularisation, helping the optimiser escape sharp local minima. In practice, mini-batch SGD is preferred because it reduces variance while retaining computational efficiency.

5.4 Backpropagation and the Chain Rule

Computing $\partial E/\partial w_{ji}^{(l)}$ for every weight in the network by hand would be intractable. **Backpropagation** is an efficient algorithm that computes all these gradients in a single backward pass through the network, exploiting the **chain rule** of calculus.

The chain rule states: if $z = f(g(x)) = f(y)$ with $y = g(x)$, then

$$\frac{dz}{dx} = \frac{dz}{dy}\frac{dy}{dx} = f'(y)\,g'(x) = f'(g(x))\,g'(x).$$

Applied to a two-layer network with one output, the gradient of the error with respect to a first-layer weight $w_{ji}^{(1)}$ decomposes as:

$$\frac{\partial E}{\partial w_{ji}^{(1)}} = \underbrace{\frac{\partial E}{\partial g}}_{\text{output error}} \cdot \underbrace{\frac{\partial g}{\partial w_{1j}^{(2)} h_j(\cdot)}}_{\text{output layer}} \cdot \underbrace{\frac{\partial w_{1j}^{(2)} h_j(\cdot)}{\partial h_j(\cdot)}}_{\text{weight}} \cdot \underbrace{\frac{\partial h_j(\cdot)}{\partial w_{ji}^{(1)} x_i}}_{\text{hidden layer}} \cdot \underbrace{\frac{\partial w_{ji}^{(1)} x_i}{\partial w_{ji}^{(1)}}}_{=x_i}$$

Concretely, for the two-layer network $g_1(\mathbf{x}_n \mid \mathbf{w}) = g_1\Big(\sum_{j=0}^{J} w_{1j}^{(2)} h_j\Big(\sum_{i=0}^{I} w_{ji}^{(1)} x_{i,n}\Big)\Big)$:

Gradient for First-Layer Weight (two-layer network)

$$\frac{\partial E(w_{ji}^{(1)})}{\partial w_{ji}^{(1)}} = -2\sum_{n=1}^{N} \underbrace{(t_n - g_1(\mathbf{x}_n,\mathbf{w}))}_{\text{residual}} \cdot \underbrace{g_1'(\mathbf{x}_n,\mathbf{w})}_{\text{output activation'}} \cdot \underbrace{w_{1j}^{(2)}}_{\text{next-layer weight}} \cdot \underbrace{h_j'\left(\sum_{i=0}^{I} w_{ji}^{(1)} x_{i,n}\right)}_{\text{hidden activation'}} \cdot \underbrace{x_i}_{\text{input}}$$

The structure of this expression reveals how backpropagation works: each factor corresponds to one link in the chain rule, and the same intermediate quantities computed in the *forward pass* (the activations h_j and g_1) are reused in the *backward pass* (to compute the derivatives h_j' and g_1'). The algorithm thus requires exactly two passes — one forward and one backward — regardless of the number of layers.

Backpropagation: Two-Pass Structure

Forward pass: compute all activations $h^{(1)}, h^{(2)}, \ldots, g$ from inputs to output. Store all intermediate values.

Backward pass: compute error at output. Propagate error signal layer by layer from output to input, accumulating the gradient contribution for each weight using the chain rule. Update weights with gradient descent.

Why backpropagation is efficient

A naïve implementation that differentiates E with respect to each weight independently would recompute the same intermediate derivatives many times. Backpropagation avoids this by computing and reusing the *error signal* (also called δ) at each neuron, which accumulates all the information flowing back from the output. The total computational cost is only a constant multiple of the forward pass cost.

6 Maximum Likelihood Estimation and Loss Functions

6.1 Maximum Likelihood Estimation: Motivation and Recipe

Maximum Likelihood Estimation (MLE) provides a principled statistical framework for choosing a loss function. Rather than minimising an ad hoc error, we ask: *what model parameters make the observed data most probable?*

MLE Recipe

Given a parameter vector $\boldsymbol{\theta} = (\theta_1, \ldots, \theta_p)^T$, find the MLE for $\boldsymbol{\theta}$:

1. Write the **likelihood** $L = P(\text{Data} \mid \boldsymbol{\theta})$ — the probability of the data under the model.
2. Take the **log-likelihood** $\ell = \log P(\text{Data} \mid \boldsymbol{\theta})$ (the log converts products to sums and does not change the argmax).
3. Compute $\partial \ell / \partial \boldsymbol{\theta}$ and solve $\partial \ell / \partial \theta_i = 0$ analytically, or use gradient ascent numerically.

The log is a monotone transformation, so maximising ℓ is equivalent to maximising L. Working with the log is much more convenient because the joint likelihood of i.i.d. samples is a product, and log turns products into sums.

6.2 Warm-up: MLE for a Gaussian (Derivation)

Suppose we observe i.i.d. samples $x_1, \ldots, x_N \sim \mathcal{N}(\mu, \sigma^2)$ with σ^2 known. We want to find the MLE for μ.

Step 1 — Likelihood:

$$L(\mu) = \prod_{n=1}^{N} p(x_n \mid \mu, \sigma^2) = \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right)$$

Step 2 — Log-likelihood:

$$\ell(\mu) = \sum_{n=1}^{N} \log \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(x_n - \mu)^2}{2\sigma^2}\right) = N \log \frac{1}{\sqrt{2\pi}\,\sigma} - \frac{1}{2\sigma^2} \sum_{n=1}^{N} (x_n - \mu)^2$$

Step 3 — Gradient:

$$\frac{\partial \ell(\mu)}{\partial \mu} = -\frac{1}{2\sigma^2}\frac{\partial}{\partial \mu}\sum_n (x_n - \mu)^2 = \frac{1}{2\sigma^2}\sum_n 2(x_n - \mu) = \frac{1}{\sigma^2}\sum_n (x_n - \mu)$$

Step 4 — Solve:

$$\frac{\partial \ell}{\partial \mu} = 0 \;\Rightarrow\; \sum_n (x_n - \mu) = 0 \;\Rightarrow\; \mu^{\text{MLE}} = \frac{1}{N}\sum_{n=1}^{N} x_n$$

The MLE for the mean of a Gaussian is just the sample mean. This result will motivate the squared error loss for neural network regression.

6.3 MLE for Neural Network Regression: Sum of Squared Errors

Model assumption: the target t_n is generated as

$$t_n = g(\mathbf{x}_n \mid \mathbf{w}) + \epsilon_n, \qquad \epsilon_n \sim \mathcal{N}(0, \sigma^2),$$

which implies $t_n \sim \mathcal{N}(g(\mathbf{x}_n \mid \mathbf{w}),\, \sigma^2)$. The network $g(\cdot \mid \mathbf{w})$ models the conditional mean of t given $\mathbf{x}$.

Step 1 — Likelihood:

$$L(\mathbf{w}) = \prod_{n=1}^{N} p(t_n \mid g(\mathbf{x}_n \mid \mathbf{w}), \sigma^2) = \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2}{2\sigma^2}\right)$$

Step 2 — Log-likelihood:

$$\ell(\mathbf{w}) = \sum_n \log \frac{1}{\sqrt{2\pi}\,\sigma} - \frac{1}{2\sigma^2}\sum_n (t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2$$

Step 3 — Maximise:

$$\arg\max_{\mathbf{w}} \ell(\mathbf{w}) = \arg\max_{\mathbf{w}} -\frac{1}{2\sigma^2}\sum_n (t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2 = \underbrace{\arg\min_{\mathbf{w}} \sum_n (t_n - g(\mathbf{x}_n \mid \mathbf{w}))^2}_{\text{Sum of Squared Errors}}$$

SSE loss = MLE under Gaussian noise assumption

Minimising the Sum of Squared Errors is *exactly equivalent* to Maximum Likelihood Estimation when we assume that the targets are generated by the network plus Gaussian i.i.d. noise. This gives the SSE a solid statistical justification: it is not arbitrary, but the unique loss that is optimal under this noise model.

6.4 MLE for Neural Network Classification: Cross-Entropy

For binary classification the output $g(\mathbf{x}_n \mid \mathbf{w})$ represents the probability of class 1: $g(\mathbf{x}_n \mid \mathbf{w}) = p(t_n = 1 \mid \mathbf{x}_n)$. With targets $t_n \in \{0, 1\}$, the label follows a Bernoulli distribution:

$$t_n \sim \text{Be}(g(\mathbf{x}_n \mid \mathbf{w})), \qquad p(t \mid g(\mathbf{x} \mid \mathbf{w})) = g(\mathbf{x} \mid \mathbf{w})^t \cdot (1 - g(\mathbf{x} \mid \mathbf{w}))^{1-t}$$

Step 1 — Likelihood:

$$L(\mathbf{w}) = \prod_{n=1}^{N} g(\mathbf{x}_n \mid \mathbf{w})^{t_n} \cdot (1 - g(\mathbf{x}_n \mid \mathbf{w}))^{1-t_n}$$

Step 2 — Log-likelihood:

$$\ell(\mathbf{w}) = \sum_n \Big[t_n \log g(\mathbf{x}_n \mid \mathbf{w}) + (1 - t_n) \log(1 - g(\mathbf{x}_n \mid \mathbf{w})) \Big]$$

Step 3 — Maximise (equivalently minimise the negative):

$$\arg\max_{\mathbf{w}} \ell(\mathbf{w}) = \underbrace{\arg\min_{\mathbf{w}} \; -\sum_n \Big[t_n \log g(\mathbf{x}_n \mid \mathbf{w}) + (1 - t_n) \log(1 - g(\mathbf{x}_n \mid \mathbf{w})) \Big]}_{\text{Binary Cross-Entropy}}$$

Cross-entropy loss = MLE under Bernoulli assumption

The binary cross-entropy loss is not a heuristic — it is the exact negative log-likelihood under the assumption that targets are Bernoulli random variables parameterised by the network output. Since the sigmoid output lies in $(0, 1)$, pairing it with cross-entropy is the statistically correct choice for binary classification.

For multi-class classification with K classes and softmax output, the same MLE derivation under a categorical (multinoulli) distribution gives the **categorical cross-entropy**:

$$E(\mathbf{w}) = -\sum_{n=1}^{N} \sum_{k=1}^{K} t_{nk} \log y_k(\mathbf{x}_n \mid \mathbf{w})$$

where t_{nk} is the one-hot encoded label and y_k is the softmax output for class k.

6.5 Summary: Choosing the Loss Function

Loss Function Selection Guide

Task	Output activation	Loss function	Noise model
Regression	Linear	Sum of Squared Errors	Gaussian
Binary classif. $\{0, 1\}$	Sigmoid	Binary Cross-Entropy	Bernoulli
Binary classif. $\{-1, +1\}$	Tanh	Binary Cross-Entropy (mod.)	Bernoulli
Multi-class (K classes)	Softmax	Categorical Cross-Entropy	Categorical

The guiding principle is always the same: choose the loss that corresponds to the MLE under a probability model appropriate for the type of output. In practice you can also design custom losses using domain knowledge or creativity, but this requires trial and error.

A common exam question: why SSE for regression and cross-entropy for classification?

The answer is *not* just "because they work well". The SSE arises from MLE under a Gaussian noise assumption on the targets. The cross-entropy arises from MLE under a Bernoulli (or categorical) assumption. Mismatching loss and output activation (e.g., SSE with sigmoid output for classification) is theoretically inconsistent and often performs poorly in practice.

6.6 Big Picture Summary

Model	Expressiveness	Learning Algorithm
Single perceptron	Linearly separable only	Hebbian (SGD on perceptron criterion)
Two-layer MLP	Convex regions	Backpropagation + Gradient Descent
Deep MLP (≥ 3 layers)	Arbitrary regions	Backpropagation + mini-batch SGD

Task	Loss $\equiv$ MLE under
Regression	SSE $\equiv$ Gaussian noise
Binary classification	Cross-entropy $\equiv$ Bernoulli
Multi-class classification	Cat. cross-entropy $\equiv$ Categorical

Unifying thread

Every concept in these notes is an instance of the same idea: *find the parameters that make the training data most likely under a chosen probability model.* The architecture determines what function class is available; the activation functions determine expressiveness and differentiability; the loss function encodes the probabilistic model; and gradient descent with backpropagation is the engine that finds the parameters. Understanding each piece and how they connect is what the exam will test.

7 Overfitting, Regularisation, and Generalisation

7.1 Model Complexity and the Bias-Variance Trade-off

The Universal Approximation Theorem guarantees that a single hidden layer is *sufficient in principle* to represent any function, but it says nothing about whether we can *find* those weights, or whether the resulting model will *generalise* to new data. Two failure modes flank the ideal model:

- **Underfitting (too simple):** the model M_1 lacks the capacity to capture the true structure of the data. Training error is high.
- **Overfitting (too complex):** the model M_2 is so expressive that it fits the training data exactly, including its noise, but fails to generalise to unseen examples. Training error is low but test error is high.

The ideal model M_n lies between these extremes: complex enough to capture the true pattern, simple enough not to memorise noise. This trade-off is captured by William of Ockham's principle: *Entia non sunt multiplicanda praeter necessitatem* — entities must not be multiplied beyond necessity. In modern terms: prefer the simplest model that explains the data adequately.

Inductive Hypothesis

A solution that approximates the target function well over a sufficiently large set of training examples will also approximate it well over unobserved examples — provided the training data is representative of the true data distribution.

7.2 Measuring Generalization: Training vs Test Error

Training error is a *pessimistic* measure of model quality: the model was optimised on those very examples, so any estimate of future performance based on training error alone is optimistic and unreliable. Three reasons:

- The classifier has been learned from the training data — any error estimate based on that same data will be biased downward.
- New data will not be exactly the same as training data (distribution shift, noise).
- Patterns can be found even in purely random data if the model is complex enough.

To obtain an honest estimate of future performance, we must evaluate on an **independent test set** that was not used during training. Three ways to obtain such a set: (1) someone provides an external dataset; (2) split the available data and hide part for later evaluation; (3) random subsampling with replacement (bootstrap). In classification, always use **stratified sampling** to preserve the class distribution in each split.

7.3 Dataset Terminology

A common source of confusion is the distinction between the various data splits. The following definitions are standard:

Data Split Terminology

Term	Role
Training dataset	All available labelled data
Training set	Subset used to *learn model parameters* (weights $\mathbf{w}$)
Validation set	Subset used for *model selection* (hyperparameter tuning)
Test set	Subset used for *final assessment only* — never touched during development
Training data	Training set + Validation set (used for fitting + selection)
Validation data	Validation set + Test set (used for selection + assessment)

The test set must never be used during development

The test set gives an unbiased estimate of generalisation error only if it has never influenced any modelling decision. Using the test set to select hyperparameters or compare models invalidates it as an unbiased estimator — you have effectively trained on it implicitly. This is one of the most common mistakes in applied machine learning.

7.4 Cross-Validation

Cross-validation is the general approach of using the training dataset both to train the model (parameter fitting) and to estimate its error on new data (model selection). The specific technique depends on how much data is available.

Hold-Out Validation

The simplest approach: split the training dataset into a training set and a validation set. Train on the training set, evaluate on the validation set. Fast and practical when data is abundant.

Limitation of Hold-Out

The hold-out error estimate may be biased by the specific split chosen. If by chance the validation set is not representative (e.g. contains easy or hard examples), the estimate will be misleading. This is why more robust procedures are needed when data is limited.

Leave-One-Out Cross-Validation (LOOCV)

At the opposite extreme: for a dataset of N examples, train N separate models, each time leaving out exactly one example as the validation sample. The generalisation error estimate is:

$$\hat{E} = \frac{1}{N} \sum_{n=1}^{N} E(x_n \mid \mathbf{w}_{-n})$$

where $\mathbf{w}_{-n}$ denotes the weights learned without example n. LOOCV is nearly unbiased (each model is trained on $N-1$ examples, very close to the full dataset) but computationally infeasible for large N since it requires training N models.

K-Fold Cross-Validation

The standard practical compromise: partition the training dataset into K roughly equal folds. In each of the K rounds, use one fold as the validation set and the remaining $K-1$ folds as the training set. The generalisation error estimate is:

K-Fold Cross-Validation Error

$$\hat{e}_k = \frac{1}{|N_k|} \sum_{n_k \in N_k} E(x_{n_k} \mid \mathbf{w}), \qquad \hat{E} = \frac{1}{K} \sum_{k=1}^{K} \hat{e}_k$$

where N_k is the set of examples in fold k and $|N_k|$ is its size.

K-fold CV is a good trade-off between LOOCV (unbiased but expensive) and hold-out (cheap but high variance). Typical values are $K = 5$ or $K = 10$. Note that LOOCV is the special case $K = N$.

What to do with the K models after cross-validation?

After K-fold CV, you have K trained models (one per fold). These are used *only to estimate the generalisation error*. Once the best hyperparameters are selected, you retrain a *single final model* on the entire training dataset with those hyperparameters.

7.5 Hyperparameter Tuning and the Two Levels of Optimisation

Model selection happens at two distinct levels:

1. **Parameters level:** learning the weights $\mathbf{w}$ of the network via gradient descent. This is done on the training set.
2. **Hyperparameters level:** choosing the architecture (number of layers L, number of neurons per layer $J^{(l)}$, regularisation strength γ, etc.). This is done using the validation set.

Cross-validation can be applied at both levels. At the hyperparameter level, one common approach is:

1. For each candidate hyperparameter configuration, train the model and measure validation error (using K-fold or hold-out).
2. Select the configuration with the lowest validation error.
3. Retrain the final model on the full training dataset using the selected hyperparameters.
4. Report the generalisation error on the (untouched) test set.

Computational cost of hyperparameter search

Each candidate hyperparameter configuration requires training a full model (possibly K times for K-fold CV). This cost grows quickly with the number of hyperparameters and candidate values. Always be mindful of the computational budget.

7.6 What is Regularisation?

A neural network with many parameters has enormous capacity: given enough data and training time, it can fit almost any training set — including its noise. When this happens, training error is low but generalisation error is high. This is **overfitting**.

Regularisation refers to any technique that reduces a model's generalisation error without necessarily reducing its training error. Conceptually, regularisation constrains the space of functions the model can learn — not by making the architecture smaller, but by adding an *inductive bias*: a preference for simpler, smoother, or more robust solutions.

Regularisation (informal definition)

Regularisation is any modification made to a learning algorithm that is intended to *reduce generalisation error* but not training error. It does this by introducing a bias toward simpler hypotheses, effectively preventing the model from over-specialising to the training data.

The bias–variance trade-off is central here. A highly expressive model has *low bias* (it can fit complex patterns) but *high variance* (small changes in the training data lead to very different models). Regularisation trades some bias for a large reduction in variance, improving expected generalisation. The following techniques are the main practical tools.

7.7 Early Stopping

Early stopping is the simplest and most widely used regularisation technique. The key observation is that, during gradient descent, the training error decreases monotonically (on average with SGD), but the validation error follows a U-shaped curve: it decreases initially as the model learns genuine patterns, then increases as the model starts memorising training noise.

Early Stopping Procedure

1. Hold out a validation set from the training data.
2. Train the network on the training set, monitoring validation error at each iteration k.
3. Stop training at iteration k_{ES} when the validation error starts to increase consistently.
4. Use the weights $\mathbf{w}^{k_{\text{ES}}}$ as the final model, which achieves validation error $E_{\text{ES}}(\mathbf{x} \mid \mathbf{w})$.

The validation error serves as an *online estimate of the generalisation error* throughout training. By stopping at k_{ES}, we select the point where the model has learned the most useful structure without yet overfitting.

Early stopping can also be used for **hyperparameter tuning**: train with different architectures (e.g. different numbers of hidden neurons $J^{(1)}$), apply early stopping to each, and select the architecture with the lowest early-stopping validation error. The model with the best validation error is then retrained or directly used.

Early stopping as implicit regularisation

Early stopping implicitly constrains the effective number of parameters: stopping early prevents the weights from growing large and fitting noise. It is approximately equivalent to L2 regularisation under certain conditions. Its main advantage is that it requires no modification to the loss function and adds essentially no computational overhead.

7.8 Weight Decay (L2 Regularisation)

From MLE to MAP: Adding a Prior on Weights

So far we have used **Maximum Likelihood Estimation**: $\mathbf{w}_{\text{MLE}} = \arg\max_{\mathbf{w}} P(\mathcal{D} \mid \mathbf{w})$. This maximises the probability of the data given the weights, but places no constraint on the weights themselves. We can reduce model "freedom" by introducing a *prior distribution* over the weights and using **Maximum A-Posteriori** (MAP) estimation:

$$\mathbf{w}_{\text{MAP}} = \arg\max_{\mathbf{w}} P(\mathbf{w} \mid \mathcal{D}) = \arg\max_{\mathbf{w}} P(\mathcal{D} \mid \mathbf{w}) \cdot P(\mathbf{w})$$

where the second equality follows from Bayes' theorem (dropping the constant denominator $P(\mathcal{D})$).

Empirically, small weights tend to improve generalisation of neural networks. A natural prior encoding this belief is a zero-mean Gaussian:

$$P(\mathbf{w}) \sim \mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{I}), \qquad p(w_q) = \frac{1}{\sqrt{2\pi}\,\sigma_w} \exp\left(-\frac{w_q^2}{2\sigma_w^2}\right)$$

Derivation of the L2-Regularised Loss

Assuming Gaussian noise on the targets (regression setting) and a Gaussian prior on weights:

$$\begin{aligned}
\mathbf{w}_{\text{MAP}} &= \arg\max_{\mathbf{w}} \; P(\mathcal{D} \mid \mathbf{w}) \cdot P(\mathbf{w}) \\
&= \arg\max_{\mathbf{w}} \prod_{n=1}^{N} \frac{1}{\sqrt{2\pi}\,\sigma} e^{-\frac{(t_n - g(\mathbf{x}_n|\mathbf{w}))^2}{2\sigma^2}} \cdot \prod_{q=1}^{Q} \frac{1}{\sqrt{2\pi}\,\sigma_w} e^{-\frac{w_q^2}{2\sigma_w^2}} \\
&= \arg\min_{\mathbf{w}} \sum_{n=1}^{N} \frac{(t_n - g(\mathbf{x}_n|\mathbf{w}))^2}{2\sigma^2} + \sum_{q=1}^{Q} \frac{w_q^2}{2\sigma_w^2} \\
&= \arg\min_{\mathbf{w}} \underbrace{\sum_{n=1}^{N} (t_n - g(\mathbf{x}_n|\mathbf{w}))^2}_{\text{Fitting term (SSE)}} + \underbrace{\gamma \sum_{q=1}^{Q} w_q^2}_{\text{Regularisation term}}
\end{aligned}$$

where $\gamma = \sigma^2/\sigma_w^2$ is the **regularisation coefficient**, which controls the relative importance of the prior versus the data likelihood.

L2-Regularised Loss (Weight Decay)

$$E_\gamma(\mathbf{w}) = \sum_{n=1}^{N}(t_n - g(\mathbf{x}_n|\mathbf{w}))^2 + \gamma \sum_{q=1}^{Q} w_q^2$$

The second term penalises large weights, pulling them toward zero. This is called **weight decay** because the gradient of the penalty $\frac{\partial}{\partial w_q}\gamma w_q^2 = 2\gamma w_q$ adds a term $-2\eta\gamma w_q$ to the gradient descent update, which shrinks (decays) every weight at each step.

Weight Decay = MAP under Gaussian prior

Minimising the L2-regularised loss is exactly equivalent to MAP estimation under a zero-mean Gaussian prior on the weights. The regularisation coefficient $\gamma = \sigma^2/\sigma_w^2$ encodes how strongly we believe weights should be small relative to the noise level. A large γ forces weights close to zero (high regularisation, underfitting risk); a small γ approaches MLE (low regularisation, overfitting risk).

Choosing γ via Cross-Validation

The optimal γ is a hyperparameter that must be selected using the validation set. The procedure is:

1. Train the model for several candidate values of γ (e.g. $0.1, 1, 5, 100, \ldots$), each minimising $E_\gamma^{\text{TRAIN}}(\mathbf{w}) = \sum_n (t_n - g(\mathbf{x}_n|\mathbf{w}))^2 + \gamma \sum_q w_q^2$ on the training set.
2. For each γ, evaluate the *unregularised* validation error: $E_\gamma^{\text{VAL}} = \sum_{n\in\text{val}}(t_n - g(\mathbf{x}_n|\mathbf{w}))^2$.
3. Select γ^* with the lowest validation error.
4. Retrain on the full dataset minimising $E_{\gamma^*}(\mathbf{w})$.

Note that the validation error is computed *without* the regularisation term — we want to measure prediction quality, not penalised loss.

7.9 Dropout: Stochastic Regularisation

Dropout is a regularisation technique that randomly disables neurons during training, forcing the network to learn robust, redundant representations. The idea is to prevent **co-adaptation**: the tendency of hidden units to rely on each other in ways that do not generalise.

Dropout

At each training step, each hidden unit j in layer l is independently set to zero with probability $p_j^{(l)}$ (a typical value is $p_j^{(l)} = 0.3$, i.e. 30% dropout rate). This is implemented via a binary mask:

$$m^{(l)} = [m_1^{(l)}, \ldots, m_{J^{(l)}}^{(l)}], \quad m_j^{(l)} \sim \text{Be}(p_j^{(l)})$$

The layer output becomes:

$$h^{(l)} = h_j^{(l)}\left(W^{(l)}\, h^{(l-1)} \odot m^{(l)}\right)$$

where $\odot$ denotes element-wise multiplication. At each training step a different random mask is sampled, so the network effectively trains on a different sub-network each time.

At test time, dropout is turned off and all neurons are active. To account for the fact that each unit was active only a fraction $(1 - p_j^{(l)})$ of training steps, the weights are scaled by that factor: $w \leftarrow (1 - p_j^{(l)}) \cdot w$. This ensures that the expected activation at test time matches the expected activation during training.

Dropout as ensemble learning

Dropout can be interpreted as training an exponentially large ensemble of different network architectures (one for each possible binary mask) and averaging their predictions at test time. Since all sub-networks share the same weights, this is computationally equivalent to running the full network with scaled weights. This ensemble interpretation explains why dropout is such a powerful regulariser.

Regularisation Techniques: Summary

Technique	Mechanism	Hyperparameter	Overhead
Early Stopping	Halt training when validation error starts rising; prevents weights from growing large and memorising noise.	patience / k_{ES}	Negligible
Weight Decay (L2)	Adds $\gamma\|\mathbf{w}\|^2$ penalty to the loss; equivalent to MAP under a Gaussian prior; shrinks weights each step.	γ	Negligible
Dropout	Randomly zeros hidden units (rate p) each step; prevents co-adaptation; approximates an ensemble of 2^n sub-networks.	p per layer	Moderate
Data Augmentation	Synthetically expands the training set via label-preserving transforms (flips, crops, colour jitter, ...); effectively reduces variance.	transform parameters	Low–Moderate
Batch Normalisation	Normalises activations within each mini-batch; reduces internal covariate shift; acts as an implicit regulariser (adds noise via batch statistics).	none (or ε)	Low

The techniques are complementary and routinely combined: weight decay + dropout + data augmentation + BN is standard in modern deep networks.

8 Convergence Speedup Techniques

Training a deep network is not just about minimising the loss — it is about doing so *quickly and reliably*. Even with a correct loss and a well-designed architecture, a naive gradient descent run can converge slowly, oscillate, or get trapped in poor regions. This section covers the practical techniques that make training fast and stable.

8.1 The Problem: Pathological Curvature and Flat Regions

The loss surface of a deep network is highly non-convex. Two concrete failure modes arise with vanilla gradient descent:

- **Oscillation.** When the loss surface has a narrow "ravine" (high curvature in one direction, low in another), the gradient zig-zags across the ravine instead of following the valley floor. Convergence is slow.
- **Saddle points and plateaux.** Gradients are near-zero in flat regions, so updates are tiny and training stalls — even though the loss could still decrease substantially by moving in a different direction.

8.2 Momentum

Momentum gives the optimiser *inertia*: instead of taking a pure gradient step, it accumulates a velocity vector that is a running average of past gradients. The update becomes:

SGD with Momentum

$$\mathbf{v}^{k+1} = \mu\,\mathbf{v}^{k} - \eta\,\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}^{k}), \qquad \mathbf{w}^{k+1} = \mathbf{w}^{k} + \mathbf{v}^{k+1}$$

where $\mu \in [0, 1)$ is the **momentum coefficient** (typically 0.9) and η is the learning rate.

The intuition: gradients in consistent directions accumulate over time (the velocity grows), while gradients that oscillate tend to cancel out. The result is faster progress along the loss valley and reduced oscillation across it. A common analogy: a ball rolling down a hill builds up speed in the direction of descent.

8.3 Adaptive Learning Rate Methods

A single global learning rate η is suboptimal: different parameters may need very different step sizes. Adaptive optimisers maintain *per-parameter* learning rates, adjusting them based on the history of gradients.

RMSprop

RMSprop (Hinton, 2012) keeps a running average of squared gradients per parameter and uses it to normalise the step size:

RMSprop

$$v_i^{k+1} = \rho\,v_i^{k} + (1-\rho)\left(\frac{\partial\mathcal{L}}{\partial w_i}\right)^2, \qquad w_i^{k+1} = w_i^{k} - \frac{\eta}{\sqrt{v_i^{k+1}+\varepsilon}}\,\frac{\partial\mathcal{L}}{\partial w_i}$$

where $\rho \approx 0.9$ is the decay rate and $\varepsilon \approx 10^{-8}$ prevents division by zero.

Parameters with large recent gradients receive a smaller effective step (the denominator $\sqrt{v_i}$ is large); parameters with small recent gradients receive a larger step. This automatically scales steps to the local curvature.

Adam

Adam (Adaptive Moment Estimation, Kingma & Ba 2015) combines momentum *and* adaptive step sizes. It maintains both a first moment (mean of gradients) and a second moment (mean of squared gradients), with bias correction for the early steps when the running averages are initialised at zero:

Adam

$$
\begin{aligned}
m^{k+1} &= \beta_1 m^k + (1-\beta_1)\, g^k && \text{(first moment — direction)} \\
v^{k+1} &= \beta_2 v^k + (1-\beta_2)\, (g^k)^2 && \text{(second moment — scale)} \\
\hat{m} &= \frac{m^{k+1}}{1-\beta_1^{k+1}}, \quad \hat{v} = \frac{v^{k+1}}{1-\beta_2^{k+1}} && \text{(bias correction)} \\
w^{k+1} &= w^k - \frac{\eta}{\sqrt{\hat{v}} + \varepsilon}\, \hat{m} && \text{(parameter update)}
\end{aligned}
$$

Typical defaults: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$, $\eta = 10^{-3}$.

Adam is the default optimiser in most deep learning practice: it is fast, requires little tuning, and works well across a wide range of architectures and learning rates.

8.4 Learning Rate Scheduling

Even with a good optimiser, a *fixed* learning rate is rarely optimal. A large η speeds up early training but causes oscillations near a minimum; a small η converges precisely but wastes time early on. **Learning rate scheduling** decays η over training to get the benefits of both.

Common schedules:

- **Step decay**: multiply η by a factor $\gamma < 1$ every T epochs (e.g. halve every 30 epochs).
- **Cosine annealing**: smoothly decrease η following a cosine curve from $\eta_{\max}$ to $\eta_{\min}$.
- **Warm-up**: start with a very small η, linearly increase it for the first few epochs, then apply a decay schedule. This stabilises training early when gradients are noisy.

8.5 Weight Initialisation

The choice of initial weights has a large effect on how quickly — and whether — training converges. Two key requirements:

1. **Break symmetry.** If all weights are initialised to the same value (e.g. zero), all neurons in a layer compute identical gradients and learn identical features — the network never differentiates its representations. Weights must be random.
2. **Control activation variance.** If weights are too large, activations grow exponentially through layers (exploding activations). If too small, activations shrink exponentially (vanishing activations). Either makes gradients useless.

He Initialisation (for ReLU networks)

$$
w \sim \mathcal{N}\left(0,\, \frac{2}{n_{\text{in}}}\right)
$$

where n_{in} is the number of input connections to the layer. The factor of 2 accounts for the fact

that ReLU zeroes out half the inputs on average, so the effective "fan-in" is halved. This keeps activation variance roughly constant across layers.
Use: ReLU and Leaky ReLU networks. The standard default in modern CNNs.

Glorot / Xavier Initialisation (for sigmoid/tanh networks)

$$w \sim \mathcal{U}\left[-\sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}},\ \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}\right]$$

Designed to keep both forward activation variance and backward gradient variance approximately constant. Assumes linear activations (a reasonable approximation near zero for sigmoid/tanh).
Use: sigmoid, tanh, or linear activations.

8.6 Summary Table: Convergence Speedup Techniques

Convergence Speedup Techniques: Summary

Technique	What it does and why it helps	Key hyperparameter(s)
Momentum	Accumulates velocity from past gradients; reduces oscillation in ravines, speeds up progress along consistent directions.	μ (e.g. 0.9)
RMSprop	Per-parameter adaptive step size using running average of squared gradients; large gradient $\Rightarrow$ smaller step, small gradient $\Rightarrow$ larger step.	ρ, η
Adam	Combines momentum (first moment) and RMSprop (second moment) with bias correction; fast and robust, widely used default.	β_1, β_2, η
LR Scheduling	Decays learning rate over training; large η early for speed, small η later for precision.	schedule type, γ, T
Warm-up	Gradually increases η at start of training to stabilise early gradient estimates.	warm-up epochs
He Initialisation	Sets initial weight variance to $2/n_{\text{in}}$; keeps activation and gradient variance constant in ReLU networks, preventing vanishing/exploding signals from the first step.	—
Glorot Init.	Sets variance to $2/(n_{\text{in}} + n_{\text{out}})$; designed for sigmoid/tanh.	—
Batch Normalisation	Normalises activations per mini-batch at each layer; removes internal covariate shift; allows higher learning rates; greatly stabilises and speeds up training.	ε, momentum

In practice, the standard "out-of-the-box" recipe is: **Adam** + **He initialisation** + **Batch Normalisation** + a **cosine or step LR schedule**. Momentum-SGD is preferred for fine-tuned, state-of-the-art results (e.g. ImageNet benchmarks) but requires more careful tuning.

9 Image Classification: Foundations

This lecture builds the conceptual and mathematical foundations for image classification. It motivates why images are hard to classify, introduces the linear classifier as the simplest trainable model, and shows why plain MLPs are insufficient for high-dimensional image data. Everything here sets the stage for Convolutional Neural Networks.

9.1 Computer Vision and Digital Images

Computer Vision

Computer Vision is an interdisciplinary scientific field concerned with enabling computers to gain high-level understanding from digital images or videos. It has grown extraordinarily fast in the past decade, largely owing to the marriage of deep learning and large annotated datasets.

Visual recognition encompasses a broad spectrum of tasks, from the simplest to the most complex:

- **Image Classification**: assign a single label $y \in \Lambda$ to an image I.
- **Object Detection**: localise and classify all instances of objects in an image (e.g. YOLOv3).
- **Pose Estimation**: predict the spatial configuration of a person's body joints.
- **Semantic / Instance Segmentation**: assign a class label to every pixel.
- **Image Captioning**: generate a natural-language description of an image.
- **Quality Inspection**: detect anomalous patterns in industrial imaging (e.g. silicon-wafer defect maps).

Historical shift in Computer Vision

Historically, computer vision relied on mathematical and statistical descriptions of images (hand-crafted features such as SIFT, HOG, Gabor filters). Since around 2012, machine-learning methods — and in particular deep neural networks — have become dominant, because they can *learn* the right representation from data rather than relying on human intuition.

RGB Images as Tensors

A digital **RGB image** is a three-dimensional array

$$I \in \mathbb{R}^{R \times C \times 3},$$

where R is the number of rows (height), C is the number of columns (width), and the last dimension indexes the three colour channels (Red, Green, Blue). In Python:

```
from skimage.io import imread
I = imread('image.jpg')    # shape: (R, C, 3), dtype uint8
R_channel = I[:, :, 0]
G_channel = I[:, :, 1]
B_channel = I[:, :, 2]
```

Each channel $I[:, :, k] \in \mathbb{R}^{R \times C}$ is a 2-D matrix of intensity values. Images are stored with 8 bits per channel per pixel, so values are integers in $[0, 255]$. The **pixel value** at position (r, c) is an RGB triplet, e.g. [253, 5, 6] for a bright red pixel.

Memory vs. disk size

On disk, images are typically compressed (JPEG, PNG). When loaded into memory for network training, they are decompressed: a single full-HD frame (1920×1080, 3 channels) occupies approximately 6 MB uncompressed. This is an important practical constraint: during training, batches of uncompressed images must fit (or be streamed) through GPU memory.

Videos as Higher-Dimensional Tensors

A colour video of T frames is naturally represented as a 4-D tensor:

$$V \in \mathbb{R}^{R \times C \times 3 \times T}.$$

A video at full HD and 24 fps consumes approximately 150 MB/s *before* compression. Visual data are highly redundant (adjacent pixels and frames are strongly correlated), which is why codecs (H.264, HEVC) can compress this by orders of magnitude. Machine-learning algorithms for video must account for this dimensionality explosion when designing architectures and memory budgets.

9.2 Basics of Image Filtering: Spatial Correlation

Before defining how neural networks process images, it is essential to understand the most fundamental image processing operation: **local spatial filtering** (also called correlation). This is the operation that convolutional layers are built upon.

Local (Spatial) Transformations

A **local spatial transformation** computes the output at pixel (r, c) as a function of the pixel values in a neighbourhood $\mathcal{U}$ centred at (r, c):

$$G(r, c) = T_{\mathcal{U}}[I](r, c),$$

where I is the input image, G is the output, $\mathcal{U}$ defines the set of displacement vectors (u, v) around the centre, and $T_{\mathcal{U}}$ is the transformation function. The output at (r, c) depends on all values $I(r + u, c + v)$ for $(u, v) \in \mathcal{U}$.

When the *same* function T is applied at every pixel regardless of position, the transformation is called **space-invariant** (or shift-equivariant). This is the regime of classical image filters and CNN convolutional layers.

Correlation: The Linear Case

If $T_{\mathcal{U}}$ is *linear*, the output is a weighted sum of the neighbourhood pixels:

$$T[I](r, c) = \sum_{(u,v)\in\mathcal{U}} w(u, v) \cdot I(r + u,\, c + v),$$

where the weights $\{w(u, v)\}$ form the **filter** (or **kernel**) $\mathbf{w}$. For a filter of size $(2L + 1) \times (2L + 1)$, this operation is called **correlation**:

Correlation

The **correlation** of image I with filter $\mathbf{w}$ is

$$(I \otimes \mathbf{w})(r, c) = \sum_{u=-L}^{L} \sum_{v=-L}^{L} w(u, v) \cdot I(r + u,\, c + v).$$

The filter $\mathbf{w}$ is a $(2L + 1) \times (2L + 1)$ matrix of learnable (or hand-designed) weights. The same filter is slid over every position (r, c) of the input — this is the *weight-sharing* principle that CNNs exploit.

In Python, the inner loop for a single output pixel looks like:

```
acc = 0
for u in range(-L, L+1):
    for v in range(-L, L+1):
        acc += image[r + u, c + v] * w[u, v]
output[r, c] = acc
# Wrap this in outer loops over r, c to obtain the full output image
```

For RGB (colour) images, correlation is computed *channel-wise* and then summed:

$$T[I](r,c) = \sum_{i=1}^{3} \sum_{(u,v)\in\mathcal{U}} w_i(u,v) \cdot I(r+u,\, c+v,\, i).$$

Here the filter $\mathbf{w}$ has three planes (one per colour channel), and the output is a scalar.

Correlation as Template Matching

Correlation of an image I with a filter $\mathbf{w}$ measures how much the image *resembles* $\mathbf{w}$ at each location. The output is high where the image patch closely matches the filter and low elsewhere. This is the basis of the *template-matching* interpretation of both classical filters and learned CNN filters.
Exam tip: When both the image and the filter have the same size (e.g. 32×32 RGB), correlation reduces to a single scalar: the inner product $\mathbf{w}^\top\mathbf{x}$ (where $\mathbf{x}$ is the flattened image). This is *exactly* the score computed by a linear classifier.

Normalisation in template matching

If the filter $\mathbf{w}$ is used for template matching (finding where a pattern occurs in an image), correlation alone is unreliable: any pixel belonging to a large uniform white region will yield a correlation as high as the correct match position (because the region is large, the sum is large). **Normalised cross-correlation** divides by the local image energy, making the score invariant to local brightness and contrast.

9.3 The Image Classification Problem

Image Classification

Given an image $I \in \mathbb{R}^{R\times C\times 3}$ and a fixed label set $\Lambda = \{\lambda_1, \ldots, \lambda_L\}$, **image classification** is the task of assigning a label $y \in \Lambda$ to I.
The classifier is a parameterised function

$$\mathcal{K}_\theta : \; I \;\longmapsto\; \mathcal{K}_\theta(I) \in \Lambda,$$

where θ denotes the learnable parameters. In practice, a neural network classifier outputs a vector of *confidence scores* (or posterior probabilities) over all L classes:

$$\mathcal{K}_\theta : \; \mathbb{R}^{R\times C\times 3} \;\longrightarrow\; [0,1]^L,$$

and the predicted class is $\hat{y} = \arg\max_i \mathcal{K}_\theta(I)_i$.

9.4 Training a Linear Classifier on Images

Feeding Images to a Neural Network: Column-wise Unfolding

A fully-connected layer expects a 1-D input vector. The standard approach is to **unfold** (flatten) the image $I \in \mathbb{R}^{R \times C \times 3}$ column-wise into a vector $\mathbf{x} \in \mathbb{R}^d$ with $d = R \times C \times 3$:

$$\mathbf{x} = \mathrm{vec}(I), \qquad d = R \cdot C \cdot 3.$$

For the **CIFAR-10** dataset (32×32 RGB images, 60,000 images in 10 classes), $d = 32 \times 32 \times 3 = 3072$. While this is modest by today's standards, it already exceeds the dimensionality of 92% of UCI Machine Learning Repository datasets.

The Linear Classifier

A single-layer network with L output neurons (one per class) and no nonlinearity in the output is a **linear classifier**:

$$\mathbf{s} = \mathcal{K}(\mathbf{x}; W, \mathbf{b}) = W\mathbf{x} + \mathbf{b},$$

where

- $W \in \mathbb{R}^{L \times d}$ is the **weight matrix**,
- $\mathbf{b} \in \mathbb{R}^L$ is the **bias vector**,
- $\mathbf{s} \in \mathbb{R}^L$ is the vector of **class scores** (s_i = score for class i).

The score for class i is:

$$s_i = W[i,:] \cdot \mathbf{x} + b_i = \sum_{j=1}^{d} w_{i,j}\, x_j + b_i.$$

The predicted class is $\hat{y} = \arg\max_i s_i$.

In Keras, this corresponds to:

```
model = Sequential([
    Input(shape=(32, 32, 3)),
    Flatten(),                # 32x32x3 -> 3072
    Dense(10)                 # 3072 -> 10 scores (no activation)
])
# Total parameters: 3072 * 10 + 10 = 30,730
```

No softmax needed at prediction time

Softmax is a monotone transformation (it preserves ordering of scores). At *prediction time*, applying softmax before taking the argmax does not change the predicted class. It is only needed when computing probabilities or the cross-entropy loss during training.

Why Nonlinearities Are Needed in Deeper Networks

Stacking multiple linear layers (no activation functions) is equivalent to a *single* linear layer. Formally, for three layers:

$$\mathbf{s} = W_3\big(W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2\big) + \mathbf{b}_3 = \widetilde{W}\,\mathbf{x} + \tilde{\mathbf{b}},$$

where $\widetilde{W} = W_3W_2W_1$ and $\tilde{\mathbf{b}} = \mathbf{b}_3 + W_3\mathbf{b}_2 + W_3W_2\mathbf{b}_1$. No matter how many linear layers we stack, the composition is still a linear map $\mathbb{R}^d \to \mathbb{R}^L$. **Nonlinear activation functions break this collapse**, giving deep networks their representational power.

Exam-critical point

Stacking linear layers without activations is pointless: the result is mathematically equivalent to a single linear layer. Always activate intermediate layers (ReLU, sigmoid, etc.).

The Parameters Are Templates

The rows of W can be reshaped back into images of size $R \times C \times 3$. Row $W[i,:]$ is the **learned template** for class i. The score $s_i = W[i,:] \cdot \mathbf{x} + b_i$ is exactly the correlation of the input image with the class template (when both have the same size).

Template-matching interpretation of the linear classifier

Training a linear classifier on images is equivalent to learning one *prototype* (template) per class. At test time, each input image is compared (via inner product / correlation) against all L templates, and the class with the highest similarity score wins.

What do CIFAR-10 templates look like? The learned templates are blurry averages of the class images: *plane* and *ship* templates are bluish (sky/sea background); *car* and *truck* are reddish/grey; *horse* has *two heads* (one facing left, one right) because the training set contains horses facing both directions.

Key insight for the exam: The two-headed horse reveals a fundamental limitation of the linear classifier — it can learn only *one* template per class. Any object that can appear in multiple orientations or configurations cannot be captured by a single prototype. This is why deeper architectures (with multiple hidden layers) are needed: they can represent multiple modes of a class.

Image augmentation follows from this insight

If you want a model to be invariant to left-right flips, you must show it flipped examples during training. The two-headed horse emerges precisely because the training set contains both orientations — the linear classifier has no built-in invariance, so it averages them. This is the founding principle of **data augmentation**: apply the transformations you want the model to be invariant to, and include the augmented examples in the training set.

Training the Linear Classifier

Training finds the parameters $W, \mathbf{b}$ that minimise the total loss over the training set $\mathcal{T}_R$:

$$W^*, \mathbf{b}^* = \arg \min_{W \in \mathbb{R}^{L \times d},\, \mathbf{b} \in \mathbb{R}^L} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{T}_R} \mathcal{L}_{W,\mathbf{b}}(\mathbf{x}_i, y_i) + \lambda \mathcal{R}(W, \mathbf{b}),$$

where $\mathcal{L}$ is a loss function (e.g. cross-entropy), $\mathcal{R}$ is a regulariser, and $\lambda > 0$ balances the two terms. This is solved by gradient descent (or its stochastic variants).

Once training is complete, the training set can be discarded — only W and $\mathbf{b}$ are needed at inference time.

Geometric Interpretation

Interpret each flattened image $\mathbf{x} \in \mathbb{R}^d$ as a point in a d-dimensional space. The score function for class i,

$$f_i(\mathbf{x}) = W[i,:] \cdot \mathbf{x} + b_i,$$

is a *linear function* of $\mathbf{x}$. The decision boundary between class i and class j is the set $\{f_i = f_j\}$, which is a **hyperplane** in $\mathbb{R}^d$. Thus, the linear classifier partitions the input space into L *convex polyhedra* separated by hyperplanes.

Why hidden layers matter geometrically

Because linear classifiers partition the input space with *hyperplanes*, they cannot separate classes whose decision boundary is non-linear (e.g. a ring-shaped class surrounded by another). Adding hidden layers with nonlinear activations allows the network to *warp* the input space before drawing the final linear boundary, making arbitrarily complex partitions possible.

Dimensionality and Scalability

Even with the modest CIFAR-10 images ($d = 3072$), a two-layer MLP with a hidden layer of $d/2 = 1536$ neurons already has:

$$1536 \times 3072 + 1536 = 4{,}720{,}128 \text{ parameters (first layer alone).}$$

For full-HD images ($d = 1920 \times 1080 \times 3 \approx 6 \times 10^6$), a single hidden layer of comparable size would require hundreds of *billions* of parameters. This motivates the need for architectures (CNNs) that exploit the spatial structure of images rather than treating them as generic vectors.

9.5 The Challenges of Image Classification

Image classification is deceptively hard. The five main challenges are:

1. High Dimensionality

Even the tiny 32×32 CIFAR-10 images live in $\mathbb{R}^{3072}$, a space far larger than most tabular ML datasets (92% of UCI datasets have fewer than 3,200 features). Full-resolution images are several orders of magnitude larger still. High dimensionality makes distance-based methods unreliable and complicates optimisation.

2. Label Ambiguity

A single image can be truthfully labelled with many different words ("man", "beer", "dinner", "restaurant", "sausages", ...). The "correct" label depends on the intended task and the granularity of the label set Λ. Collecting consistent annotations is difficult and expensive.

3. Transformations that Change the Image but Not Its Label

The same physical object generates radically different pixel arrays depending on:

- **Illumination conditions**: shadows, colour temperature, specular reflections.
- **Viewpoint**: rotation in 3-D, perspective foreshortening.
- **Deformation**: a cat sitting vs. stretching vs. curled up.
- **Occlusion**: part of the object is hidden.
- **Background clutter**: irrelevant, visually salient background pixels.

A good classifier must be *invariant* (or at least *robust*) to all of these.

4. Inter-class Variability

Images within the same class can be more different from each other (pixel-wise) than images from different classes. For instance, a white cat against a light background and a black cat against a dark background are both "cats", yet their pixel representations are very similar to completely different-class images with matching colour statistics.

5. Perceptual Similarity ≠ Pixel Similarity

Critical limitation of pixel-wise distances

Pixel-wise distances (e.g. ℓ_2: $\|\mathbf{x}_j - \mathbf{x}_i\|_2$) do *not* reflect semantic similarity. Three images can be equidistant (in ℓ_2) from a reference image while being semantically completely different: a brightness shift, a spatial translation, or random pixel noise can all produce the same ℓ_2 distance as a genuine change in scene content.
t-SNE visualisations of CIFAR-10 confirm this: when images are embedded by pixel values, classes overlap heavily — images from different classes often lie closer together (pixel-wise) than images from the same class. This means any classifier that relies purely on pixel distances will fail.

9.6 The k-Nearest Neighbour Classifier

The k-NN classifier is the simplest conceivable classifier, and understanding its failure on images powerfully motivates more sophisticated architectures.

Definition

k-Nearest Neighbour (k-NN) Classifier

Given a training set $\mathcal{T}_R = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$, the k-NN classifier assigns to a test image $\mathbf{x}_j$ the most frequent label among its k nearest neighbours in the training set:

$$\hat{y}_j = \text{mode}\big\{\, y_i : \mathbf{x}_i \in \mathcal{U}_k(\mathbf{x}_j)\big\},$$

where $\mathcal{U}_k(\mathbf{x}_j)$ is the set of k training images closest to $\mathbf{x}_j$ under some distance measure $d(\cdot,\cdot)$. Common choices for the distance:

$$d_2(\mathbf{x}_j, \mathbf{x}_i) = \|\mathbf{x}_j - \mathbf{x}_i\|_2 = \Big(\sum_k (x_j^{(k)} - x_i^{(k)})^2\Big)^{1/2} \quad (\ell_2 \text{ distance}),$$

$$d_1(\mathbf{x}_j, \mathbf{x}_i) = \|\mathbf{x}_j - \mathbf{x}_i\|_1 = \sum_k |x_j^{(k)} - x_i^{(k)}| \quad (\ell_1 \text{ / Manhattan distance}).$$

For $k = 1$: the test image is assigned the label of its single closest training image.

Decision boundary intuition. As k increases, the decision boundaries become smoother and less sensitive to individual training examples, reducing variance at the cost of increased bias. At the extreme $k = N$ the classifier always predicts the majority class.

Why k-NN Fails on Images

1. **Computational cost at test time.** Each prediction requires computing N distances, each of dimension d. For large datasets ($N = 10^6$) and high-dimensional images ($d \sim 10^5$), this is prohibitively slow.
2. **Memory.** The entire training set must be kept in memory.
3. **Pixel distance $\neq$ semantic distance.** As discussed above, ℓ_2 or ℓ_1 distances on pixel vectors fail to capture semantic content. An image shifted by one pixel is far from the original in ℓ_2, yet perceptually identical. Three images equidistant in ℓ_2 from a reference may be semantically completely different (one shifted, one colour-shifted, one with added noise).
4. **Curse of dimensionality.** In high dimensions, all points tend to become equidistant, making the concept of "nearest neighbour" meaningless.

The t-SNE CIFAR-10 experiment

When CIFAR-10 images are embedded with t-SNE (a 2-D non-linear dimensionality reduction technique that preserves local pixel-space distances), the resulting 2-D map shows that classes are heavily intermixed. Images that are semantically identical (two different cats) can be far apart in pixel space, while images from different classes (a white cat and a snowy mountain) can be close together.
This definitively shows that **raw pixel distances are not a useful measure of semantic similarity for image classification** — a fundamentally different representation is needed. This is precisely what convolutional neural networks provide.

9.7 Recap: Image Classification Foundations

Image Classification Foundations — Summary (Exam Checklist)

1. **RGB image**: $I \in \mathbb{R}^{R \times C \times 3}$; 8 bits per channel; loaded as uncompressed array during training. Video: $V \in \mathbb{R}^{R \times C \times 3 \times T}$.
2. **Correlation**: $(I \otimes \mathbf{w})(r, c) = \sum_{u,v} w(u, v)\, I(r+u, c+v)$; filter $\mathbf{w}$ slid over all positions; for RGB images, computed channel-wise and summed.
3. **Linear classifier on images**: flatten image to $\mathbf{x} \in \mathbb{R}^d$; $\mathbf{s} = W\mathbf{x} + \mathbf{b}$, $W \in \mathbb{R}^{L \times d}$; predicted class $= \arg\max_i s_i$. No softmax needed at prediction time.
4. **Template-matching interpretation**: row $W[i, :]$ reshaped to image size is the learned template for class i; score s_i is the correlation between input and template.
5. **Two-headed horse**: linear classifier learns one template per class; multiple appearances (e.g. horse facing left vs. right) get averaged — motivates data augmentation and deeper architectures.
6. **Why nonlinearities**: stacking linear layers collapses to a single linear map $\widetilde{W}\mathbf{x} + \tilde{\mathbf{b}}$ — depth without activations is useless.
7. **Five challenges of image classification**: (1) high dimensionality; (2) label ambiguity; (3) label-preserving transformations (illumination, viewpoint, deformation, occlusion); (4) inter-class variability; (5) perceptual similarity $\neq$ pixel similarity.
8. **k-NN classifier**: no training time; $\hat{y}_j =$ mode of k nearest training images. Fails on images because pixel distances are semantically meaningless, test-time cost is $O(Nd)$, and the method suffers from the curse of dimensionality.
9. **t-SNE on CIFAR-10**: classes are heavily intermixed in pixel space — raw pixel distances cannot separate semantic classes. A special architecture (CNN) is needed.

10 Convolutional Neural Networks (CNNs)

10.1 Motivation: From Hand-Crafted to Data-Driven Features

A central question in visual recognition is: *what feature representation should we feed to a classifier?* The classical pipeline uses **hand-crafted features** — human-designed algorithms that compress an image into a meaningful vector (e.g. average height, area, perimeter for parcel classification, or SIFT/HoG for natural images). A downstream classifier (tree, SVM, MLP) then operates on this fixed vector.

Pros and Cons of Hand-Crafted Features

Advantages	Disadvantages
Exploit prior / expert knowledge	Require significant design effort
Features are interpretable	Risk overfitting the design set
Adjustable when not working	Not portable across domains
Little training data needed	Not viable for complex visual tasks
Can weight features differently	Hard to scale (e.g. car vs. motorbike)

The key limitation is practical: *can you write a program that takes an image as input and decides*

whether it contains a car or a motorbike? For humans, this is trivial; for a hand-crafted feature designer, it is nearly impossible. **Deep Learning** solves this by learning both the features *and* the classifier jointly from data.

Data-Driven Feature Extraction (CNN paradigm)

In a CNN, the feature extraction stage is itself a learnable function, parameterised by convolutional filter weights. The whole pipeline — from raw pixels to class probabilities — is optimised end-to-end by gradient descent on labelled data.

10.2 Linear Filtering and Correlation

Before introducing CNNs, it is helpful to recall classical image filtering, since convolution is the core operation in CNNs.

Correlation (Cross-Correlation)

Given an image $I \in \mathbb{R}^{r \times c}$ and a filter (kernel) w defined over a spatial neighbourhood U, the **correlation** output is:

$$T_I(r, c) = \sum_{(u,v) \in U} w(u, v) \cdot I(r + u,\, c + v)$$

Each output pixel is a *linear combination* of its spatial neighbourhood in the input, weighted by the filter w. The filter w entirely defines the operation.

Common examples:

- **Identity filter** $\begin{pmatrix} 0 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0 \end{pmatrix}$: output equals input.
- **Box filter** $\frac{1}{9}\begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}$: averages all neighbours — blurs the image.
- **Edge detection filter**: highlights intensity discontinuities.
- **Motion blur filter**: simulates camera motion along a trajectory.

Convolution vs. Correlation

Convolution

$$T_I(r, c) = \sum_{(u,v) \in U} w(u, v) \cdot I(r - u,\, c - v) \equiv \sum_{(u,v) \in U} w(-u, -v) \cdot I(r + u,\, c + v)$$

Convolution is equivalent to correlation with a *flipped* filter. In image processing, convolution is preferred because it satisfies elegant Fourier-domain properties (convolution theorem). However, in CNNs the distinction is irrelevant: since filters are *learned*, using a flipped or unflipped version simply yields a mirrored filter — the network learns the right one either way. **In practice, all modern CNN frameworks implement correlation, not convolution**, despite calling it "Conv2D".

Why CNNs use correlation not convolution

The only difference between convolution and correlation is a flip of the kernel. Since CNN filters are learned from data, the network will simply learn the flipped version of whatever the "correct" filter should be. The choice of operation does not matter for expressiveness or learning dynamics; the name "convolutional neural network" is historical.

Convolution in CNNs vs. Classical Image Filtering

In classical image filtering, RGB images are filtered *channel-independently*: the filter is applied separately to R, G, B, and the output is still RGB. Channels are *not mixed*.

In CNNs, convolution always *mixes all channels*:

$$(I \circledast w_l)(r,c) = \sum_{k=1}^{C} \sum_{(u,v)\in U} w_l(u,v,k) \cdot I(r+u,\, c+v,\, k)$$

The result is a single 2D activation map per filter l. Input channels are always summed over, producing richer cross-channel features.

Padding

When computing the output near image boundaries, the filter extends beyond the valid domain. The standard solutions are:

- **Zero-padding** (`padding='same'`): pad the image with zeros. This is the most common choice because it preserves the spatial output size.
- **No padding** (`padding='valid'`): only compute outputs where the filter fits entirely inside the image. Spatial size shrinks.
- **Symmetric/reflect padding**: mirror the image at the boundary.

Stride

The **stride** controls how many pixels the filter jumps between applications:

- **Stride 1**: standard convolution, full overlap.
- **Stride 2**: filter jumps 2 pixels each step, output spatial size halved. Equivalent to convolution followed by downsampling, but more computationally efficient as a single operation.

Strided convolutions are widely used in CNNs to reduce spatial resolution, often replacing max-pooling in modern architectures.

Exam point: stride vs. pooling

Both strided convolution and max-pooling reduce spatial dimensions. The difference is that max-pooling introduces a *nonlinear* operation (taking the maximum), while strided convolution is *linear* (though followed by a nonlinear activation). Both increase the receptive field of subsequent layers.

10.3 CNN Architecture Overview

A CNN processes an input image through a sequence of **volumes** (3D tensors: height × width × depth/channels). As we progress through the network:

- Spatial dimensions (height, width) *decrease*.
- The number of channels (depth) *increases*.
- The abstract complexity of the represented features *increases*.

The typical CNN has three functional stages:

CNN Building Blocks

1. **Feature Extraction Network (FEN):** a stack of *convolutional blocks*, each consisting of Convolutional Layer → Activation → Pooling. Learns hierarchical, increasingly abstract features.
2. **Flattening:** converts the final spatial volume into a vector (or replaced by Global Average Pooling).
3. **Dense / Fully-Connected layers:** a classical MLP that maps the feature vector to class probabilities.

10.4 Convolutional Layers

Convolutional Layer

A convolutional layer with N_F filters produces an output activation volume $a \in \mathbb{R}^{R' \times C' \times N_F}$ from an input volume $x \in \mathbb{R}^{R \times C \times C_{\text{in}}}$:

$$a(r, c, l) = \sum_{\substack{(u,v) \in U \\ k=1,\ldots,C_{\text{in}}}} w^l(u, v, k) \cdot x(r + u,\, c + v,\, k) + b^l, \qquad \forall\,(r, c),\ l = 1, \ldots, N_F$$

Parameters:

- Weights: $h_r \cdot h_c \cdot C_{\text{in}} \cdot N_F$ (filter weights)
- Biases: N_F (one per filter)
- **Total:** $h_r \cdot h_c \cdot C_{\text{in}} \cdot N_F + N_F$

The filter spatial size (h_r, h_c) must be specified; the depth C_{in} equals the number of input channels automatically.

Key observations:

- Each filter w^l spans the *full depth* of the input but only a *small spatial neighbourhood* U. This is the fundamental difference from a dense layer.
- Different filters applied to the same input produce different "slices" (channels) in the output volume. Each filter learns to detect a different feature (e.g. horizontal edges, colour blobs, textures).
- The *same filter* is applied at every spatial location (weight sharing) — this is what gives CNNs their **translation equivariance**.

CNN Arithmetic: Parameter Count Example

Example: 3 input channels, 2 filters of size 5×5.
Filter weights: $3 \times 5 \times 5 \times 2 = 150$. Biases: 2. **Total: 152 parameters**.

Exam question: parameter count for a conv layer

Given a conv layer with filter size (h_r, h_c), C_{in} input channels, and N_F filters:

$$\text{Parameters} = h_r \cdot h_c \cdot C_{\text{in}} \cdot N_F + N_F$$

Two layers with identical hyperparameters (h_r, h_c, N_F) can have *different* parameter counts if they sit at different positions in the network (different C_{in}). Always check the depth of the input volume.

10.5 Activation Layers

Activation functions introduce *nonlinearity* into the network. Without them, stacking multiple convolutional layers would be equivalent to a single linear operation — the CNN would reduce to a linear classifier. Activations operate element-wise on each value in the volume and do *not* change the volume size.

Two classical pathologies arise from a bad choice of activation function in deep networks: *saturation* (leading to vanishing gradients) and the *dying neuron* problem. Both are worth understanding clearly before looking at the individual activations.

The Saturation and Vanishing Gradient Problem

Sigmoid and tanh both *saturate*: for large positive or negative inputs, their output is essentially constant (close to 1 or -1), so their derivative is almost zero. This becomes catastrophic during backpropagation.

Recall that the gradient of the loss with respect to a weight in layer l is a *product* of derivatives across all subsequent layers:

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} \propto \prod_{k=l}^{L} \sigma'\big(z^{(k)}\big)$$

If each σ' is small (as it is in the saturated regime), this product *shrinks exponentially* with depth. In a 10-layer network with sigmoid activations and saturated neurons, each factor $\sigma' \lesssim 0.25$, so the gradient reaching layer 1 is multiplied by $0.25^{10} \approx 10^{-6}$ — effectively zero. Early layers receive almost no gradient signal and barely update. This is the **vanishing gradient problem**.

Sigmoid and Tanh

$$\sigma(x) = \frac{1}{1+e^{-x}} \in (0,1), \qquad \tanh(x) = \frac{2}{1+e^{-2x}} - 1 \in (-1,1)$$

Both are smooth and differentiable everywhere, and were the standard choice in early MLPs. However, they **saturate** for $|x| \gg 0$: the gradient $\sigma'(x) = \sigma(x)(1-\sigma(x))$ approaches zero, causing the vanishing gradient problem in deep networks.
Where still used: output layer — sigmoid for binary classification (outputs a probability in $(0,1)$), softmax for multiclass.

The Dying ReLU Problem

ReLU solves saturation for positive inputs — its gradient is exactly 1 for all $x > 0$, so it does not attenuate gradients in the positive regime. However, it introduces a different problem.

For any input $x < 0$, ReLU outputs zero and its gradient is also zero. If a neuron's pre-activation is consistently negative (e.g. due to a large negative bias or an unfortunate weight initialisation), it will output zero on *every* training example. Since the gradient is zero, no update is ever applied to its weights — the neuron is permanently "off" and contributes nothing to the network. This is the **dying ReLU problem**.

Once a ReLU neuron dies, it cannot recover by itself: because the output and gradient are both zero, gradient descent has no signal to move the weights. In practice, a fraction of neurons in a deep network can die during training, reducing effective capacity.

ReLU (Rectified Linear Unit)

$$\mathrm{ReLU}(x) = \max(0,\, x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Advantages: computationally trivial; gradient is exactly 1 for $x > 0$ (no vanishing for positive activations); promotes sparse activations (exactly zero for $x < 0$).
Dying ReLU: for $x < 0$, both output and gradient are zero. A neuron that always receives negative pre-activations is permanently silenced and never updated.
Mitigation: careful weight initialisation (He initialisation), small positive bias initialisation, or switching to Leaky ReLU.

Leaky ReLU

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & x \geq 0 \\ \alpha\, x & x < 0 \end{cases} \qquad \alpha = 0.01 \text{ (typical)}$$

Fixes the dying ReLU problem by allowing a small, non-zero gradient α for negative inputs. The neuron is never completely silenced, so it can always recover. The cost is a slight loss of sparsity.

Which activation for which layer?

Hidden layers: ReLU (fast, no vanishing gradient for $x > 0$) or Leaky ReLU (no dying neurons). **Binary output:** Sigmoid. **Multiclass output:** Softmax. **Regression output:** linear (no activation). Sigmoid and tanh are largely deprecated in hidden layers of deep networks.

10.6 Pooling Layers

Pooling Layer

Pooling layers reduce the *spatial* size of the volume while preserving (or increasing) the depth. They operate independently on each channel and apply a fixed aggregation function over a local patch.
Max-pooling (most common): takes the maximum value in each patch. With a 2×2 patch and stride 2, it reduces each spatial dimension by half (discards 75% of values).
Average-pooling: takes the mean (used in LeNet-5, less common in modern nets).

Backpropagation Through Max-Pooling

During the backward pass, the gradient is routed *only* through the input element that achieved the maximum in the forward pass. All other positions receive zero gradient. Formally, the derivative is 1 at the argmax position and 0 elsewhere.

Stride in Pooling

Typically the stride equals the pooling window size (e.g. 2×2 pool, stride 2), ensuring non-overlapping regions and halving spatial size. Using stride 1 with max-pooling performs a nonlinear operation without spatial reduction — sometimes useful as a local nonlinearity.

10.7 Dense (Fully-Connected) Layers and Flattening

After the convolutional blocks, the spatial volume must be converted to a 1D vector before feeding into standard MLP layers. This is done by a **Flatten** layer, which unrolls all spatial and channel dimensions into a single vector. From this point, the network is a standard MLP: each output neuron is connected to *every* input neuron.

The final dense layer has as many neurons as there are output classes (L), followed by a Softmax for multiclass or Sigmoid for binary classification.

Flattening vs. Global Average Pooling

Flattening preserves all spatial information (large vector, many parameters). Global Average Pooling (GAP) compresses each channel to a single scalar, drastically reducing parameters. GAP is preferred in modern architectures (see Section 11.4).

Why an MLP specifically — not an SVM or random forest?

This is a common exam question. The CNN backbone (conv + pool layers) extracts features; the classifier head maps those features to class scores. In principle, *any* classifier could sit after the flattening step — an SVM, a random forest, a k-NN. However, the reason an **MLP** (dense layers) is used exclusively is **end-to-end training via backpropagation**.
Backpropagation requires every operation in the pipeline to be *differentiable* with respect to its inputs and parameters, so that gradients can flow from the loss back through the classifier and into the conv layers. An MLP satisfies this: its operations (matrix multiply, bias add, activation) are all differentiable.
An SVM, random forest, or k-NN are **not differentiable** with respect to the feature input in a way that allows gradient-based joint optimisation. If you replace the MLP head with an SVM, the conv layers receive no gradient signal and their weights cannot be updated — you would have to train the backbone and the SVM separately, losing the key advantage of end-to-end learning.
Summary: MLP = differentiable ⇒ gradients flow through it ⇒ the whole network trains jointly. Any non-differentiable classifier breaks this chain.

10.8 Sparse and Shared Weights: CNNs vs. MLPs

Convolution can be written as a matrix-vector multiplication $\mathbf{a} = W\mathbf{x} + \mathbf{b}$, just like a dense layer. The fundamental difference lies in the *structure* of W:

Dense Layer vs. Convolutional Layer

Property	Dense (FC) Layer	Conv Layer
Weight matrix	Dense (all entries free)	Sparse + block-circulant
Weight sharing	None	Same filter at all positions
Bias	One per output neuron	One per filter (shared)
Inductive bias	None	Translation equivariance
Parameters (example)	$(32\times32)\times(28\times28\times6) = 4.8\text{M}$	$25\times6+6 = 156$

Translation Equivariance (Inductive Bias)

In a CNN, all neurons in the same output channel share identical filter weights. The underlying assumption is: *if a feature is useful to detect at one spatial location, it is equally useful at all other locations.* This is a powerful prior for images, where objects can appear anywhere in the frame. As a result, CNNs generalise better from limited data on spatially structured inputs.

Why CNNs are not equivariant to scale or rotation

Weight sharing gives CNNs translation equivariance "for free." However, they do *not* natively handle scale changes or rotations — unless the training data includes examples at all relevant scales and orientations, or data augmentation is used. This is a key limitation motivating data augmentation (Section 10.14.2).

10.9 The Receptive Field

Receptive Field

The **receptive field** of a neuron in layer l is the set of input pixels that can influence its value. Due to sparse connectivity, each output neuron only depends on a limited region of the input (unlike a dense layer, where every output depends on all inputs).
Key rule: as depth increases, the receptive field grows. Each additional conv layer of size 3×3 adds 2 pixels on each side to the receptive field of the next layer.

Receptive Field Growth

For n stacked 3×3 conv layers (stride 1, no pooling), the receptive field is:

$$\text{RF} = 1+2n \quad (\text{so } n=6 \Rightarrow \text{RF} = 13\times13)$$

Max-pooling with stride 2 doubles the effective receptive field of all subsequent layers. After $n=6$ layers with 3 interleaved 2×2 max-pools:

$$\text{RF} = 22\times22 \text{ (roughly)}$$

Exam: compute the receptive field

A very common exam question. To compute the receptive field of a neuron at a given layer:

1. Work backward from the target neuron.
2. Each conv $k\times k$ (stride 1) adds $k-1$ pixels in each direction.

3. Each max-pool 2×2 (stride 2) doubles the current receptive field size.
4. Report the size as a single number (side length of the square receptive field).

Deeper = wider receptive field = higher-level features

As we go deeper in a CNN, each neuron sees a larger patch of the input. Early layers detect fine-grained, low-level patterns (edges, textures) over small regions. Deeper layers combine these into increasingly abstract, high-level features (object parts, whole objects). Simultaneously, the number of channels grows, encoding richer feature vocabularies.

10.10 CNN Training

A CNN is mathematically equivalent to an MLP with a sparse, shared weight matrix. Therefore:

- Training proceeds by **gradient descent** minimising a loss function (cross-entropy for classification, MSE for regression).
- Gradients are computed by **backpropagation** (chain rule), applied through each differentiable layer.
- **Weight sharing** is respected: the gradient for a shared weight is the *sum* of gradients from all positions where that filter is applied.

Backpropagation Details for CNN-Specific Layers

Max-pooling: gradient flows only to the input element that achieved the maximum (argmax). Switch variables (storing which element was maximum during the forward pass) are used to route the backward gradient correctly. All other positions receive gradient 0.

ReLU: $\frac{d}{dx}\text{ReLU}(x) = \mathbf{1}[x > 0]$. Gradient passes through unchanged for positive activations, is zeroed for negative ones.

10.11 The Latent Representation

The output of the FEN (just before the final dense layers) is called the **latent representation** or **feature vector** of the image. This vector encodes the image in a semantically rich space.

Latent space distances are more meaningful than pixel distances

Two images can be pixel-far (e.g. same cat in different lighting) but close in the latent space of a well-trained CNN. Conversely, two noise-free images of different classes will be far apart. This property makes the latent space ideal for: classification (MLP on top), image retrieval (nearest neighbour search), and transfer learning.

Image Retrieval in the Latent Space

A powerful application of the learned latent representation is **content-based image retrieval**: given a query image, find the most visually similar images from a database by computing nearest neighbours in the latent space.

```
1 # Feed the query image to the feature extraction network
```

```
qImg_features = fen(query_image)            # shape: (feature_dim,)

# Feed entire training set through FEN (use batches for efficiency)
TR_features = fen(X_train_val).cpu().numpy()   # shape: (N, feature_dim)

# Compute L1 distances between query and all training features
distances = np.mean(np.abs(qImg_features - TR_features), axis=-1)

# Sort by ascending distance
sorted_idx = distances.argsort()

# Retrieve the closest image and its label
img_retrieved  = X_train_val[sorted_idx[0]]
label_retrieved = y_train_val[sorted_idx[0]]
```

Code 1: 1-NN image retrieval using CNN latent representation (PyTorch)

10.12 LeNet-5: The First CNN (1998)

LeNet-5, introduced by LeCun et al. in 1998, is the first successful deep CNN and a landmark in the history of deep learning. It was designed for handwritten digit recognition (MNIST: 28×28 grayscale images, 10 classes).

LeNet-5 Architecture

Input: $32 \times 32 \times 1$ (grayscale, zero-padded from 28×28).
Stack: Conv2D(6, 5×5, tanh) → AvgPool(2×2) → Conv2D(16, 5×5, tanh) → AvgPool(2×2) → Flatten → Dense(120, relu) → Dense(84, relu) → Dense(10, softmax).
Total parameters: 61,706.

```
from keras.models import Sequential
from keras.layers import Dense, Flatten, Conv2D, AveragePooling2D

model = Sequential([
    Conv2D(filters=6, kernel_size=(5,5), activation='tanh',
           input_shape=(32,32,1), padding='valid'),
    AveragePooling2D(pool_size=(2,2)),
    Conv2D(filters=16, kernel_size=(5,5), activation='tanh',
           padding='valid'),
    AveragePooling2D(pool_size=(2,2)),
    Flatten(),
    Dense(120, activation='relu'),
    Dense(84,  activation='relu'),
    Dense(10,  activation='softmax'),
])
```

Code 2: LeNet-5 in Keras

Parameter Count Analysis

Layer-by-layer parameter breakdown:

Layer	Formula	Params
Conv2D(6, 5×5) on grayscale	$6 \times 5 \times 5 \times 1 + 6$	156
AvgPool	–	0
Conv2D(16, 5×5) on 6ch	$16 \times 5 \times 5 \times 6 + 16$	2,416
AvgPool	–	0
Dense(120) on $400 = 5 \times 5 \times 16$	$400 \times 120 + 120$	48,120
Dense(84)	$120 \times 84 + 84$	10,164
Dense(10)	$84 \times 10 + 10$	850
Total		**61,706**

Most parameters live in the dense layers

In LeNet-5, the two FC layers account for $\approx 95\%$ of parameters, while the convolutional layers account for only $\sim$ 2.6K of 61K. This illustrates a general principle: convolutional layers are parameter-efficient (weight sharing), while dense layers are parameter-heavy. Modern architectures reduce FC parameters by replacing flattening with Global Average Pooling.

CNNs vs. flat MLPs on images

An MLP taking a 32×32 grayscale image directly needs $1024 \times 84 + \cdots$ weights at the first layer alone. For RGB ($32 \times 32 \times 3$), the MLP's first-layer parameters triple, while the CNN's first-layer parameters only increase by a factor of 3 ($6 \times 5 \times 5 \times 1 \to 6 \times 5 \times 5 \times 3$, i.e. from 150 to 450). The gap in parameter efficiency widens dramatically for larger images.

Why "valid" padding at the first LeNet layer?

The input images are MNIST digits padded to 32×32 — the boundary pixels are black (zero). Using `valid` padding (no zero-padding) at the first conv layer does not lose any meaningful information, and conveniently reduces the spatial dimension, limiting the size of the latent representation downstream.

10.13 CNN Anatomy: Parameters, Weights, and Inductive Bias

Convolution as Matrix Multiplication

Because convolution is a linear operation $\mathbf{a} = W\mathbf{x} + \mathbf{b}$ over flattened input vectors. The weight matrix W for a conv layer has two special structures:

1. **Sparsity:** most entries of W are zero. Each output neuron only depends on the neurons in its local receptive field, not all input neurons.
2. **Shared weights (block structure):** the same filter weights repeat across rows of W (shifted), because the same filter is applied at every spatial position.

Conversely, a fully-connected layer *can* be interpreted as a convolutional layer with a spatial kernel equal to the full input size and no weight sharing (each position uses a different filter). This equivalence is exploited in fully-convolutional networks.

Why Sparse + Shared Weights Work

Parameter efficiency example (LeNet, first layer):

Architecture	Params (first layer only)
MLP on $32 \times 32 \times 3$	$(32 \times 32 \times 3) \times N_{\text{hidden}} = 3072 \times N$
CNN Conv$(6, 5 \times 5)$ on $32 \times 32 \times 3$	$6 \times 5 \times 5 \times 3 + 6 = 456$

The saving is the key to training CNNs with feasible data and compute. The inductive bias (translation equivariance) is not only a computational trick — it genuinely matches the structure of visual data.

10.14 Data Scarcity: Data Augmentation and Transfer Learning

Why Deep Models Need Large Datasets

Deep CNNs (e.g. AlexNet with 60M parameters) are only trainable when enough labelled data is available. AlexNet was trained on **ImageNet**: over 14 million hand-annotated images, 20,000 categories. Two additional enablers: GPU parallelism (NVIDIA GTX 580) and software frameworks (TensorFlow, PyTorch).

In many real applications, annotated data is scarce. The two main solutions are:

- **Data Augmentation:** artificially expand the training set by generating new labelled examples from existing ones.
- **Transfer Learning:** leverage features learned on a large source dataset (e.g. ImageNet) for a different target task.

Data Augmentation

Data Augmentation

Given a labelled image (I, y) and a set of label-preserving transformations $\{A_l\}_l$, augmentation expands the training set by adding all $(A_l(I), y)$ pairs. This:

1. Increases effective dataset size, reducing overfitting.
2. Encourages the network to become *invariant* to those transformations.
3. Can address class imbalance by over-sampling minority classes.

Types of Augmentation Transformations

Geometric Transformations

- Random horizontal / vertical flip
- Random rotation (e.g. ± 72)
- Random affine / perspective distortion, shear
- Random crop / translation
- Random scaling

Photometric Transformations

- Gaussian noise injection
- Brightness / contrast / saturation jitter (`ColorJitter`)
- Intensity shift / histogram manipulation
- Image superimposition

Critical: augmentation must preserve the label

Not all transformations are valid for all tasks. Example: training a network to read clock times — rotating the image by 90° changes the time displayed, so rotation would corrupt the label. Always verify that the transformation preserves the discriminative information for your specific task.

Hidden augmentation artefacts

If augmentation introduces systematic traces that correlate with class labels (e.g. blurring only images of one class, adding padding artefacts, changing colour statistics differently per class), the network will learn those artefacts as discriminative features rather than true semantic content. This causes brittle models that fail at test time without the artefact.

Mixup Augmentation

Mixup (Zhang et al., ICLR 2018) is a domain-agnostic augmentation technique that does not require specifying which geometric/photometric transformations are label-preserving:

Mixup

Given two training samples (I_i, y_i) and (I_j, y_j) (possibly from different classes), and $\lambda \sim \text{Beta}(\alpha, \alpha)$ with $\lambda \in [0, 1]$, define:

$$\tilde{I} = \lambda I_i + (1 - \lambda) I_j, \qquad \tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

where y_i, y_j are one-hot encoded class vectors. The virtual sample $(\tilde{I}, \tilde{y})$ is used as a training example. The loss (categorical cross-entropy) is computed against the soft label $\tilde{y}$.

Mixup intuition

Mixup extends the training distribution by encoding the prior belief that *linear interpolations of feature vectors should correspond to linear interpolations of their labels*. This acts as a regulariser, discourages memorisation, and promotes smoother decision boundaries. It adds minimal

computational overhead and requires no domain knowledge.

Test-Time Augmentation (TTA)

Even a perfectly augmented training regime does not achieve perfect invariance. **Test-Time Augmentation** (TTA) improves prediction accuracy at inference by:

1. Generating multiple augmented versions $\{A_l(I)\}_l$ of the test image.
2. Running each through the CNN to get posterior vectors $\{p_l\}_l$.
3. Aggregating predictions: $\hat{p} = \text{Avg}(\{p_l\})$.

TTA trade-offs

TTA can significantly improve accuracy, especially for uncertain predictions, but requires n_{aug} forward passes instead of one. Class-specific augmentations cannot be used at test time (since the class is unknown). Best suited for high-stakes inference tasks where compute is not constrained.

Augmentation in PyTorch

PyTorch implements augmentation *within the DataLoader*, applying random transformations on-the-fly at each batch. This means the network never sees the exact same augmented image twice across epochs, and no augmented images need to be stored on disk.

```python
from torchvision import transforms

# Augmentation pipeline (applied only during training)
augmentation = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.RandomVerticalFlip(p=0.5),
    transforms.RandomAffine(degrees=0, translate=(0.2, 0.2)),
    transforms.RandomRotation(degrees=72),
    transforms.ColorJitter(brightness=0.5, contrast=0.75),
])

# Preprocessing only (applied at both train and test time)
preprocess = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.CenterCrop(224),
    transforms.Grayscale(),    # if needed
])
```

Code 3: Stacking augmentation transforms in PyTorch

Transfer Learning

Transfer Learning

Transfer learning reuses a CNN pre-trained on a large source dataset (typically ImageNet) as a feature extractor for a new, smaller target dataset. The intuition: the convolutional layers of a well-trained FEN learn general-purpose visual features (edges, textures, shapes) that are useful for *any* visual task, not just the source classification problem.

Standard Transfer Learning Protocol

Transfer Learning Steps

1. **Take** a powerful pre-trained CNN (e.g. ResNet, EfficientNet, MobileNet, VGG16).
2. **Remove** the original task-specific FC layers (the "head").
3. **Design** new FC layers matching the target task (L_{new} output neurons).
4. **Freeze** the FEN weights (`requires_grad = False` for all FEN parameters). The FEN acts as a fixed feature extractor.
5. **Train** only the new head on the (small) target training set.

Transfer Learning vs. Fine-Tuning

Method	What is trained	When to use
Transfer Learning	New head only (FEN frozen)	Little data; source domain matches target
Fine-Tuning	Whole network (FEN init. from pretrained)	More data available; source/target domains differ

Best practice: TL then fine-tune

A practical strategy: first train only the new head (TL) until it converges, then unfreeze the entire network and fine-tune with a *lower learning rate* than used for the head. This avoids corrupting the pretrained FEN weights with noisy gradients from the randomly-initialised head.

Transfer Learning in PyTorch

```
import torchvision.models as models
import torch.nn as nn

# Load pretrained model (weights from ImageNet)
model = models.resnet50(pretrained=True)

# Extract the feature extraction network (everything before the head)
fen = model  # for ResNet, the head is model.fc

# Freeze all FEN parameters
for p in model.parameters():
    p.requires_grad = False

# Replace the classifier head for the new task (e.g., 6 classes)
num_classes = 6
model.fc = nn.Linear(model.fc.in_features, num_classes)
# Only model.fc parameters have requires_grad=True

# Train: only model.fc is updated
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

Code 4: Transfer learning with a pretrained model in PyTorch

Each pretrained model expects specific preprocessing

Every pretrained model was trained with a specific input normalisation (mean, std of ImageNet). Always apply the same preprocessing at inference. In `torchvision`, the `weights.transforms()` method returns the correct preprocessing pipeline.

Performance Metrics: Confusion Matrix and ROC

Confusion Matrix

For a K-class problem, the confusion matrix $C \in \mathbb{R}^{K \times K}$ has entry $C(i, j)$ equal to the *fraction of samples from class i that were classified as class j*. The ideal confusion matrix is the identity (all diagonal, all off-diagonal zero).

ROC Curve and AUC (Binary Classification)

In binary classification, the CNN output $p = \text{CNN}(I) \in (0, 1)$ is thresholded at Γ: predict class 1 if $p > \Gamma$. Varying Γ traces the **Receiver Operating Characteristic (ROC)** curve, plotting True Positive Rate (TPR = sensitivity) against False Positive Rate (FPR = 1-specificity).
The **Area Under the Curve (AUC)** summarises performance across all thresholds: AUC = 1 is perfect; AUC = 0.5 is random. The optimal operating point is the Γ giving the point on the ROC curve closest to $(0, 1)$.

Why use AUC instead of accuracy?

Accuracy depends on the threshold Γ and is misleading for imbalanced datasets (a classifier predicting the majority class always gets high accuracy). AUC is threshold-independent and measures the probability that a randomly chosen positive example scores higher than a randomly chosen negative example — a true measure of ranking quality.

11 Famous CNN Architectures

The history of deep learning for vision is largely the history of a handful of landmark architectures, each introducing a single powerful idea that unlocked a new level of performance. Understanding them in chronological order reveals a coherent design narrative: from simple stacks of layers (AlexNet, VGG), to modular multi-scale blocks (GoogLeNet), to the insight that depth itself can be a problem and needs to be engineered around (ResNet), to the modern goal of maximising accuracy per floating-point operation (MobileNet, EfficientNet).

11.1 AlexNet (2012): The Deep Learning Revolution

AlexNet (Krizhevsky, Sutskever, Hinton, NIPS 2012) won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2012 with a top-5 error of 15.4% — roughly 10 percentage points better than the second-place entry, which still used hand-crafted features. This gap shocked the computer vision community and triggered an almost immediate switch to deep CNNs across the entire field.

AlexNet Architecture

Input: $227 \times 227 \times 3$ (RGB; the paper states 224×224 but the actual forward pass uses 227×227).

Layer	Type	Output size	Details	Params
Conv1	Conv 11×11, s4	$55 \times 55 \times 96$	96 filters, stride 4	35K
	MaxPool 3×3, s2	$27 \times 27 \times 96$		
Conv2	Conv 5×5, pad2	$27 \times 27 \times 256$	256 filters	614K
	MaxPool 3×3, s2	$13 \times 13 \times 256$		
Conv3	Conv 3×3, pad1	$13 \times 13 \times 384$	384 filters	885K
Conv4	Conv 3×3, pad1	$13 \times 13 \times 384$	384 filters	1.33M
Conv5	Conv 3×3, pad1	$13 \times 13 \times 256$	256 filters	885K
	MaxPool 3×3, s2	$6 \times 6 \times 256$		
FC6	Fully Connected	4096	Dropout 0.5	37.7M
FC7	Fully Connected	4096	Dropout 0.5	16.8M
FC8	Fully Connected	1000	Softmax	4.1M
Total				≈60M

Conv layers hold only ≈6% of parameters; the three FC layers hold ≈94%.

Key innovations introduced by AlexNet:

- **ReLU activations.** Using ReLU instead of sigmoid/tanh accelerates training by ∼6× on CIFAR-10 (no vanishing gradient for positive inputs; gradients do not shrink through the non-linearity).
- **Dropout (0.5) in FC layers.** Prevents co-adaptation of neurons; acts as implicit ensemble training. At test time all neurons are active and outputs are scaled by 0.5.
- **Overlapping max-pooling.** Pooling windows of 3×3 with stride 2 (instead of non-overlapping stride-equal-to-kernel) give a slight accuracy boost and reduce overfitting.
- **Local Response Normalisation (LRN).** After Conv1 and Conv2, activations are normalised across neighbouring channels:

$$b^i_{x,y} = a^i_{x,y} \Big/ \left(k + \alpha \sum_{j=\max(0,i-n/2)}^{\min(N-1,i+n/2)} (a^j_{x,y})^2\right)^\beta$$

  This implements a form of lateral inhibition inspired by biology. **Note:** LRN was largely abandoned in later architectures (VGG showed it provided no clear benefit) and replaced by Batch Normalisation.
- **Data augmentation.** Random 224×224 crops and horizontal flips from 256×256 images; PCA colour jitter. Reduced top-5 error by over 1%.
- **Two-GPU training.** A single GTX 580 (3 GB VRAM) could not fit the full model. The network was split across two GPUs: each GPU holds half the channels in each conv layer, and the two halves communicate only at specific layers (Conv3 and all FC layers). This was an engineering necessity, not an architectural choice — modern GPUs make it obsolete.
- **Ensemble of 7 models:** reduces top-5 error from 18.2% to 15.4%.

AlexNet's historical significance

AlexNet did not invent CNNs (LeNet-5 is from 1998) nor most of its individual components. Its contribution was demonstrating — definitively and at scale — that deep CNNs trained on GPUs with large datasets outperform hand-crafted feature pipelines by a margin too large to ignore. It triggered a paradigm shift: within 2–3 years, virtually all competitive vision systems switched to deep CNNs. The key enabling factors were: ImageNet (large labelled dataset), ReLU (stable deep training), and GPUs (10×–50× speedup over CPUs for matrix operations).

11.2 VGG (2014): Going Deeper with Small Filters

AlexNet showed that depth matters, but used large filters (11×11, 5×5) in early layers. **VGGNet** (Simonyan & Zisserman, ICLR 2015) asked: *what if we use* only 3×3 *filters, everywhere, and just go deeper?* The answer turned out to be: better accuracy with a cleaner, more principled design. VGG16 (16 weight layers) and VGG19 (19 weight layers) won 1st place (localisation) and 2nd place (classification) at ILSVRC 2014.

VGG16 Architecture

Input: $224 \times 224 \times 3$.
Structure: 13 convolutional layers (all 3×3, stride 1, padding 1) grouped in 5 blocks, each block followed by 2×2 max-pooling (stride 2). Three final FC layers (4096, 4096, 1000 neurons).

Block	Layers	Output size
Block 1	2× Conv 3×3 (64 filters) + MaxPool	$112 \times 112 \times 64$
Block 2	2× Conv 3×3 (128) + MaxPool	$56 \times 56 \times 128$
Block 3	3× Conv 3×3 (256) + MaxPool	$28 \times 28 \times 256$
Block 4	3× Conv 3×3 (512) + MaxPool	$14 \times 14 \times 512$
Block 5	3× Conv 3×3 (512) + MaxPool	$7 \times 7 \times 512$
FC6	Dense 4096 (+ Dropout 0.5)	4096
FC7	Dense 4096 (+ Dropout 0.5)	4096
FC8	Dense 1000 + Softmax	1000

Parameters: ∼138M total. Conv layers: ≈15M (11%); FC layers: ≈123M (89%), of which FC6 alone contributes $7 \times 7 \times 512 \times 4096 \approx 102$M — the single most expensive layer.
Memory: ∼100 MB per image for forward activations; ∼200 MB during training.

Why Small Filters?

Three 3×3 convolutions vs. one 7×7 convolution (single input channel, C output channels):

Property	3× Conv(3×3, C ch.)	1× Conv(7×7, C ch.)
Receptive field on input	7×7	7×7
Parameters (no bias)	$3 \times (3^2 \cdot C^2) = 27C^2$	$7^2 \cdot C^2 = 49C^2$
Nonlinearities applied	3	1

Same receptive field. 45% fewer parameters. 3× more nonlinearities.

The deep-and-narrow principle

VGG demonstrates that *depth is more efficient than large filter size.* Multiple stacked small filters achieve the same receptive field as a single large filter while requiring fewer parameters and introducing more nonlinearity per parameter. This principle guides virtually all modern architecture design: from GoogLeNet's 3×3 and 5×5 branches, to ResNet's 3×3 bottleneck blocks, to MobileNet's depthwise 3×3 filters.

VGG limitations

VGG's 138M parameters are almost entirely in the three FC layers — not in the conv layers that do the actual feature extraction. This makes it very memory-hungry and slow at inference. In practice, VGG is superseded by ResNet and EfficientNet, which achieve better accuracy with a fraction of the parameters. The main reason VGG is still taught and used is its conceptual simplicity: uniform 3×3 convolutions throughout, making it easy to reason about receptive fields and layer outputs.

11.3 CNN Visualisation

Why Visualise?

Training a CNN produces millions of numerical weights — but *what has the network actually learned?* Visualisation techniques try to answer this question. The motivations are both scientific and practical:

- **Interpretability and trust.** Before deploying a model in a safety-critical application (medical imaging, autonomous driving), one needs evidence that it is using *meaningful* features rather than dataset artefacts.
- **Debugging.** If a network performs poorly, visualisation can reveal whether the issue is in early feature extraction or in high-level reasoning.
- **Detecting "Clever Hans" behaviour.** A famous failure mode: a horse-classification network achieved near-perfect accuracy because training images of horses happened to contain a watermark from one photographic agency. The network had learned to recognise the watermark, not the horse. Saliency-based visualisation revealed this. Without it, the bug would have remained invisible.
- **Scientific insight.** Visualisations reveal that CNNs trained on natural images spontaneously develop detectors reminiscent of neurons in the mammalian visual cortex — providing a productive dialogue between deep learning and computational neuroscience.

There are three complementary families of visualisation methods, ordered from simplest to most powerful: (1) **filter visualisation** (look directly at the learned weights of the first layer); (2) **maximally activating patches** (find real images that most excite a given neuron); (3) **gradient ascent / synthetic image generation** (synthesise an image from scratch that maximally activates a neuron or class score).

Visualising First-Layer Filters

The weights of the *first* convolutional layer are the only weights that can be visualised directly as images: they are small (3×3 to 11×11) tensors with 3 channels (RGB), so they can be rendered as colour patches.

Template matching interpretation

A convolutional filter $\mathbf{w}$ applied to image I produces a large activation at position (r, c) when the local patch of I centred at (r, c) has a high dot product with $\mathbf{w}$. Since the dot product measures similarity, the filter is precisely the *template* the neuron responds most strongly to. Visualising the filter = visualising the optimal stimulus for that neuron.

What do first-layer filters look like? In AlexNet (trained on ImageNet), the 96 first-layer filters split naturally into two groups:

- **Oriented edge detectors** (Gabor-like): respond to edges at a specific orientation and spatial frequency, analogous to *simple cells* in primary visual cortex (V1).
- **Colour blob detectors**: respond to patches of a specific colour (e.g. red-green or blue-yellow opponent channels), analogous to *colour-selective* V1 neurons.

This parallel with neuroscience is not coincidental: both CNNs and the visual cortex are optimising for efficient coding of natural image statistics.

Why only the first layer?

For layer $\ell > 1$, filters operate on the *feature maps* of the previous layer — abstract volumes with many channels that are not RGB images. Rendering such filters as images is meaningless. Different methods (maximally activating patches, gradient ascent) are needed for deeper layers.

Maximally Activating Patches

The idea is simple: rather than looking at the filter weights directly, we ask *"which real images produce the largest response at a given neuron?"* and then display the corresponding *input patch* (the region of the input image that falls within the neuron's receptive field).

Algorithm: Maximally Activating Patches

1. Choose a target neuron at position (r_0, c_0) in layer ℓ.
2. Forward-pass the entire dataset (or a large subset) through the network.
3. Record the activation of the target neuron for every image.
4. Sort images in descending order of activation.
5. For each top-k image, crop the input region corresponding to the neuron's receptive field and display it.

What the hierarchy looks like in practice (from Zeiler & Fergus, ECCV 2014):

Layer	Receptive field size	What neurons respond to
Layer 1	Very small ($\sim 11 \times 11$)	Oriented edges, colour gradients
Layer 2	Moderate	Corners, junctions, simple textures
Layer 3	Larger	Textures, repeated patterns (grids, fur)
Layer 4	Even larger	Object parts (dog faces, wheels, text)
Layer 5	Large	Entire objects, semantic categories

The hierarchy is real, not assumed

The layered structure in a CNN is an *architectural prior* — but the learned hierarchy of low-to-high-level features emerges from data alone. Maximally activating patch visualisations provide direct empirical evidence that the network actually uses this hierarchy rather than collapsing it.

Limitation of the method. Maximally activating patches are restricted to real images in the dataset. The neuron may respond strongly to many different patterns that happen to co-occur in the top-k images, making it hard to isolate the single feature the neuron is truly detecting. Gradient ascent (below) overcomes this.

Gradient Ascent: Synthesising Maximally Activating Inputs

Instead of searching over real images, gradient ascent *synthesises* an image from scratch by back-propagating not to the weights but to the **input pixels**. This gives the purest possible description of a neuron's preferred stimulus.

Setup. Fix the network weights $\boldsymbol{\theta}$ (no weight update). Let $S_c(\mathbf{I})$ be the score (pre-softmax logit) for class c, or the activation of any chosen neuron, viewed as a function of the input image $\mathbf{I}$. We want to find the image that maximises this score.

Gradient Ascent for Neuron Visualisation

$$\hat{\mathbf{I}} = \arg\max_{\mathbf{I}} \; S_c(\mathbf{I}) - \lambda \|\mathbf{I}\|_2^2$$

The optimisation is performed by **gradient ascent** on the pixel values:

$$\mathbf{I}^{(t+1)} = \mathbf{I}^{(t)} + \alpha \, \nabla_{\mathbf{I}} S_c\Big(\mathbf{I}^{(t)}\Big)$$

starting from $\mathbf{I}^{(0)} = \mathbf{0}$ (or small random noise). The gradient $\nabla_{\mathbf{I}} S_c$ is computed via backpropagation through the *fixed* network. The regulariser $\lambda\|\mathbf{I}\|_2^2$ penalises images with large pixel values, biasing the solution towards low-energy, visually interpretable patterns.

Sign of the regulariser — common exam trap

The ℓ_2 term is **subtracted** in the objective (i.e. it acts as a penalty on large pixel magnitudes). Writing it with a + sign is wrong: that would *reward* large pixel values and produce degenerate white images. Always read the objective as "maximise score, penalise pixel energy."

What gradient ascent produces.

- **Early-layer neurons**: the synthesised images are simple oriented gratings or colour patches — consistent with Gabor filters.
- **Intermediate-layer neurons**: the images show repeated textural patterns (fur-like, grid-like, scaly), but no coherent global structure.
- **Deep-layer / class neurons**: the images start to resemble recognisable objects (eyes, beaks, wheels), though they often look psychedelic due to the absence of natural image constraints.

Hierarchical feature learning — confirmed

Gradient ascent visualisations confirm the hierarchy end-to-end: what a deep CNN computes is a hierarchy of template matchers, each layer detecting increasingly abstract and translation-invariant patterns. The fact that class-level neurons synthesise object-like images (without ever

being told what an object looks like) is strong evidence that the network has built an internal model of visual categories.

DeepDream (optional enrichment). Google's DeepDream applies gradient ascent not to synthesise an image from scratch, but to *amplify* the features a network already detects in a real photograph: the gradient of a layer's ℓ_2-norm (rather than a single neuron) is used to update the image. The result is the well-known hallucinatory imagery where the network exaggerates patterns it finds in the image (dog faces everywhere, spiralling textures). DeepDream is a creative application of the same gradient-ascent idea, with no regulariser restricting the output to look natural.

t-SNE Visualisation of Activations

A complementary diagnostic is to visualise not individual neurons but the *distribution of the entire representation space*. One takes the activations of the penultimate layer (the feature vector just before the classifier) for a large set of images and projects them to 2D using **t-SNE** (t-distributed Stochastic Neighbour Embedding), a nonlinear dimensionality reduction method.

What t-SNE of CNN activations reveals.

- Images from the same semantic class cluster tightly together.
- Visually similar classes (e.g. different dog breeds) form nearby clusters.
- The 2D map often reveals a smooth semantic topology: cars near trucks, dogs near cats, far from buildings.

This contrasts sharply with t-SNE applied to *raw pixels* (as seen in Section 10), where classes are hopelessly intermixed. The CNN has transformed the pixel space into a semantically organised embedding space — which is precisely what makes the linear classifier on top work well.

t-SNE as a sanity check

If t-SNE of CNN activations shows well-separated clusters, the network's representations are already "almost linearly separable," and the final FC classifier just needs to draw hyperplanes. Poor clustering in t-SNE is a red flag: it suggests the backbone is not learning discriminative features, and one should inspect the training data, loss, or architecture.

Fooling Images and Adversarial Examples (Outlook)

Gradient ascent can be pushed to a disturbing extreme. Starting from a *real* image of, say, a dog, one can iteratively perturb the pixels to maximise the score for a completely different class (e.g. "ostrich"), subject to the perturbation being imperceptibly small in ℓ_∞ or ℓ_2 norm. The resulting image looks identical to the original dog photo to a human, yet the network predicts "ostrich" with 99% confidence.

These are called **adversarial examples** (Szegedy et al., 2013; Goodfellow et al., 2014). Their existence reveals that CNN decision boundaries, while highly accurate on natural images, are thin and fragile in directions that are perceptually irrelevant. This has important implications for robustness and security. Defences (adversarial training, certified robustness) are an active research area and will appear again in the context of robustness.

Fooling images vs. adversarial examples

Fooling images start from noise and are optimised to maximally activate a class — the result looks like noise but is classified with high confidence. **Adversarial examples** start from a real, correctly classified image and add a small, imperceptible perturbation that flips the prediction. Both exploit the same gradient-ascent mechanism, but adversarial examples are more practically dangerous because they are indistinguishable from legitimate inputs.

Recap: CNN Visualisation

CNN Visualisation — Summary (Exam Checklist)

1. **Why visualise?** Interpretability, debugging, Clever Hans detection, neuroscience validation.
2. **First-layer filter visualisation**: directly render 3-channel filter weights as images; AlexNet filters = Gabor edge detectors + colour blobs; deeper filters cannot be visualised this way.
3. **Template matching**: a filter is the template the neuron responds maximally to (high dot product = high activation).
4. **Maximally activating patches**: forward-pass dataset, rank by neuron activation, display input patches of the top images; reveals the feature hierarchy: edges → textures → parts → objects.
5. **Gradient ascent**: fix weights, update *pixels* via $\mathbf{I} \leftarrow \mathbf{I} + \alpha \nabla_{\mathbf{I}} S_c(\mathbf{I})$; objective $= S_c(\mathbf{I}) - \lambda \|\mathbf{I}\|_2^2$ (regulariser **subtracted**); synthesises the purest preferred stimulus; confirms the hierarchy.
6. **t-SNE of activations**: project penultimate-layer features to 2D; well-separated clusters = good representations; contrast with raw-pixel t-SNE (intermixed classes).
7. **Fooling images**: noise optimised to maximally activate a class; classified with high confidence; demonstrates that CNNs are not robust outside the natural image manifold.
8. **Adversarial examples**: real image + imperceptible perturbation ⇒ wrong prediction with high confidence; same gradient-ascent mechanism; important security implication.

11.4 Network in Network (NiN, 2014): 1×1 Convolutions and GAP

Motivation: Richer Local Representations

AlexNet and VGG use standard convolutional layers — each output feature at a spatial position is a single linear combination of the input patch, followed by a nonlinearity. This is powerful, but a single linear filter can only detect one linear combination of the input channels per spatial position.

Network in Network (Lin, Chen & Yan, ICLR 2014) asked: *what if we replaced that single linear filter with a small, local MLP?* Rather than scanning one template over the image, scan an entire nonlinear function. This increases representational power at *no extra spatial cost* — the MLP is applied identically at every position, keeping the parameter count manageable.

NiN introduced two ideas that proved to be enormously influential — far more so than NiN itself: **MLPConv layers** (which are equivalent to 1×1 convolutions) and **Global Average Pooling (GAP)**. Both were adopted by GoogLeNet, ResNet, MobileNet, and virtually every modern archi-

tecture.

MLPConv Layers and the Equivalence to 1×1 Convolutions

Standard convolutional layer. At each spatial position (r, c), a filter $\mathbf{w} \in \mathbb{R}^{k \times k \times C_{\text{in}}}$ computes a single output feature:

$$f(r,c) = \sigma\left(\sum_{u,v,d} w_{u,v,d} \cdot x(r{+}u,\, c{+}v,\, d) + b\right)$$

where σ is a nonlinearity (e.g. ReLU) and (u, v) ranges over the $k \times k$ spatial neighbourhood.

MLPConv layer. NiN replaces the single linear filter with a small fully-connected MLP (with one or two hidden layers) applied to the *same* local patch:

$$\mathbf{h}^{(1)}(r,c) = \sigma\left(W^{(1)}\mathbf{x}_{r,c} + \mathbf{b}^{(1)}\right), \qquad f(r,c) = \sigma\left(W^{(2)}\mathbf{h}^{(1)}(r,c) + \mathbf{b}^{(2)}\right)$$

where $\mathbf{x}_{r,c} \in \mathbb{R}^{k^2 C_{\text{in}}}$ is the vectorised local patch and $W^{(1)}, W^{(2)}$ are weight matrices shared across all positions.

MLPConv = Conv + 1×1 Conv

The MLPConv operation is mathematically equivalent to a standard $k \times k$ convolution followed by one or more 1×1 **convolutions**. Here is why:

- The first layer of the MLP linearly combines the $k \times k$ patch into C_1 features — this is exactly a $k \times k$ conv producing C_1 channels.
- Each subsequent MLP layer applies a linear transformation *independently at each spatial position* to the current channel vector — this is exactly a 1×1 conv (spatial extent 1, mixing only channels).

So the name "Network in Network" is slightly misleading — the "network" inside is just extra 1×1 convolutions. The key insight is that 1×1 convolutions are cheap, parameter-efficient, and add nonlinearity across channels.

1×1 *Convolutions: Channel Mixing and Bottlenecks*

A 1×1 convolutional layer is so important it deserves its own treatment.

1×1 Convolution

A 1×1 conv with C_{out} filters takes an input volume of shape $H \times W \times C_{\text{in}}$ and produces an output volume of shape $H \times W \times C_{\text{out}}$ by applying a learned linear combination *across all C_{in} channels independently at each spatial position*:

$$\text{out}(r,c,k) = \sigma\left(\sum_{d=1}^{C_{\text{in}}} w_{k,d} \cdot \text{in}(r,c,d) + b_k\right), \quad k = 1, \ldots, C_{\text{out}}$$

Total parameters: $C_{\text{in}} \times C_{\text{out}} + C_{\text{out}}$ (weights + biases).

What does it do?

- **Channel mixing**: learns new linear combinations of the existing channels; can discover cross-channel correlations.

- **Dimensionality reduction** ($C_{\text{out}} < C_{\text{in}}$): compresses the channel dimension cheaply.
- **Dimensionality expansion** ($C_{\text{out}} > C_{\text{in}}$): creates more channels for subsequent layers.
- **Introduces nonlinearity**: when followed by ReLU, adds a nonlinear transformation between channels at zero spatial cost.
- **Does NOT mix spatial information**: each position (r, c) is processed independently — spatial structure is preserved.

Concrete parameter count example. Suppose we want to apply a 5×5 conv to a $28 \times 28 \times 192$ feature map, producing 32 output channels:

- Direct 5×5 conv: $5 \times 5 \times 192 \times 32 = 153{,}600$ parameters.
- 1×1 conv first ($192 \to 16$ channels): $192 \times 16 = 3{,}072$ parameters.
  Then 5×5 conv ($16 \to 32$): $5 \times 5 \times 16 \times 32 = 12{,}800$ parameters.
  Total: 15,872 parameters — **10× fewer**.

This is the **bottleneck** pattern: use a 1×1 conv to reduce channels before an expensive spatial conv, then optionally expand again.

Uses of 1×1 convolutions in modern architectures

Architecture	Role of 1×1 conv	Effect
NiN	MLPConv hidden layers	Richer local representation
GoogLeNet	Bottleneck before $3 \times 3/5 \times 5$	Reduces computation by $\sim 10\times$
ResNet (bottleneck)	Compress $\to$ conv $\to$ expand	Deeper nets with same compute
MobileNet	Pointwise conv after depthwise	Channel mixing after spatial conv

Global Average Pooling (GAP)

The second NiN innovation is to replace the *fully connected classifier head* with Global Average Pooling followed by a softmax.

The problem with FC layers at the end of a CNN. In AlexNet and VGG, the last convolutional block produces a volume (e.g. $7 \times 7 \times 512$). This is flattened to a vector of $7 \times 7 \times 512 = 25{,}088$ values and fed into FC layers with thousands of neurons. The FC layers:

- Contain the *vast majority of parameters* (VGG's 3 FC layers hold 102M of its 138M parameters).
- Are *position-sensitive*: the k-th weight connects to a specific spatial position; shifting the object by one pixel changes which weights activate.
- Fix the *input resolution*: the FC layer expects exactly $7 \times 7 \times 512$ inputs, locking the network to one image size.
- Are prone to *overfitting* (hence the heavy use of Dropout in AlexNet/VGG).

Global Average Pooling (GAP)

Given a final feature volume of shape $H \times W \times C$, GAP produces one scalar per channel by

averaging over the entire spatial extent:

$$F_k = \frac{1}{H \cdot W} \sum_{r=1}^{H} \sum_{c=1}^{W} f_k(r, c), \quad k = 1, \ldots, C$$

Output: a vector $\mathbf{F} \in \mathbb{R}^C$, fed directly into a softmax (or a single small FC layer).
Parameter count of the classifier head:

- FC after flatten: $H \cdot W \cdot C \times N_{\text{classes}}$ parameters (e.g. $25{,}088 \times 1000 \approx 25$M).
- GAP + softmax: $C \times N_{\text{classes}}$ parameters (e.g. $512 \times 1000 \approx 0.5$M).

Advantages of GAP over FC + flatten

- **Drastically fewer parameters**: no spatial-to-class weight matrix; essentially zero overfitting risk in the head.
- **Resolution-agnostic**: any input size produces the same C-dimensional vector after GAP — the network can handle variable-resolution inputs at test time.
- **Shift invariance**: a spatially shifted object shifts the feature map, but the *average* is unchanged (or nearly so) ⇒ GAP is a structural shift-invariance mechanism. FC after flatten is not: each neuron is tied to a specific position.
- **Structural regularisation**: GAP forces the network to distribute evidence globally across the spatial map rather than relying on positional shortcuts.
- **Enables CAM**: because GAP feeds directly into the final FC layer, the weights of that layer can be used to produce Class Activation Maps (see Section 12).

Why GAP works: the "one concept per channel" inductive bias

When the architecture uses GAP + single FC, the network is implicitly pressured to make each channel f_k correspond to a coherent, spatially distributed concept — because the only way to influence class c's score is through the average $F_k = \frac{1}{HW} \sum f_k$. This is why CAM works: the spatial map $f_k(r, c)$ literally shows *where* the concept of channel k appears in the image, and the FC weight w_k^c says how much concept k contributes to class c. This interpretability is a *direct consequence* of GAP — not of any special training procedure.

```
# x: output of last convolutional block, shape (batch, H, W, C)
gap = tf.keras.layers.GlobalAveragePooling2D(name='gap')(x)
# gap shape: (batch, C)
out = tf.keras.layers.Dense(num_classes, activation='softmax')(gap)
# Total head parameters: C * num_classes  (vs H*W*C * num_classes for FC)
```

Code 5: GAP + softmax classifier head in Keras (NiN style)

GAP vs. Global Max Pooling

Global Max Pooling (GMP) takes the maximum over the spatial map rather than the average. GAP is preferred for classification because it considers the *aggregate* evidence across the whole image, not just the single most-activated location. For CAM, GAP is required: the CAM formula $M_c(x, y) = \sum_k w_k^c f_k(x, y)$ relies on the linearity of the average. GMP breaks this linearity.

The NiN Architecture

For completeness, NiN stacks three *mlpconv blocks* (each = $k \times k$ conv + two 1×1 convs + ReLU), with max-pooling between blocks to downsample, and ends with a fourth mlpconv block whose number of filters equals the number of classes, followed by GAP and softmax. There are **no FC layers**.

NiN did not achieve state-of-the-art ImageNet accuracy (partly because its training was less refined than AlexNet's), but its two ideas — 1×1 convolutions and GAP — were immediately adopted by GoogLeNet and have remained standard ever since.

Recap: NiN, 1×1 Convolutions, and GAP

NiN — Summary (Exam Checklist)

1. **Motivation**: standard conv = single linear filter at each position; NiN replaces it with a small MLP ("richer local representation").
2. **MLPConv = Conv + 1×1 convs**: the first layer is a standard $k \times k$ conv; subsequent MLP layers are 1×1 convolutions applied channel-wise.
3. **1×1 conv**: mixes channels, reduces or expands depth, adds nonlinearity, does *not* mix spatial information. Parameters: $C_{\text{in}} \times C_{\text{out}}$.
4. **Bottleneck pattern**: 1×1 conv (reduce) $\rightarrow$ expensive spatial conv $\rightarrow$ 1×1 conv (expand); reduces parameters by $\sim 10\times$.
5. **GAP**: averages each channel over all spatial positions; output shape = C (not $H \cdot W \cdot C$).
6. **GAP advantages over FC**: far fewer parameters; resolution-agnostic; shift-invariant; enables CAM.
7. **GAP vs. GMP**: GAP = average (aggregate evidence, linear $\Rightarrow$ CAM works); GMP = max (single peak, nonlinear $\Rightarrow$ CAM does not apply).
8. **Exam trap**: GAP has *zero* learnable parameters; the only parameters in the head are in the final FC/softmax layer of size $C \times N_{\text{classes}}$.

11.5 GoogLeNet / Inception v1 (2014): Multi-Scale Branches

VGG showed that depth matters, but paid for it with 138M parameters — almost all of them in the FC layers at the top. **GoogLeNet** (Szegedy et al., CVPR 2015) asked the complementary question: *can we be deeper **and** use fewer parameters?* The answer was yes: 22 layers, only **5M parameters** (12× fewer than AlexNet), and a top-5 error of 6.7% on ImageNet — better than VGG. The key innovation is the **Inception module**.

Motivation: Multi-Scale Feature Extraction

Objects in natural images appear at widely varying scales: a face may fill the entire frame or occupy a tiny corner. Standard CNNs fix one filter size per layer and stack depth to grow the receptive field — but this means early layers use the same small filter regardless of whether the relevant structure is fine-grained (a texture) or coarse (an object outline).

Naive idea: use multiple filter sizes at the same layer by running 1×1, 3×3, and 5×5 convolutions *in parallel* on the same input, then concatenate the results. The network can then learn which scale

is most informative at each layer.

The problem: on a deep feature map with C channels, 5×5 convolutions are very expensive. Consider a $28 \times 28 \times 192$ feature map with a 5×5 conv producing 32 filters:

$$\text{FLOPs} = 5^2 \times 192 \times 32 \times 28^2 \approx 120\,\text{M multiply-adds per image.}$$

Stacking nine such modules, with larger channel counts, is computationally infeasible.

The solution (Inception module): insert 1×1 convolutions as *bottlenecks* to reduce the channel dimension *before* each expensive spatial convolution. This is the key architectural insight of GoogLeNet.

The Naive Inception Module (Conceptual Starting Point)

The conceptual design of the Inception module is to take a single input volume and apply three sizes of spatial filter in parallel, alongside a pooling branch, then concatenate all results:

Branch	Operation	Input shape	Output shape
1	1×1 conv, N_1 filters	$H \times W \times C$	$H \times W \times N_1$
2	3×3 conv, N_2 filters	$H \times W \times C$	$H \times W \times N_2$
3	5×5 conv, N_3 filters	$H \times W \times C$	$H \times W \times N_3$
4	3×3 max-pool	$H \times W \times C$	$H \times W \times C$
	Concatenate (axis = channel)		$H \times W \times (N_1 + N_2 + N_3 + C)$

Why concatenation requires `padding='same'` and stride 1

All four branches must produce feature maps of *identical spatial dimensions* $H \times W$ so that they can be stacked along the channel axis. This requires:

- **All convolutions**: `padding='same'` and `stride=1` so output spatial size equals input spatial size, regardless of filter size.
- **Max-pooling**: `padding='same'` and `stride=1` (this is unusual — in a standard CNN, pooling reduces spatial size; here it must not).

The channel dimension is the *only* dimension that changes across branches. The final output has more channels than any single branch but the same $H \times W$ as the input.

This naive design is computationally impractical. For a $28 \times 28 \times 192$ input and N_1=64, N_2=128, N_3=32 output channels:

Branch	Filter size	Parameters	MAdds
1×1	$1^2 \times 192 \times 64$	12,288	∼12M
3×3	$3^2 \times 192 \times 128$	221,184	∼173M
5×5	$5^2 \times 192 \times 32$	153,600	∼120M
Max-pool	(no parameters)	0	—
Total		**387,072**	**∼305M**

The 3×3 and 5×5 branches dominate the cost. Nine stacked modules would be untrainable on 2014-era hardware.

The Inception Module with Bottlenecks (Actual GoogLeNet Design)

The fix is to insert a 1×1 **bottleneck conv** before each expensive spatial conv, reducing the channel dimension first. The max-pool branch also gets a 1×1 conv *after* pooling to compress the channels it passes through (since pooling preserves all C channels, which is expensive for subsequent layers).

The resulting four-branch structure is:

Inception Module with Dimensionality Reduction (v1)

Input: $H \times W \times C$ feature map. All ops use `padding='same'`, `stride=1`.

Br.	Operations (in order)	Output channels
1	1×1 conv (N_1 filters)	N_1
2	1×1 conv (r_2 ch) $\to$ 3×3 conv (N_2 ch), intermediate: $r_2 \ll C$	N_2
3	1×1 conv (r_3 ch) $\to$ 5×5 conv (N_3 ch), intermediate: $r_3 \ll C$	N_3
4	3×3 max-pool $\to$ 1×1 conv (N_4 ch)	N_4
Concatenate all branches along channel axis		$N_1{+}N_2{+}N_3{+}N_4$

Spatial size $H \times W$ is **unchanged** by the module. The 1×1 bottleneck reductions $r_2, r_3 \ll C$ are the key to efficiency.

How much does the bottleneck save? Using the same $28\times28\times192$ input with r_2=96, r_3=16, N_1=64, N_2=128, N

Branch	Ops	Params (naive)	Params (bottleneck)	Saving
Br. 1 (1×1)	—	12,288	12,288	$1\times$
Br. 2 (3×3)	$1 \times 1{+}3 \times 3$	221,184	$192{\cdot}96 + 9{\cdot}96{\cdot}128$ $= 129{,}024$	$1.7\times$
Br. 3 (5×5)	$1 \times 1{+}5 \times 5$	153,600	$192{\cdot}16 + 25{\cdot}16{\cdot}32$ $= 15{,}872$	$\mathbf{9.7}\times$
Br. 4 (pool)	1×1 after	0	$192{\cdot}32 = 6{,}144$	—
Total		**387,072**	**163,328**	**2.4×**

The 5×5 branch benefits most — nearly a $10\times$ parameter reduction — because its filter is expensive and the 1×1 bottleneck squeezes from 192 down to just 16 channels before the spatial conv.

Naive vs. bottleneck — a classic exam question

Exam questions often present the Inception module in its **naive form** (no 1×1 bottlenecks) and ask you to explain what the bottleneck variant changes and why. Key points:

- The naive module applies 3×3 and 5×5 convolutions directly on the full C-channel input — this is expensive.
- The bottleneck variant inserts 1×1 convolutions *before* these spatial convolutions to reduce C to $r \ll C$ first, then applies the spatial conv on the compressed volume.
- The 1×1 bottleneck reduces **both** parameters ($\propto C_{\text{in}}$) and FLOPs ($\propto C_{\text{in}}$) by the same factor C/r.

- The max-pool branch gets a 1×1 conv *after* (not before) the pool, because the pool itself has no parameters — the 1×1 is there to limit the channels passed forward, not to reduce cost of the pool.

Role of each branch — why these four?

- **Branch 1 (1×1 conv):** pure channel mixing at the current position — cheapest possible operation. Captures cross-channel correlations with zero spatial context. Acts as the "skip" of the module: it sees the full input without any spatial aggregation.
- **Branch 2 (3×3 conv):** captures fine-to-medium spatial features (e.g. edges, small textures). The 3×3 filter is the workhorse of most modern CNNs.
- **Branch 3 (5×5 conv):** captures larger-scale spatial features (e.g. object parts, coarse shapes) in a single step. More expensive but extends the effective context window.
- **Branch 4 (max-pool $\rightarrow$ 1×1):** provides a *pooled* summary of the local neighbourhood. Max-pooling is a scale- and translation-invariant summary; the 1×1 conv then selects which pooled channels to propagate forward. This branch ensures the module always has access to a spatially-robust summary, not just convolved features.

The network learns *how much* to rely on each branch via the filter weights — there is no manual scale selection.

```
# x: input tensor shape (N, H, W, C)

# Branch 1: 1x1 conv only
b1 = Conv2D(64,  kernel_size=1, padding='same', activation='relu')(x)

# Branch 2: 1x1 bottleneck (96 ch) -> 3x3 conv (128 ch)
b2 = Conv2D(96,  kernel_size=1, padding='same', activation='relu')(x)
b2 = Conv2D(128, kernel_size=3, padding='same', activation='relu')(b2)

# Branch 3: 1x1 bottleneck (16 ch) -> 5x5 conv (32 ch)
b3 = Conv2D(16,  kernel_size=1, padding='same', activation='relu')(x)
b3 = Conv2D(32,  kernel_size=5, padding='same', activation='relu')(b3)

# Branch 4: 3x3 max-pool (stride=1!) -> 1x1 conv (32 ch)
b4 = MaxPooling2D((3,3), strides=(1,1), padding='same')(x)
b4 = Conv2D(32,  kernel_size=1, padding='same', activation='relu')(b4)

# Concatenate along channel axis: output shape (N, H, W,
    64+128+32+32=256)
y = Concatenate(axis=-1)([b1, b2, b3, b4])
```

Code 6: Inception module in Keras (bottleneck version)

PyTorch vs. Keras tensor conventions

In **Keras**, tensors are `(N,H,W,C)`: concatenate on `axis=-1` (last axis = channels). In **PyTorch**, tensors are `(N,C,H,W)`: concatenate on `dim=1`. Also: the Inception module has parallel branches that *merge*, so it cannot be expressed with `nn.Sequential` — you must code the branches explicitly in `forward()`.

GoogLeNet Architecture Details

GoogLeNet stacks 9 Inception modules (with occasional 2×2 max-pool layers between groups to halve the spatial dimensions), preceded by two initial conv+pool blocks for early spatial down-sampling. Key statistics:

Model	Parameters	Top-5 error (ImageNet)
AlexNet	60M	15.3%
VGG-16	138M	7.3%
GoogLeNet	5M	6.7%

GoogLeNet uses **no FC layers** — a GAP layer (see Section 11.4) feeds directly into a softmax classifier, saving roughly 50M parameters compared to VGG's three FC layers.

Auxiliary classifiers. At 22 layers, GoogLeNet is deep enough for gradients from the final loss to be severely attenuated by the time they reach the early Inception modules — the vanishing gradient problem. The solution is to attach two **auxiliary classification heads** to intermediate feature maps (after the 3rd and 6th Inception modules).

Each auxiliary head is a small sub-network: 5×5 average pool $\to$ 1×1 conv $\to$ FC(1024) $\to$ FC(N_{classes}) $\to$ softmax. Their losses enter the total training objective with weight 0.3:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{main}} + 0.3\,\mathcal{L}_{\text{aux1}} + 0.3\,\mathcal{L}_{\text{aux2}}.$$

How auxiliary classifiers fight vanishing gradients

The chain rule tells us that the gradient of the loss with respect to a parameter in layer ℓ is a product of Jacobians from layer ℓ all the way to the output. In a 22-layer network this product can become exponentially small. An auxiliary head attached at layer k provides a *short circuit*: it introduces a direct loss term whose gradient only needs to backpropagate through layers $1, \ldots, k$, not all the way to 22. This is an early precursor to ResNet's skip connections, which solve the same problem architecturally rather than through loss engineering.
The auxiliary heads are **discarded at inference** — they are a training device only.

Recap: GoogLeNet and the Inception Module

GoogLeNet / Inception v1 — Summary (Exam Checklist)

1. **Core idea**: process the same input with 1×1, 3×3, 5×5 convolutions and 3×3 max-pool *in parallel*; concatenate along the channel axis.
2. **Concatenation requires same spatial size**: all branches use `padding='same'`, `stride=1` — output is $H \times W \times (N_1 + N_2 + N_3 + N_4)$.
3. **Naive module**: no bottlenecks; 3×3 and 5×5 branches operate on full C-channel input — computationally prohibitive.
4. **Bottleneck module**: 1×1 conv *before* each spatial conv reduces $C \to r \ll C$; saves parameters *and* FLOPs by the same factor C/r.
5. **Max-pool branch**: 1×1 conv *after* the pool (not before) to compress the C channels the pool passes through.
6. **Role of each branch**: 1×1 = channel mixing; 3×3 = fine spatial; 5×5 = coarse spatial; max-pool = spatially robust pooled summary.
7. **No FC layers**: GAP + softmax; 5M total parameters vs. 138M for VGG.
8. **Auxiliary classifiers**: two auxiliary heads (weight 0.3) injecting gradient into early layers during training; discarded at inference; precursor to residual connections.
9. **Exam trap**: the 1×1 bottleneck reduces both parameters and operations — not just one of them. The 5×5 branch benefits most from the bottleneck.

11.6 ResNet (2015): Residual Learning

GoogLeNet showed that clever architecture design could achieve more with fewer parameters. But a fundamental ceiling remained: making networks *much* deeper consistently *hurt* performance, even on the training set. **ResNet** (He et al., CVPR 2016) diagnosed the root cause and introduced a fix so structurally simple — a single addition — that it seems obvious in hindsight. The result: a 152-layer network that surpassed human performance on ImageNet, and an architecture that became the backbone of virtually every modern vision system.

The Degradation Problem

Training plain (no skip connections) CNNs deeper than about 20–30 layers produces a paradoxical result: **both training error and test error increase**.

This is not overfitting

Overfitting means training error decreases while test error increases — the network memorises the training set but fails to generalise. The degradation problem is different: the *training* error itself is higher for the 56-layer network than for the 20-layer network. The deeper model is simply worse at optimisation, not at generalisation.

Why should a deeper network not be worse, in theory? A 56-layer network strictly contains the 20-layer network as a special case: copy the shallow network's solution into the first 20 layers and set every additional layer to the *identity mapping* (output = input). The 56-layer network could, in principle, do at least as well.

The empirical failure to do this reveals a fundamental difficulty: **learning the identity function is surprisingly hard for a stack of conv layers with random initialisation**. A conv layer initialized near zero computes something close to zero — not something close to the identity. For the layer to act as an identity it must learn a specific non-trivial set of weights. This is the bottleneck that residual connections are designed to eliminate.

The Residual Reparametrisation — Core Idea

The key insight is a change of variables. Suppose a block of layers should ideally compute some function $\mathcal{H}(\mathbf{x})$. In a plain network the conv layers must directly learn $\mathcal{H}$. ResNet instead reparametrises the problem: define the **residual**

$$\mathcal{F}(\mathbf{x}) \;:=\; \mathcal{H}(\mathbf{x}) - \mathbf{x},$$

and let the conv layers learn $\mathcal{F}$ instead of $\mathcal{H}$. The original mapping is then reconstructed by adding back the input:

$$\mathcal{H}(\mathbf{x}) = \mathcal{F}(\mathbf{x}) + \mathbf{x}.$$

This addition is implemented by a **skip connection** (also called a shortcut connection) — a direct wire from the block's input to its output that bypasses the conv layers entirely. The two signals are combined by **element-wise addition**: no learnable parameters, no nonlinearity on the skip path.

Why this reparametrisation makes optimisation easier

Consider two extremes:

- **Best case: the block should do nothing.** Target: $\mathcal{H}(\mathbf{x}) = \mathbf{x}$. In a plain network: the conv layers must learn weights such that their output equals their input — a non-trivial constraint. With a skip connection: the conv layers must learn $\mathcal{F}(\mathbf{x}) = \mathbf{0}$, i.e. drive all their weights to zero. This is trivially achieved by any initialisation scheme that starts near zero and any regulariser that penalises large weights. **The identity is now the default path, achieved for free.**
- **Normal case: the block should refine the representation.** The conv layers only need to learn the *correction* $\mathcal{F}(\mathbf{x})$ relative to the input — a typically small delta. Empirically, trained ResNets show that most residuals are small in magnitude, confirming that each block makes only a minor adjustment to its input.

Worked Example: Forward and Backward Pass

To make the mechanics completely concrete, consider a toy single-channel, single-pixel residual block with two weight layers (the spatial convolution structure is the same; we use scalars for clarity).

Setup. Input $x = 1.0$. Two weight layers with scalar weights $w_1 = 0.5$, $w_2 = 0.4$ and no biases or activations (to keep the algebra clean). The skip connection adds x back after the two layers.

Forward pass.

$$\begin{aligned} h_1 &= w_1 \cdot x = 0.5 \times 1.0 = 0.5 && \text{(output of first layer)} \\ h_2 &= w_2 \cdot h_1 = 0.4 \times 0.5 = 0.2 && \text{(output of second layer)} \\ \mathcal{F}(x) &= h_2 = 0.2 && \text{(residual)} \\ \text{output} &= \mathcal{F}(x) + x = 0.2 + 1.0 = \mathbf{1.2} && \text{(skip adds input back)} \end{aligned}$$

Compare with a plain block (no skip connection):

$$\text{output}_{\text{plain}} = h_2 = 0.2.$$

The skip connection keeps the output close to the input (1.2 vs. 1.0), while the plain block's output (0.2) has drifted far from the input — information is being destroyed.

Backward pass — gradient highway. Suppose the loss gradient arriving at the block output is $\frac{\partial L}{\partial \text{output}} = \delta$.

With skip connection:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial \text{output}} \cdot \frac{\partial \left(\mathcal{F}(x) + x\right)}{\partial x} = \delta \cdot \left(\frac{\partial \mathcal{F}}{\partial x} + 1\right) = \delta \cdot (w_1 w_2 + 1) = \delta \cdot (0.5 \times 0.4 + 1) = \delta \cdot 1.2.$$

Without skip connection:

$$\frac{\partial L}{\partial x} = \delta \cdot w_1 w_2 = \delta \cdot 0.2.$$

The gradient highway: the skip connection adds the constant $+1$ to the gradient multiplier at every block. In a deep network with L residual blocks, the gradient flowing back to the first layer contains a sum of path contributions rather than a product of small numbers. Even if $\frac{\partial \mathcal{F}}{\partial x}$ is tiny, the gradient does not vanish — it travels along the skip connections at full strength. Formally, the gradient through a stack of L residual blocks satisfies:

$$\frac{\partial L}{\partial \mathbf{x}_0} = \frac{\partial L}{\partial \mathbf{x}_L} \cdot \prod_{\ell=1}^{L} \left(1 + \frac{\partial \mathcal{F}_\ell}{\partial \mathbf{x}_{\ell-1}}\right).$$

Even if every $\frac{\partial \mathcal{F}_\ell}{\partial \mathbf{x}_{\ell-1}} \approx 0$, the product of $(1+0)^L = 1$ — gradient flows without attenuation. Compare with a plain network: $\prod_\ell \frac{\partial f_\ell}{\partial \mathbf{x}_{\ell-1}}$, which vanishes exponentially if each factor is < 1.

The Basic Residual Block (ResNet-18/34)

Basic Residual Block

Two 3×3 conv layers with Batch Normalisation, plus an identity skip connection:

$$\mathbf{y} = \sigma\big(\mathcal{F}(\mathbf{x}) + \mathbf{x}\big), \qquad \mathcal{F}(\mathbf{x}) = \mathrm{BN}(W_2\, \sigma(\mathrm{BN}(W_1\, \mathbf{x}))).$$

Order of operations (critical for implementation):

1. Conv_1 (3×3, same padding, stride 1, no bias)
2. $\mathrm{BatchNorm}_1$
3. ReLU
4. Conv_2 (3×3, same padding, stride 1, no bias)
5. $\mathrm{BatchNorm}_2$
6. **Add $\mathbf{x}$** (skip connection) — *before* the final activation
7. ReLU (applied *after* the addition)

Shape constraint. Element-wise addition requires $\mathcal{F}(\mathbf{x})$ and $\mathbf{x}$ to have *identical* shape (H, W, C). Both conv layers use `padding='same'` and `stride=1`, so spatial size is preserved. Channel count must also match.

What happens when spatial size or channel count changes? At stage boundaries (e.g. transitioning from $56 \times 56 \times 64$ to $28 \times 28 \times 128$), the skip path must be adjusted to match the new

shape. This is done with a **projection shortcut**: a 1×1 conv with the appropriate stride and output channels, applied only to $\mathbf{x}$ on the skip path:

$$\mathbf{y} = \sigma\big(\mathcal{F}(\mathbf{x}) + W_s\,\mathbf{x}\big), \qquad W_s \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times 1 \times 1}, \text{ stride } s.$$

The projection shortcut *does* have learnable parameters ($C_{\text{in}} \times C_{\text{out}}$ weights), unlike the identity shortcut which has zero parameters.

The two most common implementation errors

1. **ReLU before the addition.** Some students apply ReLU after Conv_2 and *then* add the skip. This is wrong — it prevents negative residuals from being expressed (since ReLU clips to zero). The correct order: BN → add → ReLU.
2. **Downsampling inside the block.** Setting `stride=2` on Conv_1 inside the residual block while using an unstrided skip connection will cause a shape mismatch at the addition. Downsampling must either go on *both* paths (via a strided projection shortcut on the skip) or be handled outside the block entirely.

```
# x: input tensor; filters: number of channels (must match input
    channels)
shortcut = x  # identity skip path

s = Conv2D(filters, 3, padding='same', use_bias=False)(x)
s = BatchNormalization()(s)
s = ReLU()(s)
s = Conv2D(filters, 3, padding='same', use_bias=False)(s)
s = BatchNormalization()(s)          # BN before addition, no ReLU yet

out = Add()([shortcut, s])          # skip connection: add BEFORE ReLU
out = ReLU()(out)                   # final activation AFTER addition
```

Code 7: Basic residual block in Keras

Bottleneck Residual Block (ResNet-50/101/152)

For deeper variants, each residual block uses a **bottleneck** design to reduce computation while keeping representational power:

Bottleneck Residual Block

Three conv layers instead of two: 1×1 (reduce) → 3×3 (spatial) → 1×1 (restore):

1. 1×1 conv: $C \to C/4$ channels (reduces depth by $4\times$)
2. 3×3 conv: $C/4 \to C/4$ channels (spatial processing on compressed volume)
3. 1×1 conv: $C/4 \to C$ channels (restores original depth)

Skip connection adds the original $\mathbf{x}$ (shape $H \times W \times C$) to the output of step 3 (also $H \times W \times C$) — same element-wise addition as the basic block.

Parameter comparison for a block operating on 256-channel features:

Block type	**Operations**	**Parameters**
Basic ($2 \times 3 \times 3$)	$2 \times 3^2 \times 256^2$	1,179,648
Bottleneck (1+3+1)	$256{\cdot}64 + 3^2{\cdot}64^2 + 64{\cdot}256$	69,632
Saving		**17**×

The bottleneck block is *much* cheaper, enabling ResNet-50/101/152 to go deeper without exploding compute.

BatchNorm in ResNet. Every conv layer is followed by Batch Normalisation *before* the activation. BN is essential: it stabilises activation distributions across layers, keeps gradients well-conditioned, and removes the need for Dropout — ResNet is typically trained with no dropout at all.

ResNet Architecture and Scale

Model	**Layers**	**Block type**	**Params**	**Top-5 error**
ResNet-18	18	Basic	11M	10.9%
ResNet-34	34	Basic	22M	9.0%
ResNet-50	50	Bottleneck	25M	7.0%
ResNet-101	101	Bottleneck	45M	6.2%
ResNet-152	152	Bottleneck	60M	**5.7%**
GoogLeNet	22	Inception	5M	6.7%
VGG-16	16	Plain conv	138M	7.3%

ResNet-152 achieved **3.57% top-5 error** as an ensemble, surpassing the estimated human performance of ∼5%. It won ILSVRC 2015 by a large margin. The key enabler was not a larger model (ResNet-152 has fewer parameters than VGG-16) but the ability to train an unprecedented 152 layers stably.

ResNet as an implicit ensemble

An alternative interpretation of residual networks (Veit et al., 2016): unrolling the additions shows that a ResNet with L blocks defines 2^L distinct paths from input to output (each block's residual can either be traversed or bypassed). The network behaves like an *implicit ensemble* of exponentially many shallow networks of varying depth. Most of the gradient signal flows through short paths (since the skip connections provide direct gradient highways), while the longer paths contribute fine-tuning. This explains why ResNets remain robust even when individual layers are dropped at test time — a property called **stochastic depth**.

Recap: ResNet and Residual Connections

ResNet — Summary (Exam Checklist)

1. **Degradation problem**: deeper plain networks have *higher training error* — an optimisation failure, not overfitting.
2. **Root cause**: learning the identity function is hard for conv layers; they start near zero output, not near the identity.
3. **Residual reparametrisation**: learn $\mathcal{F}(\mathbf{x}) = \mathcal{H}(\mathbf{x}) - \mathbf{x}$; reconstruct $\mathcal{H}$ by adding back $\mathbf{x}$ via a skip connection.
4. **Identity as default**: if $\mathcal{F} \to \mathbf{0}$ (easy), the block acts as identity — exactly what is needed for "extra" layers to be harmless.
5. **Gradient highway**: skip connection adds +1 to gradient multiplier per block; gradients flow back through L blocks as $(1 + \partial\mathcal{F}/\partial\mathbf{x})^L$ rather than a product that vanishes.
6. **Shape constraint**: skip + residual must have same shape. Use **identity shortcut** (no params) when shape is unchanged; use **projection shortcut** (1×1 conv, has params) when spatial size or channel count changes.
7. **Order of ops in basic block**: Conv → BN → ReLU → Conv → BN → **Add skip** → ReLU. Never apply ReLU before the addition.
8. **Bottleneck block** ($1 \times 1 \to 3 \times 3 \to 1 \times 1$): 17× fewer parameters than two 3×3 convs on full channels; used in ResNet-50+.
9. **BatchNorm**: applied after every conv, before activation; essential for stability; replaces the need for Dropout in ResNet.
10. **Exam trap**: the skip connection has *zero* learnable parameters (identity shortcut) — unless it is a projection shortcut, which has $C_{\text{in}} \times C_{\text{out}}$ parameters.

11.7 MobileNet (2017): Efficient Inference

ResNet solved the optimisation problem and enabled very deep networks. But for deployment on mobile phones or embedded devices, the limiting factor is not accuracy but *computational cost per inference*. **MobileNet** (Howard et al., arXiv 2017) introduced a drop-in replacement for standard convolutions that reduces cost by ∼8× with minimal accuracy loss.

Depthwise Separable Convolution

Let the input feature map have spatial size $D_F \times D_F$ and M channels. A standard $D_K \times D_K$ convolution producing N output channels requires:

$$\text{Cost}_{\text{std}} = D_K^2 \cdot M \cdot N \cdot D_F^2 \quad \text{(multiply-adds)}.$$

Depthwise separable convolution splits this single operation into two:

Depthwise Separable Convolution

1. **Depthwise convolution:** apply one $D_K \times D_K$ filter *independently to each of the M input channels* (no cross-channel mixing). Cost: $D_K^2 \cdot M \cdot D_F^2$.
2. **Pointwise convolution:** apply N filters of size 1×1 to mix channel information and

produce N output channels. Cost: $1^2 \cdot M \cdot N \cdot D_F^2 = M \cdot N \cdot D_F^2$.

Total cost: $D_F^2 \cdot M \cdot (D_K^2 + N)$.

Computational Saving

$$\frac{\text{Cost}_{\text{dw-sep}}}{\text{Cost}_{\text{std}}} = \frac{D_F^2 \cdot M \cdot (D_K^2 + N)}{D_K^2 \cdot M \cdot N \cdot D_F^2} = \frac{1}{N} + \frac{1}{D_K^2}.$$

For $D_K = 3$ (a 3×3 filter) and $N = 256$ output channels:

$$\frac{1}{256} + \frac{1}{9} \approx \frac{1}{8.6} \quad \Longrightarrow \quad \approx \mathbf{8} \times \text{ fewer operations.}$$

The accuracy drop on ImageNet is typically only 1–2% compared to a standard conv network of the same depth.

Intuition behind depthwise separable convolution

Standard convolution conflates two operations: (1) *spatial* feature detection within each channel, and (2) *cross-channel* feature combination. Depthwise separable convolution disentangles them: the depthwise step does spatial filtering per channel independently, and the pointwise step linearly combines channels. This factorisation preserves most of the representational power at a fraction of the cost — the key insight is that spatial and channel correlations can be modelled separately without much loss of expressiveness.

11.8 Architecture Comparison

ILSVRC Historical Timeline

The ImageNet Large Scale Visual Recognition Challenge (ILSVRC) drove a decade of CNN architecture innovation. The top-5 classification error dropped from $> 25\%$ in the pre-deep-learning era to below human performance by 2015:

CNN Architecture Timeline

Year	Architecture	Key contribution
1998	LeNet-5	First CNN; conv + pool + FC pipeline
2012	AlexNet	Deep learning revival; ReLU, dropout, GPU
2013	VGG	Depth with small (3×3) filters
2014	NiN	1×1 convs; GAP replacing FC
2014	GoogLeNet	Inception modules; multi-scale parallel branches
2015	ResNet	Residual connections; 152 layers; superhuman accuracy
2017	MobileNet	Depthwise separable convolutions for mobile devices
2019	EfficientNet	Compound scaling of depth/width/resolution

Landmark CNN Architectures: Summary

Model	Year	Params	ILSVRC top-5	Key innovation
LeNet-5	1998	61K	–	First CNN; conv+pool+FC
AlexNet	2012	60M	15.4%	ReLU, dropout, GPU training
VGG16	2014	138M	7.3%	Deep + small (3×3) filters
GoogLeNet	2014	5M	6.7%	Inception modules, GAP, aux. losses
ResNet-152	2015	60M	3.57%	Residual connections
MobileNet	2017	4.2M	–	Depthwise separable conv

VGGs are least efficient; Inception/ResNet are best

VGG's huge FC layers make it memory- and compute-hungry despite moderate depth. GoogLeNet achieves competitive accuracy with 27× fewer parameters than AlexNet. ResNet achieves superhuman performance by exploiting residual connections. MobileNet / EfficientNet prioritise computational efficiency for edge deployment.

Accuracy–Efficiency Trade-off (Canziani et al., 2016)

Beyond top-5 error, a practical architecture comparison must consider **computational cost** (ops/image) and **memory footprint** (parameters). Canziani et al. plot accuracy vs. operations per image, with bubble size encoding parameter count:

Key takeaways from the accuracy vs. ops/image plot:

- **VGG** (VGG-16, VGG-19): very high ops/image and parameter count — the *least efficient* family despite decent accuracy.
- **AlexNet**: lowest accuracy and not particularly efficient — the weakest option on both axes.
- **Inception models**: best accuracy-to-efficiency ratio. **Inception-v4** explicitly combines ResNet residual connections with Inception modules, achieving the best overall performance.
- **ResNet**: excellent accuracy scaling; the standard backbone for transfer learning.
- **MobileNet** and **SqueezeNet**: designed specifically for maximising efficiency — low ops/image and few parameters, at some cost in accuracy. Ideal for mobile/embedded deployment.

Inspecting Architecture Parameter Counts

When building or loading a model, always verify the parameter count before training:

```
# --- PyTorch ---
total     = sum(p.numel() for p in model.parameters())
trainable = sum(p.numel() for p in model.parameters() if p.requires_grad
    )
print(f"Total params: {total:,}  |  Trainable: {trainable:,}")

# --- Keras ---
```

```
model.summary()    # prints layer-by-layer output shape + param count
```

Code 8: Inspecting model parameters in PyTorch and Keras

What to check in model.summary()

For each layer: output shape (to verify spatial dimensions are preserved/reduced as intended) and parameter count. For BatchNorm2d with C channels: $4C$ total params ($2C$ trainable γ, β + $2C$ non-trainable running mean/var). For a Conv2d with $k \times k$ kernel, C_{in} input channels, C_{out} output channels: $k^2 \cdot C_{in} \cdot C_{out} + C_{out}$ params (weights + biases).

11.9 ResNeXt, Wide ResNet, and DenseNet: Variations

Wide ResNet

Instead of making the network deeper, **Wide ResNet** makes it *wider*: multiply the number of filters in each block by a factor k (width multiplier). A 50-layer wide ResNet with $k = 2$ outperforms the 152-layer original ResNet. Wider networks are also easier to parallelise on GPUs.

ResNeXt

ResNeXt (Xie et al., CVPR 2017) combines the ideas of residual learning (ResNet) and parallel branches (Inception). Each residual block is replaced by multiple parallel paths of the same topology:

$$\text{output} = \mathbf{x} + \sum_{i=1}^{C} \mathcal{T}_i(\mathbf{x})$$

where C is the *cardinality* (number of parallel paths) and each $\mathcal{T}_i$ is a bottleneck transform. Unlike Inception, all paths share the same topology (no expert design needed). Increasing cardinality is more effective than increasing width or depth at the same parameter budget.

DenseNet

DenseNet (Huang et al., CVPR 2017) takes skip connections to the extreme: *every layer receives feature maps from all preceding layers within a dense block*:

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}])$$

where $[\cdot]$ denotes concatenation along the channel axis. Benefits:

- Alleviates vanishing gradients (direct gradient paths to all layers).
- Promotes feature reuse (each layer has access to all earlier features).
- Reduces the number of parameters (features are reused, not recomputed).

Transition layers (conv + pool) between dense blocks reduce spatial size and channel count.

11.10 EfficientNet (2019): Compound Scaling

All previous architectures scaled along a *single* dimension: VGG went deeper, Wide ResNet went wider, and higher-resolution inputs improved accuracy. **EfficientNet** (Tan & Le, ICML 2019) asked: *what is the best way to scale all three dimensions simultaneously?*

Compound Scaling

EfficientNet scales network depth d, width w, and input resolution r together using a single compound coefficient ϕ:

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \text{subject to} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad \alpha \geq 1,\ \beta \geq 1,\ \gamma \geq 1.$$

The constraint $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ ensures that the total **FLOP cost** doubles with each unit increase in ϕ. This is because:

- FLOPs scale linearly with depth d.
- FLOPs scale as β^2 with width (doubling channels quadruples parameters *and* multiply-adds in each conv layer).
- FLOPs scale as γ^2 with resolution (doubling spatial size quadruples the number of positions to convolve over).

Fixing the total FLOP budget and optimising α, β, γ jointly via a small grid search gives the best balanced allocation.

The base network (EfficientNet-B0) is found by neural architecture search (NAS) on a small budget, and α, β, γ are determined by a small grid search at $\phi = 1$. Scaling up with $\phi = 1, \ldots, 7$ gives EfficientNet-B1 through B7. EfficientNet-B7 achieves state-of-the-art accuracy with an order of magnitude fewer parameters than comparable ResNets.

Why compound scaling outperforms single-axis scaling

Intuitively: larger input resolutions require deeper networks to capture the full receptive field, *and* wider networks to capture the finer details. Scaling only one dimension is suboptimal because the other two become bottlenecks. Compound scaling finds the right balance so that each unit of compute budget is used as efficiently as possible.

11.11 Recap: Design Principles Across All Architectures

Key Design Principles (Exam Summary)

1. **Depth beats width** (at the same parameter budget): more layers → larger receptive field, more abstraction.
2. **Small filters, stacked** (3×3): same receptive field as large filters with fewer parameters and more nonlinearities (VGG).
3. 1×1 **convolutions as bottlenecks**: cheap channel mixing and dimensionality reduction (GoogLeNet, ResNet bottleneck, MobileNet pointwise).
4. **Skip/residual connections**: facilitate gradient flow and make identity the default mapping, enabling very deep networks (ResNet, DenseNet).
5. **GAP over FC**: reduces parameters, improves shift invariance, enables variable-size inputs (NiN, GoogLeNet, ResNet).
6. **Parallel branches**: capture multi-scale features simultaneously (Inception).
7. **Depthwise separable convolutions**: factorised computation for mobile/edge deployment (MobileNet, EfficientNet).

12 CNN for Localisation, Weakly Supervised Localisation, and Explainability

12.1 Data Pre-processing and Batch Normalisation

Why Pre-processing Matters

Gradient-based optimisers work best when the data live in a well-conditioned region of input space. Normalisation achieves two goals:

- **Centring**: brings the distribution close to the origin, removing constant offsets that the optimiser would otherwise have to absorb into the weights.
- **Rescaling**: makes each input dimension comparable in magnitude, producing a more isotropic loss landscape and, in practice, faster convergence with less sensitivity to the initial learning rate.

Pre-processing Strategies for CNNs

PCA whitening is uncommon for CNNs — its cost is prohibitive on raw images and the per-channel statistics approach works well in practice. The three standard strategies, illustrated by landmark architectures:

CNN Pre-processing: Common Choices

Model	Pre-processing
AlexNet	Subtract the *mean image* — a full $32 \times 32 \times 3$ array (one mean value per pixel and channel).
VGG	Subtract the *per-channel mean* (three scalars, one per colour channel).
ResNet	Subtract the per-channel mean *and* divide by the per-channel std (six scalars).

Normalisation statistics are model parameters

They must be computed *exclusively on the training split* and then applied identically to the validation and test splits. Computing statistics on the full dataset before splitting introduces data leakage and leads to optimistically biased generalisation estimates. When using a pre-trained model, always import and apply its original pre-processing function.

Batch Normalisation

Even after input normalisation, intermediate activations can shift and grow during training — a phenomenon called **internal covariate shift** (Ioffe & Szegedy, ICML 2015). Batch Normalisation (BN) addresses this by normalising activations on the fly at each layer.

The transformation. Given a mini-batch of activations $\{x_i\}$ for a single channel, BN first applies z-score normalisation:

$$x_i' = \frac{x_i - \mathbb{E}[x_i]}{\sqrt{\mathrm{Var}[x_i] + \varepsilon}},$$

where $\mathbb{E}[x_i]$ and $\mathrm{Var}[x_i]$ are computed *per channel* over the current mini-batch, and $\varepsilon > 0$ ensures numerical stability. To avoid over-constraining the representation (e.g. always forcing a sigmoid into its linear regime), BN adds **learnable affine parameters**:

Batch Normalisation

$$y_{i,j} = \gamma_j \, x'_i + \beta_j$$

where γ_j (scale) and β_j (shift) are learned per channel via backpropagation. The mean and variance are *non-trainable*: they are computed from each mini-batch during training and replaced by running averages at test time.

Parameter count. For a BN layer with C input channels:

Parameter type	Count
Trainable (scale γ + shift β)	$2C$
Non-trainable (running mean + running variance)	$2C$
Total	$4C$

For example, a BN layer after a convolution producing 64 feature maps has 128 trainable and 128 tracked parameters (256 total).

Training vs. inference. During inference the running averages are frozen, making BN a *fixed linear operator* that can be **fused with the preceding layer** at zero extra cost.

Benefits and caveats of Batch Normalisation

Benefits: makes deep networks much easier to train; improves gradient flow; allows higher learning rates and faster convergence; reduces sensitivity to initialisation; acts as a regulariser.
Watch out: BN behaves differently during training and inference — always call `model.eval()` when evaluating, just as with dropout.

PyTorch API:

```
# affine=True (default): learns gamma and beta
m = nn.BatchNorm2d(num_features=64)

# affine=False: pure z-score normalisation, no learnable params
m = nn.BatchNorm2d(num_features=64, affine=False)
```

Code 9: BatchNorm2d in PyTorch

Key parameters: `num_features` $= C$ (number of channels); `eps` $= 10^{-5}$ (numerical stability); `momentum` $= 0.1$ (controls running average update); `affine` = learns γ, β when `True`.

12.2 Multi-Label Classification

Standard classifiers output exactly one class per image. When an image can contain *multiple simultaneous objects* (e.g. both a cat and a dog), the output head must change.

Multi-Label CNN Architecture

Replace the **softmax** output with **independent sigmoid** activations, one per class:

$$p_i = \sigma(z_i) = \frac{1}{1 + e^{-z_i}} \in [0, 1].$$

Unlike softmax, the outputs $\{p_i\}$ do *not* sum to 1 — each is an independent probability of class membership. The model is trained with **binary cross-entropy** loss:

$$\mathcal{L} = -\frac{1}{L}\sum_{i=1}^{L}\big[y_i \log p_i + (1-y_i)\log(1-p_i)\big],$$

where $y_i \in \{0,1\}$ is the ground-truth indicator for class i and L is the total number of classes.

12.3 Class Activation Maps (CAM)

Motivation: The GAP Layer Revisited

The GAP layer was originally introduced as a structural regulariser. Zhou et al. (CVPR 2016) showed it also preserves *spatial localisation information* up to the final prediction. By computing a weighted combination of the last convolutional feature maps, one can produce a spatial heat-map — the **Class Activation Map** — highlighting the image regions most responsible for a given class prediction.

Mathematical Derivation

Let the last convolutional block produce C feature maps $f_k(x, y)$. The GAP layer computes:

$$F_k = \sum_{(x,y)} f_k(x, y), \qquad k = 1, \ldots, C.$$

A single FC layer then scores each class c:

$$S_c = \sum_{k=1}^{C} w_k^c \, F_k,$$

where w_k^c encodes the importance of feature map k for class c. Substituting:

$$S_c = \sum_k w_k^c \sum_{(x,y)} f_k(x, y) = \sum_{(x,y)} \underbrace{\sum_k w_k^c f_k(x, y)}_{M_c(x,y)}.$$

Class Activation Map

$$M_c(x, y) = \sum_k w_k^c \, f_k(x, y)$$

$M_c(x, y)$ directly indicates the importance of spatial location (x, y) for predicting class c.

GAP + FC vs. NiN

Unlike NiN (which feeds GAP directly into the softmax), adding a separate FC layer means the number of feature-map channels C need not equal the number of classes L, giving much more flexibility in network design.

CAMs are low-resolution (same spatial size as the last feature maps). They are upsampled to the input image resolution via **bilinear interpolation** before visualisation.

Class Specificity

CAMs are **class-specific**: the same feature-map activations $\{f_k(x, y)\}$ are shared across all queries, but the weights $\{w_k^c\}$ differ per class. For an image containing both a cat and a dog, querying CAM for "cat" highlights the cat region; querying for "dog" highlights the dog region.

Practical Considerations

- CAM can be inserted into any pre-trained network by removing the original FC head and replacing it with GAP + a single dense layer, which must be fine-tuned.
- In VGG, removing the FC layers discards $> 90\%$ of all parameters, causing a noticeable accuracy drop.
- **Resolution/semantic trade-off**: placing GAP on larger feature maps (earlier in the network) gives finer-grained CAMs but less semantically meaningful ones; later placement is coarser but richer.
- **GAP vs. GMP**: GAP encourages capturing the *entire* object extent; Global Max Pooling focuses on the single most discriminative location per channel, yielding sparser localisations.

12.4 Weakly Supervised Localisation (WSL)

Motivation: The Cost of Spatial Annotations

Object detection requires bounding-box labels. These are expensive: a human annotator must carefully draw a rectangle around every object instance in every training image. In contrast, image-level labels ("this image contains a dog") are nearly free — they can be obtained from search-engine tags, social media metadata, or simple yes/no questions. This gap in annotation cost motivates **Weakly Supervised Localisation** (WSL): can we obtain bounding-box predictions at test time having trained with *only* image-level labels?

Weak Supervision

Supervision is *strong* when the training labels match the inference target exactly (e.g. training on bounding boxes to predict bounding boxes). Supervision is *weak* when the training labels are *cheaper and less specific* than the inference target:

- **Training labels**: image-level class label $c \in \{1, \dots, K\}$ — tells us *what* is in the image.
- **Inference target**: bounding box $\mathbf{b} \in \mathbb{R}^4$ — specifies *where* the object is.

The model $\mathcal{M}$ is trained for classification ($X \to K$) but at test time is queried for localisation ($X \to \mathbb{R}^4$). Localisation is a *by-product*, not an explicit training objective.

Why GAP Is the Key Architectural Requirement

The critical observation is that a network with FC layers after the last convolutional block *discards all spatial information*: the FC weight matrix sees only global averages and cannot tell where in the image a feature activated. GAP, by contrast, preserves the spatial structure of the feature maps all the way to the class score — via the identity $S_c = \sum_{(x,y)} M_c(x, y)$ derived earlier. This is what makes CAM possible: the class score S_c is literally the *spatial integral* of the class activation map, so the map is implicitly encoded in the network weights.

In other words, **any classifier trained with a GAP layer has, for free, learned a spatial detector for each class it was trained on**. It simply did not know it.

WSL Pipeline via CAM

1. **Train** a CNN classifier with a GAP layer followed by a single dense output layer. No bounding-box labels needed.
2. **Compute** the CAM $M_c(x, y) = \sum_k w_k^c f_k(x, y)$ for the predicted class c.
3. **Upsample** M_c to the input image resolution via bilinear interpolation.
4. **Threshold** the upsampled map at $0.2 \times \max_{(x,y)} M_c(x, y)$, producing a binary mask of the discriminative region.
5. **Fit** the smallest axis-aligned bounding box around the thresholded mask.

Localisation for free

The model is never shown a bounding box during training. Yet it implicitly learns *where* objects are because the only way to correctly classify an image using spatial feature maps is to have the relevant feature maps activate in the right places. The CAM simply makes this implicit spatial knowledge explicit. The classifier answers *what*; the CAM map answers *where*.

Limitations of WSL via CAM

- The produced bounding boxes are **loose and incomplete**: CAM highlights the most discriminative region, not the full object extent. A face classifier may fire only on the eyes; the whole face is not highlighted.
- The approach requires the network to have a **GAP layer**, which excludes standard architectures with FC layers unless they are retrofitted and fine-tuned (with potential accuracy loss).
- WSL cannot produce instance-level separations: if two dogs appear in the same image, the CAM merges their regions.

12.5 Explaining Neural Network Predictions: Saliency Maps

Motivation

Deep networks have millions of parameters and largely opaque internals. In safety-critical applications (medical diagnosis, fraud detection, autonomous driving), it is often mandatory to understand *why* a model made a prediction. **Saliency maps** are the most widespread tool for this.

A saliency map is a spatial heat-map overlaid on the input image: higher values indicate regions that contributed more to the final prediction. It answers the question: *"which pixels or image regions were most important for this specific prediction?"*

Key applications:

- **Understanding mistakes**: visualise what the network "looked at" when it predicted incorrectly.
- **Discovering systematic errors** (*Clever Hans* phenomena): a model may be correct for the wrong reason — e.g. a horse classifier that fires on a watermark rather than the horse. A saliency map immediately reveals this failure.

- **Regulatory compliance**: in many domains (e.g. credit scoring in the EU, medical AI), a model must provide human-interpretable justification for its decisions.

CAM as a Saliency Map — and Its Limitations

CAM, introduced in the previous subsection, *is* a saliency map method: it produces a spatial heat-map highlighting which regions drove the class prediction. However, it has a fundamental architectural constraint that severely limits its use as a *post-hoc explainability tool*.

CAM as an explainability tool — limitations

Limitation 1 — Architectural requirement (most critical). CAM requires the network to have a specific architecture: the last convolutional block must be followed by a *GAP layer* and a *single dense layer*. Standard production architectures (e.g. VGG with three FC layers) do not have this structure. To apply CAM to VGG, one must:

1. Remove the three FC layers (discarding $> 90\%$ of VGG's parameters).
2. Insert GAP + a single FC layer.
3. **Retrain or fine-tune** the modified network on the target dataset.

The model you end up explaining is therefore *different* from the model you originally deployed. This undermines the purpose of explainability: you are not explaining the decision of your actual model, but of a structurally different surrogate.

Limitation 2 — Low spatial resolution. The CAM is produced at the spatial resolution of the last convolutional feature maps, which in deep networks like VGG or ResNet can be as small as 7×7 or 14×14 pixels. Upsampling to the input resolution via bilinear interpolation produces blurry, imprecise maps — often sufficient to identify the general object region, but not detailed enough for fine-grained localisation.

Limitation 3 — Only highlights discriminative, not complete, regions. CAM identifies the regions that are most discriminative for a class, not the full spatial extent of the object. A network trained to classify dogs might produce a CAM that fires only on the head, ignoring the body — because the head alone is sufficient for classification. This makes CAM-based bounding boxes systematically underestimate object size.

These limitations motivate Grad-CAM, which removes the first and most critical limitation by working with *any* CNN architecture without any modification.

The Relationship Between Saliency Maps and CAMs

It is useful to place CAM and Grad-CAM in the broader family of saliency methods:

Family of Saliency Methods

Method	How it computes importance	Architecture constraint
Gradient saliency	$\lvert\partial y^c/\partial x_{ij}\rvert$ — input pixel gradient; measures how sensitive the score is to each pixel.	None
CAM	Weighted sum of last-layer feature maps $\sum_k w_k^c f_k(x,y)$; weights from FC layer.	GAP + single FC required
Grad-CAM	Weighted sum of last-layer feature maps; weights from *spatial average of gradients* $\frac{1}{nm}\sum_{i,j}\partial y^c/\partial a_k(i,j)$.	None (any CNN)
Augmented Grad-CAM	Super-resolved Grad-CAM via augmentation + inverse problem.	None

All methods produce class-specific heat-maps of the same form: a weighted combination of spatial signals. The key difference is *where the weights come from* (learned FC weights vs. gradients) and *what signal is weighted* (feature maps vs. input pixels).

The conceptual link is this: CAM and Grad-CAM are both performing the same operation — weighting spatial feature maps by their importance to a class and summing — but CAM reads the importance weights from the trained FC layer, while Grad-CAM *computes* them on-the-fly using gradients.

Exam trap: do CAM and Grad-CAM give the same maps on a GAP + single-FC network?

It can be shown that on a network with GAP followed by a *single* FC layer, the Grad-CAM weights $w_k^c = \frac{1}{nm}\sum_{i,j}\frac{\partial y^c}{\partial a_k(i,j)}$ are **proportional** to the CAM weights w_k^c (the FC layer weights) — they encode the same ranking of feature-map importance.
However, **the two maps are not identical**: the absolute scale of the weights differs (gradients are scaled by the softmax derivative, which depends on the current prediction confidence), so the resulting heat-maps will look similar in shape but have different numerical values.
The correct exam statement is: Grad-CAM *reduces to* CAM (up to a scalar factor) on a GAP + single-FC architecture. The maps highlight the same spatial regions, but the aggregation weights are technically different quantities.
In contrast, on any architecture with *multiple FC layers or no GAP*, CAM cannot be computed at all, while Grad-CAM can — this is the key advantage of Grad-CAM.

Grad-CAM

Grad-CAM (Selvaraju et al., ICCV 2017) generalises CAM to any CNN by replacing the learned FC weights with the **gradient of the class score with respect to the last convolutional feature maps**.

Let $a_k(i,j)$ be the activation of feature map k at location (i,j), and y^c the pre-softmax score for class c. The Grad-CAM importance weight for feature map k is the global spatial average of the gradient:

Grad-CAM

$$w_k^c = \frac{1}{nm}\sum_i\sum_j \frac{\partial y^c}{\partial a_k(i,j)} \qquad \text{(importance of feature map } k \text{ for class } c\text{)}$$

$$g^c = \text{ReLU}\left(\sum_k w_k^c \, \mathbf{a}_k\right) \qquad \text{(Grad-CAM heat-map)}$$

The ReLU discards feature maps that *decrease* the score for class c (negative w_k^c), retaining only those that push the prediction upward. This makes g^c a one-sided, class-specific localisation map.

Desiderata for saliency maps

1. **Class discriminativeness**: different classes must produce different heat-maps on the same image. Both CAM and Grad-CAM satisfy this by design (the weights w_k^c depend on c).
2. **High spatial resolution**: critical in medical and industrial imaging where small regions matter. Grad-CAM operates on the last convolutional layer and therefore inherits its low spatial resolution. There is an inherent *depth trade-off*: early layers retain fine spatial detail but have little semantic content; late layers are semantically rich but spatially coarse. Augmented Grad-CAM addresses this.

Augmented Grad-CAM

Augmented Grad-CAM (Morbidelli et al., ICASSP 2020) improves spatial resolution via a **super-resolution** approach, without modifying the underlying network.

Key idea. A CNN is approximately prediction-invariant to small geometric transformations, but the Grad-CAM g_ℓ computed from the ℓ-th augmented input $\mathcal{A}_\ell(\mathbf{x})$ contains *different spatial information* about the true high-resolution map $\mathbf{h}$. Each g_ℓ is modelled as a linear downsampled version of a perturbed $\mathbf{h}$:

$$g_\ell \approx \mathcal{D}\big(\mathcal{A}_\ell(\mathbf{h})\big),$$

where $\mathcal{D} : \mathbb{R}^{N\times M} \to \mathbb{R}^{n\times m}$ is a linear downsampling operator.

Super-resolution formulation. The high-resolution map is recovered by solving a convex inverse problem:

Augmented Grad-CAM Optimisation

$$\hat{\mathbf{h}} = \arg\min_{\mathbf{h}} \; \frac{1}{2}\sum_{\ell=1}^{L} \big\|\mathcal{D}(\mathcal{A}_\ell(\mathbf{h})) - g_\ell\big\|_2^2 + \lambda \, \mathrm{TV}_{\ell_1}(\mathbf{h}) + \frac{\mu}{2}\|\mathbf{h}\|_2^2$$

where $\mathrm{TV}_{\ell_1}(\mathbf{h}) = \sum_{i,j} |\partial_x h(i,j)| + |\partial_y h(i,j)|$ is the **anisotropic total-variation** regulariser, which preserves edges in the reconstructed map. The problem is convex and non-smooth; it is solved via **sub-gradient descent**.

Generality of the framework

Augmented Grad-CAM is a general approach: the augmentation + super-resolution pipeline can be combined with *any* visualisation tool, not only Grad-CAM, to produce high-resolution saliency maps.

12.6 Application: Weakly Supervised Localisation for Environmental Crime

These techniques have direct real-world impact. The **ODIN** platform, developed in collaboration with ARPA Lombardia and Prof. Fraternali's group at Politecnico di Milano, applies WSL to detect

illegal landfills in very-high-resolution (VHR) satellite imagery:

- A binary classifier ("waste" vs. "no waste") is trained on image-level labels only.
- Grad-CAM highlights sub-regions most responsible for positive predictions, effectively segmenting candidate landfill areas without pixel-level annotation.
- The system has automatically scanned more than 100 municipalities in Lombardy on Google Earth imagery.
- Regular survey cycles detect both existing and new disposal sites.

This is a compelling demonstration of how cheap classification-level supervision can yield actionable spatial predictions at scale.

12.7 Recap: Localisation, CAM, and Explainability

Lecture 10 — Summary (Exam Checklist)

1. **Batch Normalisation**: z-score normalisation per channel per mini-batch + learnable γ, β; $4C$ total parameters; fused linear operator at test time; behaves differently in train vs. eval.
2. **Multi-label classification**: sigmoid outputs (not softmax) + binary cross-entropy loss; outputs do not sum to 1.
3. **CAM**: $M_c(x, y) = \sum_k w_k^c f_k(x, y)$; requires GAP + single FC layer; class-specific; needs bilinear upsampling.
4. **Weakly Supervised Localisation**: train on image-level labels, threshold CAM at $0.2\times$ max, fit bounding box — localisation for free.
5. **Saliency maps**: visualise which spatial regions drive a prediction; useful for debugging and Clever Hans detection.
6. **Grad-CAM**: uses $\partial y^c / \partial a_k$ as importance weights; works on any CNN without modifying the architecture.
7. **Augmented Grad-CAM**: super-resolution of low-resolution heat-maps via augmentation + TV-regularised inverse problem.

13 Convolutional Neural Networks for Semantic Segmentation

13.1 What is Image Segmentation?

Before diving into neural approaches, it is worth understanding what segmentation formally means and why it is hard. Given an image $I : \Omega \to \mathbb{R}^3$ with spatial domain Ω, **image segmentation** consists in estimating a partition $\{R_1, R_2, \ldots, R_K\}$ of Ω such that $\bigcup_k R_k = \Omega$ and $R_i \cap R_j = \emptyset$ for $i \neq j$.

There are two broad flavours of segmentation. *Unsupervised* segmentation (e.g. superpixel algorithms such as SLIC) groups pixels by low-level visual similarity without any class vocabulary. *Supervised*, or **semantic segmentation**, assigns each pixel a label from a fixed category set, using annotated training data. This section is entirely about the supervised case.

Semantic Segmentation — Formal Problem Statement

Given an image $I \in \mathbb{R}^{H\times W\times 3}$, **semantic segmentation** consists in assigning to each pixel $(i,j) \in \Omega$ a label $\hat{y}(i,j)$ drawn from a fixed set of categories $\mathcal{C} = \{c_1, c_2, \ldots, c_K\}$. The output is a *label map* $\hat{Y} \in \mathcal{C}^{H\times W}$.
Formally, we seek a model $f_\theta : \mathbb{R}^{H\times W\times 3} \to \mathcal{C}^{H\times W}$ trained on pairs (I_n, Y_n) from a labelled dataset $\mathcal{D} = \{(I_n, Y_n)\}_{n=1}^{N}$, where Y_n is a pixel-wise ground-truth annotation.

Semantic vs. Instance Segmentation

Semantic segmentation assigns a single class to every pixel, but does *not* distinguish between different instances of the same class. If two persons overlap, all of their pixels receive the label `person`. **Instance segmentation** goes further, separately delineating each individual object instance (e.g. `person#1`, `person#2`). This lecture addresses semantic segmentation only.

Why Semantic Segmentation Matters

In natural-image benchmarks, object detection and counting may seem more practically relevant than dense pixel labelling. However, in domains such as **medical imaging** and **remote sensing**, semantic segmentation is indispensable: one must measure the *area* covered by a tumour, a lesion, or a land-use class — none of which a bounding box can capture faithfully. The same applies to autonomous driving, where the full drivable surface must be identified, not merely bounding boxes around obstacles.

Annotations and Their Cost

Training a semantic segmentation model requires *dense, pixel-wise annotations* — every pixel in every training image must be labelled. This is extremely labour-intensive: the Microsoft COCO dataset provides 200 000 annotated images across 80 categories, with humans annotating each instance's contour. In specialised domains (e.g. medical imaging), annotation time can reach *2 hours per image*, creating a severe bottleneck that motivates weakly supervised and transfer-learning approaches.

13.2 Naive Approaches and Their Shortcomings

Patch-Based (Plain Vanilla) Classification

The most straightforward approach is to repurpose a standard CNN classifier as a sliding-window segmenter:

1. Extract every patch $P_{i,j}$ of a fixed size centred at pixel (i,j).
2. Assign to each patch the ground-truth label of its *central pixel*.
3. Train a CNN classifier on this patch dataset.
4. At inference, slide the classifier over all patch positions and write the predicted class back to the central pixel.

This is conceptually clean but **computationally catastrophic**: for an $H \times W$ image, one must run $H \times W$ separate forward passes, each recomputing the shared convolutional features of hugely overlapping patches. The patch size is also a difficult hyperparameter — too small a patch yields insufficient semantic context; too large a patch loses precise spatial grounding.

Convolutions Only (No Pooling)

A second naive idea is to build a purely convolutional network with no pooling or striding at all, so that the spatial resolution is preserved throughout, and the final layer directly predicts one label per pixel. While this avoids the resolution loss, it comes at a severe cost: the *receptive field* of each output unit is tiny (growing only linearly with depth), so the network cannot integrate the global context needed to understand what an object is. It is also extremely memory- and compute-inefficient for deep architectures.

The Core Tension in Semantic Segmentation

Semantic segmentation faces an inherent tension between **semantics** and **localisation**:

- **Global context resolves *what***: pooling and striding allow large receptive fields, enabling the network to recognise objects from their holistic appearance.
- **Local detail resolves *where***: fine spatial resolution is needed to draw sharp, accurate boundaries.

Neither extreme (purely local convolutions vs. aggressive pooling) is satisfactory on its own. The right architecture must do both, which motivates the encoder–decoder designs discussed below.

13.3 From CNN Classifier to Fully Convolutional Network (FC-CNN)

A seminal conceptual step, introduced by Long, Shelhamer, and Darrell (CVPR 2015), is to observe that a CNN classifier can be *surgically converted* into a dense predictor without retraining from scratch, simply by replacing its fully connected (FC) layers with equivalent 1×1 convolutions.

Why FC Layers Constrain Input Size

In a standard CNN, the convolutional and pooling layers slide over the input and thus accept any spatial size — they always produce a spatial feature map whose dimensions scale proportionally with the input. The *fully connected* layer, however, has a fixed number of input neurons equal to the spatial size of the feature map it receives. A larger input image produces a larger feature map, which then has too many entries to feed into the fixed-size FC layer. As a result, the FC layer acts as a hard constraint that locks the CNN to one fixed input resolution.

The FC $\rightarrow$ Conv Equivalence

The crucial observation is that an FC layer is a *linear* operation, and any linear operation on a spatial feature map can be written as a convolution. Concretely, suppose the feature map just before the first FC layer has spatial extent $H' \times W'$ and C channels, so it is a tensor of shape $(H' \times W' \times C)$. An FC layer with L output neurons applies a weight matrix $W \in \mathbb{R}^{L \times (H'W'C)}$ to the flattened input.

Equivalence of FC and 1×1 Convolution

An FC layer with L outputs operating on a feature map of shape (H', W', C) is **exactly equivalent** to a 2D convolution with L filters of spatial size $H' \times W'$ and depth C, applied with no padding and stride 1. The weight of the ℓ-th filter is the reshaped ℓ-th row of the FC weight matrix.
In the special case $H' = W' = 1$ (feature map already collapsed to 1×1), this reduces to a 1×1 **convolution** — a pointwise linear combination across channels.

This equivalence means we can replace every FC layer in a pre-trained CNN with a convolutional layer carrying the same weights, obtaining a **Fully Convolutional Network** (FC-CNN) that: (i) is numerically identical to the original classifier when fed the original fixed-size input; and (ii) seamlessly accepts larger inputs, producing a *spatial heatmap* of class probabilities instead of a single class vector.

FC-CNN Output: Heatmaps

A fully convolutional CNN can process input images of different sizes. If a larger image fo size $(H + \delta) \times (W + \delta)$ is given as input, the network produces a correspondigly larger ouput heatmap. Each spatial position (u, v) in this heatmap corresponds to a specific region of the input image (its receptive field), and the network outputs class scores for that region. Because of pooling and striding, the heatmap has lower spatial resolution than the input image.

Applying softmax independently at each spatial position gives a K-channel output tensor (one channel per class). The predicted label map is obtained by taking arg max along the class dimension:

$$\hat{Y}(u, v) = \arg\max_{c \in \mathcal{C}} \text{softmax}_c(h_{u,v}) \,,$$

where $h_{u,v} \in \mathbb{R}^K$ is the logit vector at position (u, v).

PyTorch: Migrating a Pre-trained CNN to FC-CNN

Concretely, once a CNN classifier is trained, its FC weights can be transferred directly into an equivalent convolutional layer as follows:

```
# dense1 is the FC layer in the classifier;
# conv1x1 is the corresponding layer in the FC-CNN.
# L = number of output neurons; C = input depth to dense1.
w = dense1.weight.data.clone()    # shape (L, C*H'*W')
b = dense1.bias.data.clone()

# Reshape to convolutional filter tensor (L, C, H', W')
w1 = w.view(L, C, 1, 1)    # if H'=W'=1 already collapsed

conv1x1.weight.data = w1
conv1x1.bias.data   = b
```

Code 10: Migrating a dense layer to a 1×1 convolutional layer in PyTorch

The resulting FC-CNN is far more efficient than the patch-extraction approach: convolutional computations in overlapping regions are *shared* across all output positions rather than repeated.

Limitations of FC-CNN for Segmentation

The FC-CNN produces heatmaps whose spatial resolution is much smaller than the input image — typically by a factor of 16 or 32, determined by the cumulative stride of all pooling and strided-convolution layers. The resulting predictions are very coarse and cannot capture fine object boundaries. There are three strategies to recover full-resolution predictions:

Direct upsampling. Simply bilinearly interpolate the low-resolution heatmap back to the original image size. This is fast but yields blurry, spatially imprecise predictions.

Shift-and-stitch. Compute heatmaps for all f^2 possible integer shifts of the input (where f is the downsampling factor), then interleave the resulting maps to reconstruct a full-resolution prediction. This is equivalent to removing strides from pooling layers and rarefying (dilating) subsequent filters, giving the same receptive field at full resolution. While this exploits the full network depth, the upsampling is rigid and does not improve prediction quality over learned upsampling.

Learned upsampling (transpose convolution). Add a learnable decoder that progressively upsamples the heatmap. This is the approach taken by modern segmentation networks and is discussed in depth below.

FC-CNN is not the end of the story

Moving a classifier to an FC-CNN is the conceptual foundation of segmentation networks, but it is *not* sufficient on its own: the output is too low-resolution, and there is no trainable upsampling. FC-CNNs are mainly useful when only *image-level* labels (not pixel-level annotations) are available for training.

13.4 Encoder–Decoder Architectures and Upsampling

The natural solution to the resolution–semantics tension is a two-stage architecture: an **encoder** (contracting path) that progressively downsamples the input to extract rich semantic features at low resolution, followed by a **decoder** (expanding path) that progressively upsamples back to the original spatial resolution. The two halves are symmetric.

Standard Upsampling Primitives

Several operations can increase spatial resolution. We describe them in increasing order of sophistication.

Nearest-neighbour upsampling. The simplest strategy: each input value is *replicated* to fill all positions of its corresponding $s \times s$ output block (where s is the upsampling factor). The result is blocky but fast and parameter-free.

Bilinear interpolation. Rather than replicating values, bilinear interpolation *smoothly blends* neighbouring input values to estimate the content at each output position. It proceeds in two passes of 1-D linear interpolation: first along the horizontal axis, then along the vertical axis (hence *bi*-linear).

Formally, suppose we want to estimate the value at a continuous position (x, y) that falls inside the unit cell defined by the four nearest input grid points

$$Q_{11} = (x_1, y_1), \quad Q_{12} = (x_1, y_2), \quad Q_{21} = (x_2, y_1), \quad Q_{22} = (x_2, y_2).$$

Define the normalised offsets $t = (x - x_1)/(x_2 - x_1)$ and $u = (y - y_1)/(y_2 - y_1)$, both in $[0, 1]$. First, interpolate horizontally at each of the two rows:

$$R_1 = (1 - t)\, Q_{11} + t\, Q_{21}, \qquad R_2 = (1 - t)\, Q_{12} + t\, Q_{22}.$$

Then interpolate vertically between the two results:

$$\hat{f}(x, y) = (1 - u)\, R_1 + u\, R_2.$$

Expanding, this is a weighted average of the four corners with weights equal to the area of the *opposite* sub-rectangle:

$$\hat{f}(x, y) = (1-t)(1-u)\, Q_{11} + t(1-u)\, Q_{21} + (1-t)u\, Q_{12} + tu\, Q_{22}.$$

Bilinear interpolation is smooth, parameter-free, and fast. Its limitation in segmentation is that the weights depend only on *geometry* (how close the query point is to each corner), not on image content. It therefore cannot recover sharp boundaries or fine structures — it always produces a blurry, smoothly varying surface.

Bed-of-nails upsampling. Each input activation is placed at a *single* position inside its $s \times s$ output block (typically the top-left corner), with all other positions set to zero. Unlike nearest-neighbour, this does *not* replicate the value — the output is sparse. Bed-of-nails is not useful on its own, but it is the conceptual stepping stone to transpose convolution: a subsequent learned convolution can then spread and blend the sparse activations in a task-adaptive way.

Max-unpooling. This is the exact inverse of max-pooling and requires information from the encoder. During the downsampling forward pass, each max-pooling operation records not just the maximum value but also its *location* within the pooling window — these are called **pooling switches**. During upsampling, each value is placed back at its recorded location, with zeros elsewhere. Because the precise spatial position of the dominant activation is preserved, max-unpooling produces sharper feature maps than nearest-neighbour or bilinear upsampling. Networks such as SegNet store the pooling switches from each encoder level and pass them to the corresponding decoder level, creating a symmetric encoder-decoder coupling.

Trainable Upsampling: Transpose Convolution

All the primitives above are *non-learnable*: the upsampling kernel is fixed by geometry, not trained from data. The network therefore cannot adapt the reconstruction to the task. The fundamental question is: can we design a learnable operation that, like a standard convolution, is parameterised by a filter that is trained end-to-end, but that *increases* spatial resolution instead of preserving or reducing it?

Transpose convolution (also called *fractionally strided convolution*; colloquially but misleadingly, *deconvolution*) answers this question.

Intuition: reversing a strided convolution. Recall that a standard convolution with stride $s > 1$ *reduces* spatial resolution by a factor of s: it slides a small filter over the input and computes one output value for every s input positions. A transpose convolution does the opposite: for each *input* activation, it *spreads* a scaled copy of the filter across the output, advancing by s output positions per input step. Where multiple spread copies overlap, their contributions are summed. The result is an output that is roughly s times larger than the input.

The zero-insertion view. A cleaner way to see this is to split the operation in two steps:

1. **Zero insertion.** Upsample the input by inserting $s-1$ zeros between every pair of adjacent input values along each axis. An input of size n becomes an intermediate tensor of size $s(n-1)+1$.
2. **Standard (full) convolution.** Apply a regular convolution (with no stride, full padding) of kernel size k to the zero-inserted tensor. This replaces each zero with a learned weighted combination of its neighbours, effectively filling in the gaps.

The kernel in step 2 is the *learned* parameter. Training it end-to-end allows the network to discover the best way to fill in the missing spatial detail for the task at hand.

Output size formula. For a 1-D input of size n, kernel size k, stride s, and zero-padding p applied to the output, the output size is:

$$n_{\text{out}} = s(n-1) + k - 2p.$$

The most common setting for $2\times$ upsampling uses $s = 2$, $k = 2$, $p = 0$, giving $n_{\text{out}} = 2n$.

The matrix-transpose connection (origin of the name). A standard convolution can be written as a matrix multiplication: $\mathbf{y} = C\,\mathbf{x}$, where C is a sparse Toeplitz matrix whose non-zero entries are the filter weights. A strided convolution has more columns than rows in C — it maps a large input to a smaller output. Backpropagating through this operation requires multiplying by $C^\top$, which maps from the smaller output space back to the larger input space. A transpose convolution is exactly this operation $C^\top\mathbf{x}$ applied as a *forward* pass. This is why the operation is called a transpose convolution, and why it is also known as the *backward pass of a standard convolution.*

Transpose Convolution — Summary

Forward pass. For each input activation at position i, multiply the learned filter $F \in \mathbb{R}^{k\times k\times C_{\text{in}}\times C_{\text{out}}}$ by that scalar and add the result to the output at position $s \cdot i$ (with appropriate indexing). Overlapping contributions are accumulated by summation.
Output size (1-D, padding p):

$$n_{\text{out}} = s(n-1) + k - 2p.$$

Equivalently: zero-insert $s-1$ zeros between each pair of input values, then apply a standard full convolution with kernel F.
Matrix view: if the corresponding standard convolution is $\mathbf{y} = C\mathbf{x}$, the transpose convolution computes $C^\top\mathbf{x}$.

Naming confusion

The name "deconvolution" is common in the deep learning literature but is **highly misleading**. In signal processing, deconvolution is the operation that inverts the blurring effect of a known filter — it *undoes* a convolution. Transpose convolution does no such thing: it does not invert or undo any previous convolution, nor does it recover the original signal. It is simply a learnable upsampling layer. The correct names are *transpose convolution*, *fractionally strided convolution*, or *backward strided convolution.*

A typical decoder block pairs a transpose convolution (for upsampling) with standard convolutions (for feature refinement):

$$\underbrace{\text{TransposeConv}(\times 2)}_{\text{upsample}} \rightarrow \underbrace{\text{Conv2D} + \text{BN} + \text{ReLU} \rightarrow \text{Conv2D} + \text{BN} + \text{ReLU}}_{\text{refine features}}$$

By controlling the number of output channels of the transpose convolution and the subsequent layers, one adjusts the channel depth at each decoder stage. The block is repeated until the target spatial resolution is reached.

Upsampling Filters Initialised as Bilinear Kernels

An important practical observation: the zero-insertion view of transpose convolution shows that the learned filter is responsible for *interpolating* the sparse, zero-inserted input. A natural initialisation is therefore the bilinear interpolation kernel — since bilinear interpolation is already a good general-purpose interpolator, it provides a sensible starting point that can then be fine-tuned by gradient descent on the target task. Long et al. (FCN, 2015) showed that this initialisation accelerates training and improves final segmentation quality compared to random initialisation, because the network starts from a state that already produces smooth, reasonable upsampled outputs rather than noise.

Training Loss for Dense Prediction

Because the network produces a per-pixel prediction, the standard **categorical cross-entropy** loss is simply summed (or averaged) over all pixels:

Pixel-wise Cross-Entropy Loss

Let Ω denote the set of all spatial positions in an annotated image, and let $y(i,j) \in \mathcal{C}$ be the ground-truth label at position (i,j). The segmentation loss is:

$$\mathcal{L}(\theta) = \arg\min_{\theta} \sum_{(i,j)\in\Omega} \mathcal{L}_{\text{CE}}(f_\theta(I)_{i,j},\ y(i,j))\,.$$

Each image in a mini-batch already contributes $|\Omega|$ individual loss evaluations, so the gradients are computed over a very large effective batch even with a small number of images. For regression outputs (e.g. depth estimation), the same pixel-wise summation applies with MSE as the per-pixel loss.

For full-image training one must account for potential *class imbalance*: background pixels are far more numerous than foreground classes. Two standard remedies are (i) reweighting the loss by the inverse class frequency w_c, and (ii) drawing a random binary mask $r(i,j) \sim \text{Bernoulli}(p)$ at each iteration to subsample pixels and introduce stochasticity.

13.5 U-Net

U-Net (Ronneberger, Fischer, and Brox, MICCAI 2015) is arguably the most influential architecture for biomedical image segmentation, and one of the most widely adopted encoder–decoder designs across all domains. It won the 2015 ISBI cell-tracking challenge and introduced a key architectural innovation: *skip connections* that concatenate encoder feature maps directly into the decoder.

The network has 23 layers and no fully connected layers at all — predictions emerge from convolutional feature maps alone, making it fully convolutional and therefore applicable to images of arbitrary size (by tiling with overlap).

Contracting Path (Encoder)

The encoder progressively captures context by repeatedly applying:

1. 3×3 convolution + ReLU (no padding, so each convolution slightly reduces spatial size),
2. 3×3 convolution + ReLU,
3. 2×2 max-pooling with stride 2.

At each downsampling step, the number of feature channels is *doubled*. The structure is identical to the feature-extraction front end of standard CNN classifiers — this is precisely why the encoder of a U-Net can be initialised from a pre-trained ImageNet model via transfer learning.

Skip Connections

The decisive innovation of U-Net is the use of **skip connections**: before each max-pooling step, the current feature map (which still retains high spatial resolution) is saved and later *concatenated* (along the channel axis) with the corresponding upsampled feature map in the decoder.

Why skip connections matter

The encoder progressively compresses spatial information to build a semantically rich, low-resolution bottleneck. In doing so, fine boundary detail is lost. The decoder must somehow recover this detail. Skip connections provide a direct, high-resolution "memory" of early encoder activations, allowing the decoder to refine coarse semantic predictions with precise spatial detail. This is the architectural solution to the what/where tension.

Expanding Path (Decoder)

The decoder mirrors the encoder. Each expanding block consists of:

1. Upsampling (transpose convolution that doubles spatial size and halves the channel count),
2. Concatenation with the corresponding cropped encoder feature map via the skip connection,
3. 3×3 convolution + ReLU,
4. 3×3 convolution + ReLU.

Cropping of the encoder feature map before concatenation is necessary because the repeated "no-padding" convolutions in the encoder slightly reduce spatial extent, so the encoder and decoder maps are not exactly the same size.

Network Top and Output

The final layer is a 1×1 convolution that maps the last feature map to a K-channel output (one channel per class). For the original binary segmentation task (cell vs. background) $K = 2$. The output image is smaller than the input by a fixed border — predictions are only made at pixels for which the full context falls within the input image. In practice, a valid prediction for the entire image is recovered by tiling with a sufficient overlap area.

U-Net Block in PyTorch

```
import torch.nn as nn

class UNetBlock(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1),
            nn.ReLU(inplace=True),
```

```
        )
    def forward(self, x):
        return self.conv(x)
```

Code 11: One contracting block of U-Net in PyTorch

`padding=1` with `kernel_size=3` preserves the spatial dimensions after each convolution; the original U-Net used no padding (spatial shrinkage is small), but padding=1 is common in modern implementations.

Training U-Net: Data Augmentation and Weighted Loss

U-Net was trained on only about **30 annotated images**, which would normally be far too few. The key to making this work is *aggressive data augmentation*, in particular **elastic deformations**: random smooth displacement fields are applied to both the raw image and its ground-truth segmentation map simultaneously, generating realistic-looking cell shapes that the network has never seen. This augmentation strategy is domain-specific — elastic deformations would be unrealistic for rigid objects in natural scenes, but are entirely realistic for biological cells.

A second challenge in cell segmentation is the **separation of touching cells of the same class**. Two touching cells have the same label; the thin background strip between them must be correctly classified as background, but it is extremely underrepresented in the training data. U-Net addresses this with a **weighted loss function**:

U-Net Weighted Loss

$$\mathcal{L}(\theta) = \min_{\theta} \sum_{(i,j)\in\Omega} w(i,j)\, \mathcal{L}_{\text{CE}}(f_\theta(I)_{i,j},\, y(i,j))\,,$$

where the per-pixel weight $w(i,j)$ is pre-computed as:

$$w(i,j) = w_c(y(i,j)) + w_0 \exp\left(-\frac{\big(d_1(i,j) + d_2(i,j)\big)^2}{2\sigma^2}\right).$$

- $w_c(y)$ is a class-frequency balancing term (inverse frequency of class y).
- $d_1(i,j)$ and $d_2(i,j)$ are the distances from pixel (i,j) to the borders of the nearest and second-nearest cell instances, respectively.
- The exponential term is large where $d_1 + d_2$ is small — i.e. in the narrow background gap *between two touching cells* — forcing the network to pay special attention to these critical boundary pixels.

The intuition is:

- **Background near a single cell**: d_1 is small but d_2 is large ⇒ moderate weight.
- **Background between two touching cells**: both d_1 and d_2 are small ⇒ very large weight (thin red strip in the weight map).
- **Pixels inside a cell or far from all cells**: d_1 and d_2 are both large ⇒ weight determined mainly by w_c.

Training uses a single full image per batch (each image provides many individual pixel-level loss contributions), runs for about 10 hours, and employs momentum-based SGD with a high momentum

coefficient to stabilise updates.

13.6 Patch-wise vs. Full-Image Training

There are two distinct training regimes for segmentation networks, each with its own trade-offs.

Patch-wise Training

In patch-wise training one prepares a classification dataset: patches are cropped from annotated images and each is assigned the label of its central pixel. A CNN classifier (or the encoder of an FC-CNN) is then trained on this dataset. After training, the FC layers are convolutionalised (Section 13.3) and the upsampling decoder is appended.

Patch-wise training offers one key advantage: *patch resampling*. One can oversample rare classes to compensate for severe class imbalance, making the effective mini-batch distribution approximately uniform across classes. However, it is computationally wasteful: convolutions over heavily overlapping patches are recomputed many times.

Full-Image Training

In full-image (end-to-end) training, the entire segmentation network — encoder, decoder, and all — is trained directly on annotated images. The loss is the sum of per-pixel cross-entropy losses over all pixels in the image:

$$\mathcal{L}(\theta) = \arg\min_{\theta} \sum_{(i,j)\in\Omega} \mathcal{L}_{\text{CE}}(f_\theta(I)_{i,j},\, y(i,j))\,.$$

This is equivalent to patch-wise training where the batch contains *all* receptive fields of the image simultaneously. It avoids redundant convolution computations and naturally handles arbitrary image sizes.

Two limitations arise and their standard remedies are:

1. **Non-random spatial sampling.** Neighbouring pixels in the same image are highly correlated. To inject stochasticity, one applies a random binary mask $r(i,j) \sim \text{Bernoulli}(p)$ and minimises $\sum_{(i,j)} r(i,j)\,\mathcal{L}_{\text{CE}}$.
2. **Class imbalance.** Since one cannot resample patches, one instead weights the loss by class-frequency: $\sum_{(i,j)} w_c(y(i,j))\,\mathcal{L}_{\text{CE}}$.

Long et al. on full-image training

Long, Shelhamer, and Darrell (CVPR 2015) note that whole-image fully convolutional training is equivalent to patch-wise training where the batch contains all receptive fields of the image, but is significantly more efficient because it avoids repeating convolutional computations for overlapping regions.

13.7 Research Application: Semantic Segmentation with Scarce Annotations

A compelling real-world scenario illustrates the challenges that arise in practice. In a biomedical application (developed in collaboration with clinical partners), the goal is to segment anatomical structures in medical images of patients. Two fundamental difficulties compound each other:

- **Scarce dense annotations.** Dense pixel-level labelling takes approximately 2 hours per image, so only a handful of fully annotated images exist.
- **High anatomical variability.** Different patients, and different pathologies within the same diagnostic category, exhibit substantial morphological differences that a model trained on a small sample may not generalise across.

One practical workaround is to train on **grid-sampled patch annotations**: rather than labelling every pixel, an annotator labels the class of the central pixel of patches placed at regular grid positions. This dramatically reduces annotation time while still providing supervised signal over the image. The workflow then proceeds:

1. Crop patches at grid points; assign to each patch the class of its central pixel.
2. Train a CNN classifier on these patches.
3. Convolutionalise the trained classifier (replace FC layers with 1×1 convolutions).
4. Apply shift-and-stitch to obtain full-resolution segmentation maps.

This approach achieves efficient inference from a vanilla classification network trained on fewer than 31 000 patches of size 200×200, with segmentation quality competitive with direct full-annotation methods.

13.8 Recap: Semantic Segmentation

Lecture 11 — Summary (Exam Checklist)

1. **Semantic segmentation**: assign a class label to every pixel; output is $\hat{Y} \in \mathcal{C}^{H \times W}$; differs from instance segmentation (no per-instance distinction).
2. **Plain vanilla**: slide a CNN classifier over all patches — correct but computationally intractable; patch size is a difficult hyperparameter.
3. **FC-CNN**: replace FC layers with equivalent 1×1 convolutions; accepts arbitrary input size; outputs a low-resolution heatmap. Migration is weight-preserving: reshape FC weight matrix to convolutional filter tensor.
4. **Upsampling strategies**: (i) direct bilinear — fast but blurry; (ii) shift-and-stitch — full-resolution but rigid; (iii) learned transpose convolution — flexible, data-driven, recommended.
5. **Transpose convolution**: learnable upsampling; kernel size k, stride s maps $n \to s \cdot n$; initialised as bilinear interpolation; also called fractionally strided convolution (never "deconvolution").
6. **Encoder–decoder**: contracting path (semantic richness) + expanding path (spatial recovery); symmetric; skip connections concatenate encoder and decoder feature maps at matching scales.
7. **U-Net**: 23-layer fully convolutional encoder–decoder with skip connections; trained on ∼30 images with elastic deformation augmentation; weighted loss $w(i,j) = w_c + w_0 \exp(-(d_1 + d_2)^2/2\sigma^2)$ to handle touching cells.
8. **Pixel-wise loss**: sum categorical cross-entropy over all pixels; each image is a mini-batch of $|\Omega|$ loss evaluations.
9. **Full-image vs. patch-wise training**: full-image is more efficient (no redundant convolutions); patch-wise allows class resampling; full-image uses loss weighting and random masking to compensate.

14 Localization and Object Detection

The previous lecture addressed *semantic segmentation*, which assigns a class label to *every* pixel. This lecture takes a different perspective: instead of labelling every pixel, we want to *locate* one or more objects in an image using **bounding boxes** and optionally identify them by category. We will progress through four increasingly general tasks — classification, localization, multi-task classification-localization, and full object detection — before studying the landmark architectures (R-CNN, Fast R-CNN, Faster R-CNN, YOLO, Mask R-CNN, and FPN) that made real-time detection practical.

14.1 A Hierarchy of Visual Recognition Tasks

It is useful to arrange the visual recognition tasks on a spectrum of increasing complexity.

Classification

The input image contains *one* dominant object. The task is to assign a single label $l \in \Lambda$ from a fixed set of categories Λ. The output is a discrete class, and the training signal is categorical cross-entropy.

Localization

The input image still contains a *single* dominant object, but the network must additionally predict a **bounding box** (x, y, w, h) that tightly encloses it. Formally:

$$\mathbf{I} \longrightarrow (x,\, y,\, w,\, h), \qquad \mathbf{I} \in \mathbb{R}^{R \times C \times 3}.$$

Here (x, y) is the top-left corner of the box and (w, h) its width and height. This is a *regression* problem on 4 real-valued coordinates.

Object Detection

There are now *multiple* objects in the image, each requiring a class label and a bounding box. The number of objects is unknown and varies per image:

$$\mathbf{I} \longrightarrow \big\{(x, y, h, w, l)_1,\, \ldots,\, (x, y, h, w, l)_N\big\}.$$

This is the most general and most challenging setting.

14.2 Bounding Box Regression

Training a network to predict a single bounding box is conceptually straightforward: replace the classification head with a *regression head* containing 4 output neurons with *linear activations*, and optimise a regression loss.

Training a Bounding Box Regressor

1. Build a CNN backbone (e.g. a pretrained EfficientNet-B0) and replace the final classifier with a linear layer of size 4, one neuron per coordinate (x, y, w, h).
2. Use a regression loss such as the ℓ_2 (MSE) or ℓ_1 loss between predicted and ground-truth coordinates:
$$\mathcal{R} = \big\| \,(\hat{x}, \hat{y}, \hat{w}, \hat{h}) - (x, y, w, h)\, \big\|_2^2.$$
3. All 4 output neurons use *linear* (identity) activations — **not** softmax.

A minimal PyTorch implementation illustrates the design:

```
class BoxRegressorModel(nn.Module):
    def __init__(self, dropout_rate=0.5):
        super().__init__()
        self.backbone = torchvision.models.efficientnet_b0()
        in_features = self.backbone.classifier[1].in_features
        # 4 output neurons with linear activation for (x, y, w, h)
        self.backbone.classifier = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(in_features, 4)
        )

# Instantiate model, optimizer, and loss
```

```
box_model     = BoxRegressorModel().to(device)
box_optimizer = torch.optim.Adam(box_model.parameters(), lr=
    LEARNING_RATE)
box_criterion = nn.MSELoss()
```

Code 12: Bounding box regressor in PyTorch

Application: Human Pose Estimation

Bounding box regression generalises immediately to predicting *any* set of spatial coordinates. **Human pose estimation** frames the task as regressing the locations of k body joints, yielding a $2k$-dimensional output vector. The seminal DeepPose paper (Toshev & Szegedy, CVPR 2014) trained an AlexNet-like network to predict 14 joint locations (28 output neurons) directly from the full image, using an ℓ_2 regression loss with data augmentation (translations and flips) to reduce overfitting.

Why predict from the full image?

Feeding the full image to the network lets it capture the *global context* of each body joint — the position of the shoulder constrains the likely position of the elbow — without requiring an explicit part model. The approach is simple to design and train, and can be refined by cascading a second network that improves each joint position within a crop centred on the initial estimate.

Modern systems such as **OpenPose** (Cao et al., CVPR 2017) extend this idea to *multi-person* settings using Part Affinity Fields to associate detected joints with individual people, enabling real-time full-body pose estimation.

14.3 Multi-Task Learning: Classification *and* Localization

A natural next step is to train a single network to solve *both* classification and localization simultaneously. This is an instance of **multi-task learning**: the two outputs have different natures (categorical vs. continuous), so the network needs two specialised heads sharing a common backbone.

The Multi-Task Problem

Given an image $\mathbf{I}$, simultaneously predict:

- a class label $l \in \Lambda$ (classification head, softmax output);
- bounding box coordinates (x, y, w, h) (regression head, linear output).

The training set must provide both a label *and* a bounding box annotation for every image. Extended problems involve regression over more complex geometries (e.g. a full human skeleton).

The Multi-Task Loss

Because gradient descent requires a *scalar* objective, the two per-task losses must be merged into one. The standard approach is a **convex combination**:

$$\mathcal{L}(x) \;=\; \alpha\,\mathcal{S}(x) \;+\; (1-\alpha)\,\mathcal{R}(x), \qquad \alpha \in [0,1],$$

where $\mathcal{S}$ is the softmax (categorical cross-entropy) classification loss and $\mathcal{R}$ is the regression loss (e.g. MSE or ℓ_1). The scalar α is a hyperparameter, but with an important subtlety.

Choosing α

α is **not** a standard hyperparameter like learning rate. Changing α changes the *definition* of the loss itself, so its absolute value is not comparable across different values of α. Cross-validation on the combined loss is therefore meaningless. In practice, α is fixed based on prior experience or tuned via a secondary evaluation metric (e.g. mean IoU on a held-out set). It is always preferable to fine-tune both heads *jointly* rather than sequentially: joint training allows the shared backbone to benefit from both learning signals simultaneously.

PyTorch Architecture and Loss

```python
class MultiTaskModel(nn.Module):
    def __init__(self, num_classes=2, dropout_rate=0.5):
        super().__init__()
        backbone = torchvision.models.efficientnet_b0()
        self.features = backbone.features
        self.avgpool  = backbone.avgpool
        in_features   = backbone.classifier[1].in_features
        # Classification head
        self.classifier_head = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(in_features, num_classes)
        )
        # Regression head (bounding box: x1, y1, x2, y2)
        self.regressor_head = nn.Sequential(
            nn.Dropout(dropout_rate),
            nn.Linear(in_features, 4)
        )

    def forward(self, x):
        features     = self.avgpool(self.features(x))
        features     = torch.flatten(features, 1)
        class_output = self.classifier_head(features)
        bbox_output  = self.regressor_head(features)
        return class_output, bbox_output
```

Code 13: Multi-task model with classification and regression heads

```python
cls_criterion = nn.CrossEntropyLoss()    # for classification
box_criterion = nn.MSELoss()             # for regression

class_output, bbox_output = model(img)
cls_loss   = cls_criterion(class_output, label)
box_loss   = box_criterion(bbox_output, box)
# Weighted combination of the two losses
total_loss = lambda_cls * cls_loss + lambda_box * box_loss
```

Code 14: Multi-task loss computation

14.4 Object Detection: the Full Problem

Object detection generalises localization to an *unknown and varying* number of objects per image. The output is a *set* of tuples (x, y, h, w, l), one per detected instance. This creates two fundamental challenges: the network must produce a *variable-length* output, and training requires a loss that simultaneously accounts for missed detections, false positives, and imperfect localization.

Measuring Detection Quality: Intersection over Union

The standard measure for comparing a predicted bounding box $\hat{B}$ to a ground-truth box B is **Intersection over Union (IoU)**:

$$\text{IoU}(B, \hat{B}) = \frac{|B \cap \hat{B}|}{|B \cup \hat{B}|} \in [0, 1].$$

A perfect prediction gives IoU $= 1$; disjoint boxes give IoU $= 0$. Typical evaluation thresholds are IoU ≥ 0.5 (lenient) or IoU ≥ 0.75 (strict).

The overall **detection loss** $\mathcal{L}(y, \hat{y})$ aggregates across all predicted and ground-truth boxes and penalises:

- **Missed objects** (false negatives): ground-truth boxes with no sufficiently overlapping prediction.
- **Spurious detections** (false positives): predictions with low IoU against any ground-truth box.
- **Poor localization**: predictions with correct class but low IoU with the annotation.

14.5 The Sliding Window Approach (Baseline)

The simplest conceivable approach to detection is the **sliding window**: slide a fixed-size window across the image at multiple positions and scales, classify each crop with a pretrained CNN (adding a *background* class), and report windows classified as non-background.

Background class

Unlike pure localization, detection requires a **background** class so the network can output "no object here" for windows that do not contain any instance.

While correct in principle, the sliding window has severe drawbacks:

- **Computational inefficiency**: convolutional computation is repeated independently for every overlapping crop, with no feature sharing whatsoever.
- **Scale ambiguity**: objects can appear at any size, so many window sizes must be tried.
- **Combinatorial explosion**: for a 1000×2000 image and a moderate number of scales and strides, the total number of crops is enormous.

On the positive side, no retraining is required — any pretrained classifier can be repurposed. But the inefficiency of sliding windows motivated the *region proposal* paradigm.

14.6 Region-Based CNN (R-CNN)

R-CNN (Girshick et al., CVPR 2014) introduced the idea of coupling an external **region proposal algorithm** with a CNN feature extractor. The letter R stands for "Regions".

R-CNN Pipeline

1. **Region proposal**: apply an external algorithm (e.g. Selective Search) to generate ∼2000 candidate regions with very high recall but low precision. There is no learning in this step.
2. **Warping**: resize each proposed region to a fixed size (e.g. 227×227) required by the fully connected layers of the CNN backbone.

3. **Feature extraction**: run a pretrained AlexNet on each warped region to obtain a latent feature vector (∼2048-dimensional).
4. **Classification**: train a per-class linear SVM on the extracted features to classify each region (including a background class). The pretrained CNN is fine-tuned over the target classes (21 vs. 1000 of AlexNet) by placing a FC layer after feature extraction.
5. **Bounding box refinement**: train a per-class linear regressor to correct the bounding box estimate from the region proposal algorithm.

Although R-CNN achieved large accuracy gains over the prior state of the art, it suffered from critical limitations:

- **Non-end-to-end training**: the CNN, SVMs, and BB regressors were trained with different objectives (softmax loss, hinge loss, least squares) and separately — no joint gradient flow.
- **Slow training**: features had to be stored on disk; training took ∼84 hours and required substantial disk space.
- **Slow inference**: the CNN had to be evaluated on each of the ∼2000 proposals independently, with *no feature reuse* across overlapping crops. With VGG16 this amounted to ∼47 seconds per image.

Efficiency is crucial in detection

In object detection, inference speed directly determines applicability. A 47 s/image detector is useless for autonomous driving or any real-time scenario. This inefficiency drove the design of Fast R-CNN and Faster R-CNN. As a rule of thumb: if a detection network is too slow, one might prefer training a segmentation network instead.

14.7 Fast R-CNN

Fast R-CNN (Girshick, ICCV 2015) eliminated the redundant per-proposal convolution by reversing the order of operations: the *entire image* is processed once by the convolutional backbone, and region proposals are then cropped directly from the resulting *feature maps* rather than from the original image.

Fast R-CNN Pipeline

1. Feed the whole image to a CNN backbone (e.g. VGG16 up to `conv5`) to compute a single set of convolutional feature maps. Convolutional computation is performed **once** and **shared** across all proposals.
2. Project the region proposals (still from an external algorithm) onto the feature maps.
3. **ROI Pooling**: convert each variable-size region of interest into a fixed-size $H \times W$ feature map (see the full explanation below).
4. Feed the pooled features through FC layers that simultaneously predict:
   - class probabilities via softmax (replacing the external SVM);
   - bounding box offsets via a linear regression head.
5. The combined multi-task loss is optimised **end-to-end** — back-propagation flows through the ROI Pooling layer and through the shared backbone.

ROI Pooling in Detail

The problem ROI Pooling solves. After the backbone, we have a spatial feature map of fixed stride (e.g. 1/16 of the input resolution for VGG16). Each region proposal, originally a bounding box on the *image*, maps to a rectangle on this feature map — but the rectangles have different sizes, because different proposals have different sizes. The fully-connected layers that follow require a *fixed-length* input vector. ROI Pooling bridges this mismatch.

ROI Pooling

Let a region of interest (ROI) project onto a rectangle of height h and width w on the convolutional feature map. **ROI Pooling** converts this $h \times w$ region into a fixed $H \times W$ output by the following procedure:

1. **Divide into bins.** Partition the $h \times w$ feature-map rectangle into an $H \times W$ grid of sub-regions (**bins**). Each bin covers approximately $\lfloor h/H \rfloor \times \lfloor w/W \rfloor$ feature-map cells (integer rounding is used when h or w is not exactly divisible).
2. **Max-pool within each bin.** Compute the maximum activation value over all cells inside each bin. The result is a single scalar per bin.
3. **Assemble the output.** The $H \times W$ grid of max-pooled scalars forms the output feature map of size $H \times W \times C$ (where C is the number of feature channels; the operation is applied identically to each channel).

The output size $H \times W$ is a *fixed hyperparameter* (typically 7×7 in Fast R-CNN). Regardless of whether the input ROI projects to a 5×10 or 20×30 region on the feature map, the output is always $7 \times 7 \times C$, which is then flattened to a fixed-length vector for the FC layers.

Worked example. Suppose $H = W = 2$ (a 2×2 output) and a proposal maps to a 4×6 region on the feature map (single channel, for simplicity). The ROI is divided into a 2×2 grid of bins, each of size 2×3 cells:

Step 1 — ROI on feature map

$$\begin{pmatrix} 1 & 3 & 2 & 5 & 0 & 1 \\ 4 & 6 & 8 & 2 & 3 & 4 \\ 2 & 1 & 0 & 7 & 1 & 2 \\ 5 & 3 & 4 & 6 & 2 & 0 \end{pmatrix}$$

Step 2 — bins (max-pool each)

$\{1,3,4,6\} \to \mathbf{6}$	$\{2,5,8,2,0,1,3,4\} \to \mathbf{8}$
$\{2,1,5,3\} \to \mathbf{5}$	$\{0,7,4,6,1,2,2,0\} \to \mathbf{7}$

Step 3 — output

$$\begin{pmatrix} 6 & 8 \\ 5 & 7 \end{pmatrix}$$

The 4×6 ROI is split into a 2×2 grid of bins (each 2×3 cells); the maximum within each bin is taken, yielding the fixed 2×2 output.

Why bins matter for gradient flow

Because each output value is the max of a whole bin, the gradient during back-propagation flows only to the *single cell with the maximum value* within each bin (same as in standard max-pooling). This means ROI Pooling is sub-differentiable but fully usable with standard SGD. The size of each bin ($\approx h/H$ cells) changes with the ROI size — larger proposals have larger

bins, so each output neuron summarises a larger receptive field. This is the key mechanism by which ROI Pooling is *size-invariant*: big and small objects both produce the same $H \times W$ representation, allowing a single set of FC weights to classify proposals of any scale.

Quantisation artefacts in ROI Pooling

ROI Pooling requires snapping bin boundaries to integer feature-map coordinates (since activations exist only at discrete grid positions). For small ROIs or precise localisation tasks, this *quantisation* introduces alignment errors. **ROI Align** (used in Mask R-CNN) removes this by computing bin values via bilinear interpolation at non-integer grid positions, yielding significantly better mask and keypoint accuracy.

After Fast R-CNN, the bottleneck shifted entirely to the external region proposal step (e.g. Selective Search), which is not learned and contributes the vast majority of the total test time.

14.8 Faster R-CNN

Faster R-CNN (Ren et al., NIPS 2015) resolved the remaining bottleneck by replacing the external region proposal algorithm with a **Region Proposal Network (RPN)** that operates on the *same* convolutional feature maps as the detection head, enabling the full pipeline to be trained end-to-end.

Region Proposal Network (RPN)

The RPN is a small fully-convolutional network that slides over the last convolutional feature map (spatial size $r_b \times c_b$) and, at each spatial location, predicts k **anchor boxes** — candidate regions of predefined scales and aspect ratios (e.g. $k = 9$: 3 scales $\times$ 3 aspect ratios).

Anchor Boxes

An **anchor box** A_i is a reference bounding box of a specific size and aspect ratio centred at a given spatial location. For each of the k anchors per location, the RPN predicts:

- **Objectness score**: the probability that the anchor contains *any* object (object vs. background). The network outputs $2k$ probability estimates per spatial location via sigmoid activations.
- **Bounding box refinement**: four correction deltas $(\delta x, \delta y, \delta w, \delta h)$ to adjust the anchor toward the true object boundary. The $4k$ corrections per location are expressed in log space to ease convergence.

RPN Architecture

1. An **intermediate 3×3 convolutional layer** (512 filters) embeds each spatial location into a latent representation; output shape $r_b \times c_b \times 512$.
2. A **cls branch** (stack of 1×1 convolutions) outputs $2k$ objectness probabilities per location (or equivalently k sigmoid scores).
3. A **reg branch** (1×1 convolutions) outputs $4k$ bounding box delta estimates per location.
4. **Non-maximum suppression (NMS)**: proposals are sorted by objectness score, redundant highly-overlapping proposals are suppressed by IoU thresholding, and only the top n_{prop} proposals (typically $\sim$300) are passed to the detection head.

Detection Head

The detection head is identical to Fast R-CNN:

- ROI Pooling (or the more precise *ROI Align*) maps each of the n_{prop} proposals to a fixed-size feature vector.
- A **cls head** outputs L class scores including a background class.
- A **bb head** outputs $4 \times (L-1)$ bounding box corrections — one 4-tuple per non-background class.

Training

Faster R-CNN is trained with **four losses** combined in a weighted sum:

1. RPN objectness classification loss.
2. RPN bounding box regression loss.
3. Detection head classification loss.
4. Detection head bounding box regression loss.

The recommended four-step alternating training procedure is:

1. Train RPN alone (frozen backbone; multi-task cls + reg loss).
2. Train Fast R-CNN detection head using proposals from the trained RPN (fine-tune entire network including backbone).
3. Fine-tune RPN cascaded to the updated backbone.
4. Freeze backbone and RPN; fine-tune only the detection head layers.

Inference and Summary

At test time the top ∼300 anchors by objectness score are fed to the detection head. Faster R-CNN achieves ∼0.2 seconds per image on VGG16, compared to 47 s for R-CNN, while also improving accuracy. It remains a **two-stage detector**: stage 1 produces proposals (backbone + RPN), stage 2 classifies them (ROI Pooling + detection head).

Why rectangular anchors?

In principle anchors could have arbitrary shapes (ellipses, rotated rectangles). In practice, axis-aligned rectangular boxes are universally preferred because computing their IoU has a simple closed-form expression, making the loss fast to evaluate and differentiate. Non-rectangular shapes do not admit such a simple intersection formula.

14.9 One-Stage Detectors: YOLO and SSD

R-CNN-based methods are inherently sequential: proposals are generated, then classified. **One-stage (single-shot) detectors** collapse both stages into a single forward pass, trading some accuracy for significant speed gains.

The two landmark one-stage architectures are **YOLO** (You Only Look Once, Redmon et al., CVPR 2016) and **SSD** (Single Shot MultiBox Detector, Liu et al., ECCV 2016).

YOLO

YOLO reframes detection as a single regression problem from raw image pixels to bounding box coordinates and class probabilities, solved all at once by a single large CNN.

YOLO Pipeline

1. Divide the image into a coarse $S \times S$ grid (e.g. 7×7).
2. Associate B anchor boxes with each grid cell.
3. For each of the $S \times S \times B$ anchors, predict:
   - the bounding box offset $(dx, dy, dh, dw, \text{objectness\_score})$ relative to the anchor;
   - the classification score vector over C categories (including background).
4. The final output tensor has shape $S \times S \times B \times (5 + C)$.
5. The entire prediction is computed in a **single forward pass** through one convolutional network.

YOLO's design shares its foundations with the RPN in Faster R-CNN, but extends it to predict class scores directly and performs everything in one shot. The empirical trade-off is well established: two-stage detectors are more accurate; one-stage detectors are faster but historically less accurate. Later versions (YOLOv3 onward) significantly closed this accuracy gap.

14.10 Instance Segmentation

Instance segmentation goes beyond bounding boxes: for each detected object, the network must produce a *pixel-level mask* indicating exactly which pixels belong to that instance. It combines:

- **Object detection**: multiple instances, each with a bounding box and class label.
- **Semantic segmentation**: fine-grained pixel masks, but assigned to *individual instances* rather than merged into a single class map.

Formally, the output per image is $\{(x, y, h, w, l, S)_i\}$ where S_i is the set of foreground pixels within bounding box i.

Mask R-CNN

Mask R-CNN (He, Gkioxari, Dollár & Girshick, ICCV 2017) extends Faster R-CNN by adding a third parallel output branch — the **mask head** — alongside the existing classification and bounding box branches.

Mask R-CNN Architecture

1. **Shared backbone + RPN**: identical to Faster R-CNN; produces n_{prop} region proposals.
2. **ROI Align**: a more precise variant of ROI Pooling that uses bilinear interpolation to avoid quantization artifacts; produces a fixed 14×14 feature map per ROI.
3. **Classification head**: predicts C class scores (as in Faster R-CNN).
4. **Bounding box head**: predicts $4C$ box corrections (as in Faster R-CNN).
5. **Mask head**: a small FCN applied to each ROI that predicts a 28×28 binary mask *for each of the C classes* independently. During training only the mask for the ground-truth class

contributes to the mask loss, decoupling mask prediction from classification.

The total training loss is:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{mask}},$$

where $\mathcal{L}_{\text{mask}}$ is a per-pixel binary cross-entropy loss averaged over the ROI.

A key design insight is that the mask branch is *independent* of the classification branch: the network predicts a mask for every class and selects the appropriate one at inference time, avoiding competition between classes during training.

Mask R-CNN can further incorporate additional heads for related tasks. For instance, by adding a keypoint prediction branch, the same architecture simultaneously performs instance segmentation *and* human pose estimation. Trained on the Microsoft COCO dataset (200,000 images, 80 categories, per-person joint annotations), the unified model achieves compelling results on all tasks at once.

14.11 Feature Pyramid Networks (FPN)

A fundamental challenge in detection is **scale variation**: objects can appear at vastly different sizes in the same image. A backbone CNN naturally produces feature maps of decreasing spatial resolution and increasing semantic richness as depth grows. Shallow layers have high spatial resolution but low semantic content; deep layers have rich semantics but coarse spatial resolution.

Feature Pyramid Networks (Lin et al., CVPR 2017) build a *multi-scale feature hierarchy* that combines the best of both worlds.

Motivation

Early detection systems (e.g. Fast R-CNN) used features from a single convolutional scale, which offers a good speed/accuracy trade-off but struggles with small objects. Multi-scale detection consistently performs better, especially at small scales — but naïvely running the backbone at multiple input resolutions is prohibitively slow. FPN achieves multi-scale feature richness *within a single forward pass.*

Architecture

FPN has three components operating on the same backbone:

1. **Bottom-up pathway**: the standard forward pass of the backbone (e.g. ResNet). Feature maps $\{C_2, C_3, C_4, C_5\}$ are extracted at each resolution stage, each halved in spatial size relative to the previous.
2. **Top-down pathway**: starting from the deepest map C_5, features are progressively *upsampled* by a factor of 2 at each level.
3. **Lateral connections**: at each level, a 1×1 convolution reduces the channel dimension of the bottom-up features to match the top-down features, and the two are added *element-wise.* A 3×3 convolution is then applied to reduce aliasing. This produces the output pyramid $\{P_2, P_3, P_4, P_5\}$, each carrying strong semantics *and* high resolution.

The 1×1 convolutions in the lateral connections are needed to align channel dimensions; "+" denotes element-wise addition.

ROI-to-Level Assignment

A ROI of width w and height h is assigned to pyramid level k according to:

$$k = \left\lfloor k_0 + \log_2\left(\sqrt{wh} \,/\, 224\right) \right\rfloor,$$

where $k_0 = 4$ is the reference level for a 224×224 ROI. Larger objects are assigned to coarser (deeper) pyramid levels; smaller objects to finer (shallower) levels. ROI Pooling is then applied on the selected level P_k.

FPN vs. Other Multi-Scale Architectures

Architectures such as U-Net also combine features at multiple resolutions via skip connections, but they produce a *single* output feature map for inference. FPN, by contrast, makes *independent predictions at each pyramid level*, which is essential for detecting objects across different scales. FPN is not a detector by itself; it is a *feature extractor backbone* that plugs into any region-based detector (RPN, Fast R-CNN, Faster R-CNN) to give it multi-scale capability without significantly increasing test-time cost.

14.12 Recap: Localisation and Object Detection

Lecture 12 — Summary (Exam Checklist)

1. **Task hierarchy**: classification → localization (single BB) → multi-task classification + localization → object detection (multiple instances, variable count).
2. **Bounding box regression**: 4-neuron linear head; ℓ_1 or ℓ_2 regression loss; generalises to pose estimation ($2k$ outputs for k joints).
3. **Multi-task loss**: $\mathcal{L} = \alpha\,\mathcal{S} + (1-\alpha)\,\mathcal{R}$; α is not a standard hyperparameter (loss scale changes with α); always train both heads jointly.
4. **IoU**: $\text{IoU}(B, \hat{B}) = |B \cap \hat{B}| / |B \cup \hat{B}|$; universal metric for bounding box quality; backbone of NMS.
5. **Sliding window**: conceptually correct, computationally intractable; no feature sharing; scale ambiguity.
6. **R-CNN**: external region proposals (Selective Search) + per-proposal CNN + SVM + BB regressor; non-end-to-end; ∼47 s/image.
7. **Fast R-CNN**: one backbone forward pass over the whole image; ROI Pooling extracts fixed-size features from shared feature maps; end-to-end training; bottleneck moves to the region proposal step.
8. **Faster R-CNN**: RPN replaces external proposals; k anchors per location with objectness + BB delta heads; NMS retains top n_{prop} proposals; full end-to-end training with 4 losses; ∼0.2 s/image; two-stage detector.
9. **YOLO/SSD**: one-stage detectors; single forward pass; output tensor $S \times S \times B \times (5+C)$; faster but historically less accurate than two-stage methods.
10. **Mask R-CNN**: extends Faster R-CNN with a mask head (per-ROI binary mask per class); ROI Align instead of ROI Pooling; simultaneous bounding box + class + mask prediction; extensible to pose estimation.
11. **FPN**: bottom-up backbone + top-down upsampling + lateral (1×1) connections; output pyramid $\{P_k\}$; ROIs assigned to levels by size; multi-scale detection in a single forward pass.

15 Recurrent Neural Networks

15.1 Motivation: From Static to Sequential Data

All the architectures we have discussed so far — perceptrons, fully connected networks, convolutional networks — operate on *static* inputs: each example $\mathbf{x}$ is fed to the network independently, and there is no notion of time or order among examples. This assumption is natural when dealing with, say, individual images or fixed-length feature vectors.

Many real-world problems, however, involve data that is *sequential* by nature. Consider natural language: the meaning of a word depends on the words that preceded it. Consider audio: a phoneme carries meaning only in the context of adjacent phonemes. Consider video, financial time-series, biological signals — in all these domains the data arrives as a sequence $X_0, X_1, X_2, \ldots, X_t$ and the order among observations is not merely incidental but is the very carrier of information.

Formally, a sequential dataset is a collection of variable-length sequences

$$\mathcal{D} = \left\{ (\mathbf{X}^{(n)}, \mathbf{y}^{(n)}) \right\}_{n=1}^{N}, \qquad \mathbf{X}^{(n)} = (X_0^{(n)}, X_1^{(n)}, \ldots, X_{T_n}^{(n)}),$$

where T_n may differ across examples. Each element $X_t \in \mathbb{R}^I$ is an I-dimensional observation at time step t.

The central modelling challenge is: how do we incorporate the *history* of past observations when predicting the output at time t?

15.2 A Taxonomy of Sequential Models

Two broad families of approaches exist for handling sequential data, distinguished by whether they maintain an internal *memory* across time steps.

Memoryless Models

These methods represent the entire history via a fixed-size window of past observations and process each window independently.

Autoregressive Models

An **autoregressive model** of order p predicts the next observation from the p most recent ones using *delay taps*:

$$\hat{X}_t \;=\; f\big(X_{t-1},\, X_{t-2},\, \ldots,\, X_{t-p}\big),$$

where f is a linear or nonlinear function. The model has no internal state — if p is fixed, only a finite window of history is used.

Feedforward Neural Networks on Sequences

A natural generalisation of autoregressive models replaces the linear function f with a multi-layer feedforward network. The inputs are the p delay-tapped observations; hidden layers learn nonlinear combinations of the window. While this gives greater representational power, the fundamental limitation remains: the context window is fixed at p steps, and any dependency beyond p time steps is invisible to the model.

Models with Memory

Rather than truncating history to a window, these models maintain a *hidden state* $\mathbf{h}_t$ (also called the *context* or *cell state*) that is updated at every time step and implicitly encodes a summary of the entire past. The output at time t is then a function of both X_t and $\mathbf{h}_{t-1}$.

Three classical architectures belong to this family:

- **Linear Dynamical Systems (LDS).** The hidden state $\mathbf{h}_t$ is real-valued and Gaussian, and all transitions are linear. The LDS is tractable by Kalman filtering but incapable of modelling nonlinear dynamics.
- **Hidden Markov Models (HMM).** The hidden state is *discrete*, taking values in a finite set of K states. State transitions are governed by a stochastic transition matrix $A_{ij} = P(h_t = j \mid h_{t-1} = i)$, and observations are stochastic functions of the hidden state. Efficient inference is possible via the Viterbi or forward-backward algorithms. However, the discrete state space limits the information that can be stored.
- **Recurrent Neural Networks (RNNs).** The hidden state is *real-valued and high-dimensional*, updated by a learned *nonlinear* function. This combines the continuous representation of LDS with the nonlinear dynamics of neural networks, yielding a deterministic system capable of storing rich contextual information.

Why RNNs?

The key advantage of RNNs over both LDS and HMMs is the combination of (i) a *distributed* real-valued hidden state that can store exponentially more information than a discrete state of the same size, and (ii) *nonlinear* state update dynamics that allow the network to learn complex temporal patterns. As Siegelmann (1995) showed, with sufficient neurons and time an RNN can compute anything that can be computed by a Turing machine.

15.3 Recurrent Neural Networks: Architecture

An RNN processes a sequence one step at a time, maintaining a hidden (context) state $\mathbf{c}_t \in \mathbb{R}^B$ that is passed from one time step to the next. At each step t, the network reads the current input $\mathbf{x}^t \in \mathbb{R}^I$ and updates its state:

RNN Forward Pass Equations

Let B be the number of context units and J the number of hidden units. Define:

$$h_j^t = g_h\left(\sum_{i=0}^{I} w_{ji}^{(1)} x_i^t + \sum_{b=0}^{B} v_{jb}^{(1)} c_b^{t-1}\right), \tag{1}$$

$$c_b^t = g_c\left(\sum_{i=0}^{I} w_{bi}^{(1)} x_i^t + \sum_{b'=0}^{B} v_{bb'}^{(1)} c_{b'}^{t-1}\right), \tag{2}$$

$$\hat{y}_n^t = g_{\text{out}}\left(\sum_{j=0}^{J} w_{nj}^{(2)} h_j^t + \sum_{b=0}^{B} v_{nb}^{(2)} c_b^t\right), \tag{3}$$

where $w^{(1)}, w^{(2)}$ are input-to-hidden and hidden-to-output weight matrices, $v^{(1)}, v^{(2)}$ are recurrent weight matrices coupling the context state to hidden and output units, and g_h, g_c, g_{out} are element-wise activation functions.

The three equations above describe the full recurrent computation:

1. **Hidden unit activation** (Eq. 1): each hidden unit h_j^t receives a weighted combination of the current input $\mathbf{x}^t$ *and* the previous context state $\mathbf{c}^{t-1}$. This is the point of departure from feedforward networks — the context state serves as a *memory* of the past.
2. **Context state update** (Eq. 2): the context state $\mathbf{c}^t$ is similarly computed from the current input and the previous context. Different cells in $\mathbf{c}$ can specialise to track different temporal features.
3. **Output computation** (Eq. 3): the output $\hat{\mathbf{y}}^t$ depends on both the hidden units $\mathbf{h}^t$ and the context $\mathbf{c}^t$, giving the network a richer view of its current state.

In matrix-vector notation, collecting Eqs. 1–3:

$$\mathbf{h}^t = g_h\big(\mathbf{W}^{(1)}\mathbf{x}^t + \mathbf{V}^{(1)}\mathbf{c}^{t-1}\big), \qquad \mathbf{c}^t = g_c\big(\mathbf{W}_B^{(1)}\mathbf{x}^t + \mathbf{V}_B^{(1)}\mathbf{c}^{t-1}\big), \qquad \hat{\mathbf{y}}^t = g_{\text{out}}\big(\mathbf{W}^{(2)}\mathbf{h}^t + \mathbf{V}^{(2)}\mathbf{c}^t\big).$$

Note

The weight matrices $\mathbf{W}^{(1)}, \mathbf{V}^{(1)}, \mathbf{W}^{(2)}, \mathbf{V}^{(2)}$ are *shared across all time steps*. This parameter sharing is what allows an RNN to process sequences of arbitrary length with a fixed number of parameters, and it is the central reason why RNNs generalise across time.

15.4 Backpropagation Through Time (BPTT)

Training an RNN requires computing gradients of the loss with respect to the shared weight matrices. Because the same weights appear at every time step, the standard backpropagation algorithm must be extended to account for their repeated use — this extension is called **Backpropagation Through Time (BPTT)**.

Unrolling the Network

The key idea is to *unroll* (or *unfold*) the recurrent computation graph in time. For a sequence of length T and an unroll depth of $U \leq T$ steps, we create U copies of the RNN cell, one for each time step, all sharing the same weights. The unrolled graph is an ordinary feedforward network with tied weights, and standard backpropagation can be applied to it.

BPTT Algorithm

1. **Forward pass**: unroll the RNN for U steps, computing $\mathbf{h}^t$, $\mathbf{c}^t$, $\hat{\mathbf{y}}^t$ at each step.
2. **Backward pass**: compute gradients of the loss E at each time step; propagate error signals backwards through time using the chain rule.
3. **Gradient accumulation**: since $\mathbf{W}$ appears at every step, the total gradient is the *sum* of contributions from all unrolled copies:

$$\frac{\partial E}{\partial \mathbf{W}} = \sum_{u=0}^{U-1} \frac{\partial E}{\partial \mathbf{W}^{t-u}}.$$

4. **Parameter update**: update the *single* shared weight matrix using the average gradient:

$$\mathbf{W}_B \leftarrow \mathbf{W}_B - \eta \cdot \frac{1}{U}\sum_{u=0}^{U-1} \frac{\partial E}{\partial \mathbf{W}_B^{t-u}}, \qquad \mathbf{V}_B \leftarrow \mathbf{V}_B - \eta \cdot \frac{1}{U}\sum_{u=0}^{U-1} \frac{\partial E^t}{\partial \mathbf{V}_B^{t-u}}.$$

The Vanishing (and Exploding) Gradient Problem

A critical question is: how far back in time can BPTT propagate useful gradient information? To understand the difficulty, consider a simplified single-unit RNN:

$$y^t = g\big(w^{(2)} h^t\big), \qquad h^t = h\big(v^{(1)} h^{t-1} + w^{(1)} x\big).$$

The total gradient of the loss E with respect to weight w summed over the sequence is:

$$\frac{\partial E}{\partial w} = \sum_{t=1}^{S} \frac{\partial E^t}{\partial w}, \qquad \frac{\partial E^t}{\partial w} = \sum_{k=1}^{t} \frac{\partial E^t}{\partial y^t} \frac{\partial y^t}{\partial h^t} \frac{\partial h^t}{\partial h^k} \frac{\partial h^k}{\partial w},$$

where the key term is the product of Jacobians along the sequence:

$$\frac{\partial h^t}{\partial h^k} = \prod_{i=k+1}^{t} \frac{\partial h^i}{\partial h^{i-1}} = \prod_{i=k+1}^{t} v^{(1)}\, h'\big(v^{(1)} h^{i-1} + w^{(1)} x\big).$$

Taking norms:

$$\left\| \frac{\partial h^t}{\partial h^k} \right\| \leq \big(\gamma_v \cdot \gamma_{h'}\big)^{t-k},$$

where $\gamma_v = \|v^{(1)}\|$ and $\gamma_{h'} = \max |h'(\cdot)|$ is the maximum absolute value of the activation derivative.

Vanishing Gradient

If $\gamma_v \cdot \gamma_{h'} < 1$, the product $\left\|\partial h^t / \partial h^k\right\|$ decays *exponentially* in the temporal distance $t - k$. For sigmoid and tanh activations we always have $\gamma_{h'} \leq 1/4$ and $\gamma_{h'} \leq 1$ respectively, so for any reasonable weight scale the product shrinks exponentially. The gradient of E^t with respect to inputs at time step $k \ll t$ is thus effectively zero — the network cannot learn long-range temporal dependencies.
Conversely, if $\gamma_v \cdot \gamma_{h'} > 1$, gradients grow exponentially — the **exploding gradient** problem. In practice, exploding gradients are handled by *gradient clipping* (capping the norm of the gradient vector). Vanishing gradients are the harder problem and require architectural solutions.

Solutions to the Vanishing Gradient Problem

Two principal strategies have been proposed:

1. Use ReLU activations with $v^{(1)} = 1$. With $g(a) = \max(0, a)$, the derivative is $g'(a) = \mathbf{1}_{a>0}$, which is either 0 or 1. Setting the recurrent weight $v^{(1)} = 1$ means the recurrent transition simply *accumulates* the input, and the gradient neither grows nor shrinks. However, this eliminates forgetting and can cause representations to grow without bound.

2. Design memory cells with gated mechanisms (LSTMs, GRUs). The principled solution — described in the next section — is to redesign the recurrent cell so that information can flow through time *without* multiplication by the Jacobian, replacing the additive recurrence with a *gated* mechanism that maintains a fixed-weight internal loop.

15.5 Long Short-Term Memory (LSTM)

The **Long Short-Term Memory** (LSTM) was introduced by Hochreiter & Schmidhuber (1997) specifically to address the vanishing gradient problem. The core idea is elegant: instead of using a standard nonlinear recurrence, the LSTM maintains a **cell state** $\mathbf{C}_t$ that flows through time with only *linear* interactions, controlled by a set of learned **gates**. Because the cell state is updated linearly, gradients can flow through it without exponential decay.

Motivation: The Memory Cell

Think of the cell state as a conveyor belt running through time. Information can be written onto it, kept in it, or erased from it, according to three gating mechanisms:

- A **write gate** (input gate): controls how much of new information is written into the cell.
- A **keep gate** (forget gate): controls how much of the old cell state is retained.
- A **read gate** (output gate): controls how much of the cell state is exposed as the output hidden state.

Because the keep gate creates a direct, multiplication-free path through which gradients can flow from t back to any earlier step, the vanishing gradient is prevented — at least for information that the network learns to preserve.

LSTM Equations

Let $\mathbf{x}_t \in \mathbb{R}^d$ be the input at step t, $\mathbf{h}_{t-1} \in \mathbb{R}^m$ the previous hidden state, and $\mathbf{C}_{t-1} \in \mathbb{R}^m$ the previous cell state. The LSTM update consists of four gates and two state updates:

LSTM Forward Pass

$$\mathbf{f}_t = \sigma\big(\mathbf{W}_f[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_f\big) \quad \text{(forget gate)} \tag{4}$$

$$\mathbf{i}_t = \sigma\big(\mathbf{W}_i[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_i\big) \quad \text{(input gate)} \tag{5}$$

$$\tilde{\mathbf{C}}_t = \tanh\big(\mathbf{W}_C[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_C\big) \quad \text{(candidate values)} \tag{6}$$

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \quad \text{(cell state update)} \tag{7}$$

$$\mathbf{o}_t = \sigma\big(\mathbf{W}_o[\mathbf{h}_{t-1}; \mathbf{x}_t] + \mathbf{b}_o\big) \quad \text{(output gate)} \tag{8}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \quad \text{(hidden state)} \tag{9}$$

where $\sigma(\cdot)$ denotes the sigmoid function, $\odot$ is the Hadamard (element-wise) product, and $[\mathbf{a}; \mathbf{b}]$ denotes vector concatenation.

Let us trace through the logic of each gate:

Forget gate (Eq. 4). The forget gate $\mathbf{f}_t \in [0, 1]^m$ decides which dimensions of the cell state to erase. A value close to 0 means "forget this"; a value close to 1 means "keep this". For instance, in a language model, when the topic shifts from one subject to another, the forget gate can clear irrelevant stored information.

Input gate and candidate values (Eqs. 5–6). The input gate $\mathbf{i}_t$ decides which new information to write into the cell. The candidate values $\tilde{\mathbf{C}}_t$ are a tanh-squashed version of the new content, playing the role of the standard RNN hidden activation. Only the component selected by $\mathbf{i}_t$ is written.

Cell state update (Eq. 7). The cell state is updated by *adding* two terms: (i) the fraction of the old cell state retained by the forget gate, and (ii) the new information admitted by the input gate. Crucially, this update is *additive* in the cell state — the gradient of $\mathbf{C}_t$ with respect to $\mathbf{C}_{t-1}$ is simply $\mathbf{f}_t$ (a diagonal matrix), not a dense Jacobian. If the forget gate keeps values close to 1, gradients flow back through many steps without attenuation.

Output gate (Eqs. 8–9). The output gate $\mathbf{o}_t$ controls how much of the cell state is exposed as the hidden state $\mathbf{h}_t$. The cell state is first passed through tanh to squash it to $[-1, 1]$, then filtered by $\mathbf{o}_t$.

Why LSTMs Solve Vanishing Gradients

In a vanilla RNN the gradient of $\mathbf{h}_t$ with respect to $\mathbf{h}_k$ involves a product of $t-k$ weight matrices multiplied by $t-k$ Jacobians of the activation, each of which can be smaller than 1. In an LSTM the analogous quantity for the *cell state* gradient involves only the element-wise product of forget gate values $\prod_{\tau=k+1}^{t} \mathbf{f}_\tau$. Since the forget gate values are learned, the network can in principle set them close to 1 for dimensions it wants to remember, allowing gradients to flow over hundreds of time steps. This is the formal reason LSTMs can capture long-range dependencies.

Comparison: Vanilla RNN vs. LSTM

A vanilla RNN collapses the cell state and hidden state into a single vector, updated by a single nonlinear transformation: $\mathbf{h}_t = \tanh(\mathbf{W}\mathbf{h}_{t-1} + \mathbf{U}\mathbf{x}_t)$. The LSTM replaces this single vector with two states ($\mathbf{h}_t$ and $\mathbf{C}_t$) and four learned gates that modulate information flow. The key structural difference is the additive cell state update, which provides the gradient highway through time.

15.6 Gated Recurrent Unit (GRU)

The **Gated Recurrent Unit** (GRU), introduced by Cho et al. (2014), is a simplified variant of the LSTM that achieves comparable performance with fewer parameters.

The GRU makes two simplifications relative to the LSTM:

1. It **merges the forget and input gates** into a single **update gate** $\mathbf{z}_t$. The update gate simultaneously controls how much of the old state to discard and how much of the new candidate state to write.
2. It **merges the cell state and hidden state** into a single vector $\mathbf{h}_t$, eliminating the separate cell state.

GRU Forward Pass

$$\mathbf{z}_t = \sigma\big(\mathbf{W}_z[\mathbf{h}_{t-1}; \mathbf{x}_t]\big) \quad \text{(update gate)} \tag{10}$$

$$\mathbf{r}_t = \sigma\big(\mathbf{W}_r[\mathbf{h}_{t-1}; \mathbf{x}_t]\big) \quad \text{(reset gate)} \tag{11}$$

$$\tilde{\mathbf{h}}_t = \tanh\big(\mathbf{W}[\mathbf{r}_t \odot \mathbf{h}_{t-1}; \mathbf{x}_t]\big) \quad \text{(candidate hidden state)} \tag{12}$$

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \quad \text{(hidden state update)} \tag{13}$$

The **reset gate** $\mathbf{r}_t$ decides how much of the previous hidden state contributes to the candidate. When $\mathbf{r}_t \approx 0$, the candidate is computed almost entirely from the current input — the network effectively "forgets" the past for that dimension. The update gate $\mathbf{z}_t$ then interpolates between the old and new state.

GRUs are computationally cheaper than LSTMs (one fewer gate, no separate cell state) and often match LSTM performance on tasks with moderate-length dependencies. On tasks requiring very long-range memory, LSTMs may have an edge.

15.7 Deep and Bidirectional RNNs

Stacked (Deep) LSTMs

Just as feedforward networks benefit from depth, RNNs can be stacked to form **deep recurrent networks**. In a stacked LSTM with L layers, the hidden state of layer ℓ at time t serves as the input to layer $\ell + 1$:

$$\mathbf{h}_t^{(\ell)} = \text{LSTM}\big(\mathbf{h}_{t-1}^{(\ell)},\ \mathbf{h}_t^{(\ell-1)}\big), \quad \ell = 1, \dots, L,$$

with $\mathbf{h}_t^{(0)} = \mathbf{x}_t$. Additional non-recurrent transformations (e.g. ReLU layers) can be inserted between the LSTM layers to increase the model's expressive power. The result is a *hierarchical* representation: lower layers capture local, short-range patterns; higher layers integrate information over longer timescales.

Bidirectional RNNs

A standard RNN processes the sequence from left to right, so the hidden state at time t depends only on $x_1, \dots, x_t$. In many tasks — for example, named entity recognition or machine translation — the prediction at position t benefits from *future* context as well.

Bidirectional RNN

A **Bidirectional RNN (BiRNN)** runs two separate RNNs over the input:

- A **forward** RNN that processes the sequence left-to-right: $\overrightarrow{\mathbf{h}}_t = f(x_1, \dots, x_t)$.
- A **backward** RNN that processes the sequence right-to-left: $\overleftarrow{\mathbf{h}}_t = f(x_T, \dots, x_t)$.

The final representation at each time step is the *concatenation* (or sum) of the two hidden states:

$$\mathbf{h}_t = \big[\overrightarrow{\mathbf{h}}_t\,;\ \overleftarrow{\mathbf{h}}_t\big].$$

Note

Bidirectional processing requires access to the *entire* sequence before computing the representations — it is not applicable to online/streaming prediction tasks where future inputs are unavailable.

Initialising the Hidden State

Every RNN requires an initial hidden state $\mathbf{h}_0$ (and $\mathbf{C}_0$ for LSTMs) before processing the first token.

- **Zero initialisation**: set $\mathbf{h}_0 = \mathbf{0}$. Simple and standard; works well when no prior context is available.
- **Learned initialisation**: treat $\mathbf{h}_0$ as a trainable parameter and optimise it by backpropagating the prediction error all the way back to $\mathbf{h}_0$. This allows the network to find a starting state that, on average over the training distribution, leads to better performance.

15.8 Sequence-to-Sequence Models

Many practical problems require mapping an input sequence of length m to an output sequence of length n, where $m \neq n$ in general. Examples include machine translation (English sentence → French sentence), speech recognition (audio frames → text), and image captioning (image → sentence).

A Taxonomy of Sequence Tasks

Following Karpathy's widely cited survey ("The Unreasonable Effectiveness of RNNs"), we can categorise sequence tasks by the relationship between input and output lengths:

- **One-to-one**: fixed input to fixed output (e.g. image classification). No recurrence required.
- **One-to-many**: fixed input to variable-length output (e.g. image captioning: one image → a sentence of words).
- **Many-to-one**: variable-length input to fixed output (e.g. sentiment analysis: a tweet → positive/negative label).
- **Many-to-many (asynchronous)**: variable-length input to variable-length output (e.g. machine translation: English sentence → French sentence, where the two have different lengths).
- **Many-to-many (synchronised)**: aligned sequences of the same length (e.g. video frame classification: label each frame of a video).

Conditional Language Models

The formal framing of sequence-to-sequence modelling starts from the *language model*: a model of the joint probability of a sequence $P(y_1, y_2, \ldots, y_n)$. By the chain rule of probability:

$$P(y_1, y_2, \ldots, y_n) = \prod_{t=1}^{n} p(y_t \mid y_{<t}),$$

where $y_{<t} = (y_1, \ldots, y_{t-1})$ denotes all tokens generated before step t.

A **conditional language model** extends this to condition on a source input:

$$P(y_1, \ldots, y_n \mid x_1, \ldots, x_m) = \prod_{t=1}^{n} p(y_t \mid y_{<t},\, x_1, \ldots, x_m).$$

The sequence-to-sequence goal is to find the output sequence that maximises this conditional probability:

$$y^* = \arg\max_{y} \; P(y_1, \ldots, y_n \mid x_1, \ldots, x_m).$$

Note

In image captioning, the source $x_1, \ldots, x_m$ is replaced by a single image feature vector $\mathbf{x}$ (e.g. the output of a CNN), giving $P(y_1, \ldots, y_n \mid \mathbf{x})$. The structure of the conditional language model is otherwise identical.

The Encoder–Decoder Framework

The standard architecture for sequence-to-sequence learning is the **encoder–decoder** (or *seq2seq*) model (Sutskever, Vinyals & Le, 2014).

Encoder–Decoder Architecture

1. **Encoder**: an RNN (typically an LSTM) reads the source sequence $\mathbf{x} = (x_1, \ldots, x_m)$ one token at a time. After reading all m tokens, the final hidden state $\mathbf{h}_m^{\text{enc}}$ (and cell state $\mathbf{C}_m^{\text{enc}}$ for LSTMs) is taken as a **context vector** $\mathbf{c}$ — a fixed-dimensional summary of the entire

source sequence.

2. **Decoder**: a separate RNN (typically another LSTM) is initialised with the context vector $\mathbf{c}$ and generates the target sequence one token at a time. At each step t, the decoder takes as input the previously generated token y_{t-1} (or, during training, the ground-truth token via teacher forcing) and produces a probability distribution over the vocabulary $P(y_t \mid y_{<t}, \mathbf{c})$.
3. **Output**: at each decoder step, a softmax layer over the full vocabulary gives the probability of each possible next token.

Training: Teacher Forcing

During training, a ground-truth input–output pair $\langle \mathbf{x}, \mathbf{y} \rangle$ is available. The decoder does *not* feed its own output back as the next input — instead, the ground-truth target token y_{t-1} is fed at each decoder step. This technique is called **teacher forcing** and greatly stabilises training by preventing error accumulation in early stages.

The per-step loss is the cross-entropy between the predicted distribution $\mathbf{p}_t$ and the one-hot ground-truth target $\mathbf{y}_t \in \{0,1\}^{|V|}$:

$$\ell_t(\mathbf{p}_t, \mathbf{y}_t) = -\sum_{i=1}^{|V|} y_t^i \log p_t^i = -\mathbf{y}_t^\top \log \mathbf{p}_t.$$

The total training loss is summed over the entire output sequence:

$$\mathcal{L} = -\sum_{t=0}^{n} \mathbf{y}_t^\top \log \mathbf{p}_t.$$

During *inference*, teacher forcing is not available (the ground truth is unknown). The decoder instead feeds its own output back as the next input, and a decoding strategy must be chosen.

Special Tokens

Practical seq2seq systems rely on a small set of special tokens to manage variable-length sequences in batched training:

Special Tokens in Seq2Seq Training

- `<PAD>`: pads shorter sequences in a batch to the same length, enabling batched matrix operations.
- `<EOS>` (End-of-Sequence): marks the end of a source or target sequence; signals the decoder to stop generating.
- `<SOS>`/`<GO>` (Start-of-Sequence): fed as the first decoder input to trigger generation.
- `<UNK>` (Unknown): replaces out-of-vocabulary tokens to keep the vocabulary manageable.

A standard batch preparation procedure is:

1. Sample `batch_size` pairs of (source, target) sequences.
2. Append `<EOS>` to each source sequence.
3. Prepend `<SOS>` to the target to form the *decoder input* sequence; append `<EOS>` to the target to form the *decoder output* (gold) sequence.

4. Pad to the maximum length in the batch with `<PAD>`.
5. Map tokens to integer indices via a fixed vocabulary; replace out-of-vocabulary tokens with `<UNK>`.

15.9 Decoding Strategies

After training, inference requires selecting a concrete output sequence from the learned conditional distribution. The goal is:

$$y' = \arg\max_{y} \prod_{t=1}^{n} P(y_t \mid y_{<t},\, x_1, \ldots, x_m,\, \theta).$$

Exactly solving this optimisation would require enumerating all possible sequences — exponential in the sequence length. Two practical approximations are standard.

Greedy Decoding

Greedy Decoding

At each time step, select the single most probable token:

$$y'_t = \arg\max_{y_t} \; P(y_t \mid y_{<t},\, \mathbf{x},\, \theta).$$

This approximation replaces the argmax over all sequences with n separate argmax operations, one per step.

Greedy decoding is fast (linear time in the sequence length) but **sub-optimal**: the locally best token at step t may lead to a low-probability continuation, and there is no mechanism to backtrack. A token chosen greedily at step 1 cannot be revised in light of future evidence.

Beam Search

Beam search is a best-first search that trades computation for quality by maintaining the k most probable *partial sequences* (hypotheses) at each step, where k is the **beam width**.

Beam Search

1. Initialise the beam with k copies of the start token.
2. At each step, expand each of the k hypotheses by all $|V|$ possible next tokens, yielding $k|V|$ candidate extensions.
3. Retain only the k extensions with the highest *cumulative* log-probability:

$$\log P(y_1, \ldots, y_t) = \sum_{\tau=1}^{t} \log P(y_\tau \mid y_{<\tau},\, \mathbf{x},\, \theta).$$

4. Terminate when all k hypotheses emit `<EOS>`, or when a maximum length is reached.
5. Return the highest-scoring complete hypothesis.

Beam search with $k = 1$ reduces to greedy decoding. Larger k increases the chance of finding a higher-quality sequence at the cost of k times more computation. Typical values in machine translation are $k \in [4, 10]$.

Beam Search vs. Greedy Decoding

Beam search does not guarantee the globally optimal sequence (that would require exhaustive search), but it consistently outperforms greedy decoding by exploring a diverse set of hypotheses and recovering from locally suboptimal choices. The gain is most pronounced in tasks with long outputs and large vocabularies, such as machine translation and summarisation.

15.10 Practical Tips and Best Practices

- **Gradient clipping**: clip the norm of the gradient vector to a threshold (typically 1–5) before applying the parameter update. This prevents the exploding gradient problem without affecting gradient direction, only magnitude.
- **Truncated BPTT**: instead of unrolling the full sequence (which can be thousands of steps), truncate the backpropagation to U steps. This reduces memory and computation at the cost of ignoring very long-range dependencies.
- **Dropout for RNNs**: standard dropout applied to the recurrent connections destroys the temporal correlations. Use *variational dropout* (Gal & Ghahramani, 2016) or *zoneout*, which apply the same dropout mask across all time steps.
- **Learned initial states**: rather than initialising $\mathbf{h}_0 = \mathbf{0}$, treat $\mathbf{h}_0$ as trainable and optimise it via BPTT. This can improve performance on tasks where the "average" input has a non-trivial initial context.
- **Bidirectionality**: use BiLSTMs whenever the full input sequence is available before prediction is required (e.g. sentiment analysis, NER, reading comprehension).
- **Depth**: stack 2–4 LSTM layers for most tasks. Very deep RNNs may require residual connections between layers (analogous to ResNets) to prevent gradient issues in the depth direction.

15.11 Recap: Recurrent Neural Networks

RNN Lecture — Summary (Exam Checklist)

1. **Motivation**: sequential data requires models with memory; static networks (fixed-window autoregressive models, FFNNs) lose long-range context.
2. **Model taxonomy**: memoryless (autoregressive, FFNN) vs. memory models (LDS: continuous linear; HMM: discrete stochastic; RNN: continuous nonlinear).
3. **RNN equations**: $\mathbf{h}^t = g_h(\mathbf{W}^{(1)}\mathbf{x}^t + \mathbf{V}^{(1)}\mathbf{c}^{t-1})$; context state $\mathbf{c}^t$ updated similarly; weights *shared across time*.
4. **BPTT**: unroll for U steps; gradients are sums over unrolled copies; average gradient used for shared weight update.
5. **Vanishing gradient**: gradient norm $\leq (\gamma_v \cdot \gamma_{h'})^{t-k}$; with sigmoid/tanh this decays exponentially; long-range dependencies cannot be learnt.
6. **LSTM gates**: forget ($\mathbf{f}_t$), input ($\mathbf{i}_t$), candidate ($\tilde{\mathbf{C}}_t$), output ($\mathbf{o}_t$); cell state update $\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t$ is *additive* $\Rightarrow$ gradient highway.
7. **GRU**: simplified LSTM; merges forget and input gates into update gate $\mathbf{z}_t$; no separate cell state; fewer parameters, comparable performance.
8. **Deep & Bidirectional RNNs**: stacking L LSTM layers gives hierarchical representations; BiLSTM uses both forward and backward passes, concatenates hidden states.
9. **Seq2seq / Encoder–Decoder**: encoder LSTM reads source $\rightarrow$ context vector $\mathbf{c}$; decoder LSTM initialised from $\mathbf{c}$, generates target; training uses teacher forcing (feed ground truth y_{t-1}, not $\hat{y}_{t-1}$).
10. **Conditional LM**: $P(\mathbf{y} \mid \mathbf{x}) = \prod_t p(y_t \mid y_{<t}, \mathbf{x})$; total loss $= -\sum_t \mathbf{y}_t^\top \log \mathbf{p}_t$.
11. **Special tokens**: `<PAD>` (batching), `<EOS>` (end signal), `<SOS>` (decoder start), `<UNK>` (OOV).
12. **Greedy decoding**: $y'_t = \arg\max_{y_t} P(y_t \mid y_{<t}, \mathbf{x})$; fast but locally suboptimal; no backtracking.
13. **Beam search**: maintain k best hypotheses; at each step expand all k by $|V|$ tokens, keep top k; $k = 1$ reduces to greedy; improves output quality at $k\times$ cost.

16 Unsupervised Learning: Word Embeddings

16.1 Context: Representations Matter

Before diving into the technical machinery, it is worth stepping back and appreciating *why* the way we represent words fundamentally limits — or enables — everything a model can learn. The performance of any NLP system, be it a chatbot, a document classifier, or an information retrieval engine, is deeply entangled with how its inputs are encoded.

Recall the three major machine learning paradigms. In **supervised learning** we are given labelled pairs $\{(x_i, t_i)\}$ and learn a mapping from inputs to outputs. In **reinforcement learning** an agent learns by interacting with an environment and receiving reward signals. And in **unsupervised learning** — the focus of this chapter — we exploit the statistical regularities in an unlabelled dataset $\mathcal{D} = \{x_1, x_2, \ldots, x_N\}$ to build a useful internal representation of the data, without any explicit supervision signal. Word embedding is a canonical example of unsupervised representation learning: given only a large corpus of raw text, we learn vector representations of words that capture semantic relationships.

The central question is: what makes a *good* representation of a word?

16.2 The Neural Autoencoder: A Prelude

A useful entry point into unsupervised representation learning is the **neural autoencoder**. Understanding its structure motivates why word embeddings take the particular form they do.

Neural Autoencoder

An **autoencoder** is a network trained to reproduce its own input — that is, to learn the identity function:

$$\mathbf{x} \xrightarrow{\text{enc}} \mathbf{h} \xrightarrow{\text{dec}} \hat{\mathbf{x}} \approx \mathbf{x}, \qquad \mathbf{x} \in \mathbb{R}^I,\ \mathbf{h} \in \mathbb{R}^J,\ J \ll I.$$

The bottleneck constraint $J \ll I$ forces the network to learn a compressed representation $\mathbf{h}$ that retains the most informative structure in $\mathbf{x}$.

The training objective combines a **reconstruction loss** with an optional **sparsity regulariser**:

$$E(\mathbf{w}) = \left\| g_i(\mathbf{x} \,|\, \mathbf{w}) - \mathbf{x} \right\|^2 \; + \; \lambda \sum_j \left| h_j \left(\sum_i w_{ji}^{(1)} x_i \right) \right|,$$

where the first term penalises reconstruction error and the second term encourages hidden activations h_j to be close to zero (sparse), so that any active unit carries a meaningful, non-redundant signal. The ideal autoencoder satisfies $\hat{x}_i = g_i(\mathbf{x} \,|\, \mathbf{w}) \approx x_i$ at the output while keeping most $h_j \approx 0$ in the hidden layer.

Applications of Autoencoders

Autoencoders have a surprisingly broad range of uses, all stemming from the learned compressed representation $\mathbf{h}$:

- **Data compression**: the encoder half compresses the input to a compact code; the decoder reconstructs it. Unlike classical codecs, the encoding is learned end-to-end from data.
- **Nonlinear dimensionality reduction**: a nonlinear generalisation of PCA. While PCA finds a linear projection that maximises variance, an autoencoder with nonlinear activations can discover curved low-dimensional manifolds in high-dimensional data.

- **Feature extraction and transfer learning**: the representation $\mathbf{h}$ learned on a large unlabelled dataset can be used as input features for a supervised task with limited labelled data (pre-training).
- **Denoising and anomaly detection**: by training on clean data and then testing with noisy or anomalous inputs, reconstruction error serves as an anomaly score.
- **Generative modelling**: Variational Autoencoders (VAEs) impose a probabilistic prior on $\mathbf{h}$, enabling the generation of new samples by decoding latent vectors drawn from the prior.
- **Word embedding**: as we shall see, learning distributed representations of words is structurally equivalent to training a shallow autoencoder — or, more precisely, a predictive model — over a symbolic vocabulary.

16.3 The Problem with Discrete Word Representations

Natural language processing has traditionally treated words as **discrete atomic symbols**. Each word in a vocabulary V is assigned a unique integer identifier, and at the network input level it is represented as a **one-hot vector**: a vector of length $|V|$ with a single 1 at the position of the word index and 0s everywhere else.

- `"cat"` $\rightarrow$ Id 537 $\rightarrow$ $[\ldots, 0, 1, 0, \ldots]$
- `"dog"` $\rightarrow$ Id 143 $\rightarrow$ $[\ldots, 1, 0, 0, \ldots]$

When words from a text document are combined, the standard aggregation is the **bag-of-words** (BoW) representation: a vector of length $|V|$ counting how many times each word appears in the document, ignoring word order.

This scheme has two severe shortcomings.

The Curse of Dimensionality

Real-world vocabularies have $|V| \sim 10^5$–10^6 unique words. Every one-hot vector therefore lives in a space of dimension 10^5 or more. When input vectors are this sparse and this high-dimensional, the sample complexity of learning grows explosively — this is the **curse of dimensionality**. Most pairwise distances between vectors are nearly equal, nearest-neighbour search becomes meaningless, and any statistical model must estimate parameters in a space that is vastly underpopulated by training examples.

Semantic Blindness

One-hot vectors are orthogonal by construction:

$$\langle \mathbf{e}_{\text{cat}}, \mathbf{e}_{\text{dog}} \rangle = 0, \qquad \langle \mathbf{e}_{\text{obama}}, \mathbf{e}_{\text{president}} \rangle = 0, \qquad \langle \mathbf{e}_{\text{illinois}}, \mathbf{e}_{\text{chicago}} \rangle = 0.$$

Consider the two sentences:

- *Obama speaks to the media in Illinois.*
- *The President addresses the press in Chicago.*

These sentences are semantically near-identical, yet under a one-hot BoW representation they share *zero* lexical overlap: every content word maps to a different, orthogonal dimension. Any model

relying on this representation cannot generalise from one sentence to the other. We say the representation is *semantically blind*: it cannot capture synonymy, hypernymy, or any other relationship between words.

Distributional Hypothesis

The theoretical foundation of word embeddings is the **distributional hypothesis** (Firth, 1957): *"You shall know a word by the company it keeps."* Words that appear in similar contexts tend to carry similar meanings. This means that the co-occurrence statistics of a word with its surrounding words are a rich, fully unsupervised signal for learning semantic representations — without any need for labelled data.

16.4 Word Embedding: The Idea

Word Embedding

A **word embedding** is a mapping $C : V \to \mathbb{R}^m$ that assigns to each word $w \in V$ a dense, real-valued vector $\mathbf{c}_w \in \mathbb{R}^m$ such that words with similar meanings are mapped to nearby vectors in $\mathbb{R}^m$.
Typical parameters: $|V| \sim 10^5$–10^6 (vocabulary size); $m \in [100, 500]$ (embedding dimension), so $m \ll |V|$.

The mapping C is represented as a matrix $\mathbf{C} \in \mathbb{R}^{|V| \times m}$, where row i is the embedding vector of the i-th vocabulary word. Looking up the embedding of word w is therefore just an indexed row access:

$$C(w) = \mathbf{C}[w, :] \ \in \ \mathbb{R}^m.$$

Compared to one-hot representations, embeddings achieve three simultaneous improvements:

- **Compression**: dimension reduces from $|V| \sim 10^5$ to $m \sim 300$ — a reduction of several orders of magnitude.
- **Smoothing**: the discrete vocabulary is mapped to a continuous space, making small perturbations in meaning correspond to small distances in $\mathbb{R}^m$.
- **Densification**: from sparse one-hot vectors to dense real-valued vectors, making every dimension informative.

Semantically related words cluster together in the embedding space. Words like *cat*, *dog*, *rabbit* form a cluster; words like *Paris*, *London*, *Rome* form another; and the geometric relationships *between* clusters encode higher-order semantic structure, as we shall see in Section 16.8.

16.5 Language Modelling: The Backdrop

Most word embedding methods are motivated by, or directly derived from, the task of **language modelling**: estimating the probability of a sequence of words.

Language Model

A **language model** assigns a probability to every sequence of words $s = w_1, w_2, \ldots, w_k$:

$$P(s) = P(w_1, w_2, \ldots, w_k) = \prod_{i=1}^{k} P(w_i \mid w_1, \ldots, w_{i-1}).$$

The factorisation follows the chain rule of probability and is exact.

Computing $P(w_i \mid w_1, \ldots, w_{i-1})$ exactly requires conditioning on an arbitrarily long history — which is computationally intractable. Two classical approximations exist.

N-gram Language Models

The **n-gram approximation** assumes that the probability of a word depends only on the preceding $n-1$ words (the *Markov assumption* of order $n-1$):

$$P(w_i \mid w_1, \ldots, w_{i-1}) \;\approx\; P(w_i \mid w_{i-n+1}, \ldots, w_{i-1}).$$

The model can then be estimated from counts:

$$\hat{P}(w_i \mid w_{i-n+1}, \ldots, w_{i-1}) = \frac{\#(w_{i-n+1}, \ldots, w_{i-1}, w_i)}{\#(w_{i-n+1}, \ldots, w_{i-1})},$$

with smoothing (e.g. Katz back-off, Kneser-Ney) applied to handle unseen n-grams.

The Curse of Dimensionality for N-grams

Consider a 10-gram language model on a vocabulary of $|V| = 100{,}000$ words. The model must assign a probability to each possible 10-word sequence, which lives in a 10-dimensional space where each axis has 10^5 values. The total number of parameters grows as $|V|^{n-1} = 10^{45}$ — an astronomical number that no corpus can cover. In practice:

- Vocabularies reach $|V| \sim 10^6$ unique words.
- Context is limited to $n = 2$ or $n = 3$ (trigram); the famous Katz (1987) smoothed trigram model is a canonical example.
- Even with $n = 3$, most trigrams are unseen in any finite corpus, causing severe sparsity.
- Short contexts mean long-range dependencies are systematically ignored.

Continuous Representations: Beyond N-grams

An alternative family of methods replaces the discrete count-based approach with *continuous* representations of words. They all exploit the **distributional hypothesis**: the meaning of a word can be inferred from the contexts in which it appears. Three major approaches exist, and understanding all three clarifies exactly what distributed neural embeddings add over their predecessors.

Latent Semantic Analysis (LSA). **Latent Semantic Analysis** (Deerwester et al., 1990) is the oldest and most principled of the three. The idea is to build a co-occurrence matrix and then compress it with linear algebra.

Step 1 — Build the term-document matrix. Given a corpus of N documents and a vocabulary of $|V|$ words, construct a matrix $\mathbf{X} \in \mathbb{R}^{|V| \times N}$ where entry X_{ij} is (some function of) how many times word i appears in document j. Typically, raw counts are replaced by **TF-IDF** weights (Term Frequency–Inverse Document Frequency):

$$\text{TF-IDF}(i,j) = \underbrace{\frac{n_{ij}}{\sum_k n_{kj}}}_{\text{term frequency}} \times \underbrace{\log \frac{N}{|\{j : n_{ij} > 0\}|}}_{\text{inverse document frequency}},$$

which up-weights words that are frequent in a document but rare across the corpus (hence informative), and down-weights stop-words like *the*, *is*, that appear everywhere.

Step 2 — Truncated SVD. Decompose the matrix as:

$$\mathbf{X} \approx \mathbf{U}_k \, \mathbf{\Sigma}_k \, \mathbf{V}_k^\top,$$

where $\mathbf{U}_k \in \mathbb{R}^{|V| \times k}$ contains the top-k left singular vectors, $\mathbf{\Sigma}_k \in \mathbb{R}^{k \times k}$ is the diagonal matrix of the k largest singular values, and $\mathbf{V}_k \in \mathbb{R}^{N \times k}$ contains the right singular vectors. This is the best rank-k approximation to $\mathbf{X}$ in the Frobenius norm (Eckart-Young theorem).

Step 3 — Use the rows of $\mathbf{U}_k$. The i-th row of $\mathbf{U}_k$ is the k-dimensional *latent semantic vector* of word i. Words with similar co-occurrence patterns across documents will have similar latent vectors. Typical values: $k \in [100, 300]$.

What LSA captures and what it misses

What LSA captures: words used in similar documents get similar vectors, so synonyms and topically related words cluster together. LSA was the first method to demonstrate that semantic similarity could be extracted automatically from raw text statistics.
What LSA misses:

- **Word order**: $\mathbf{X}$ is a bag-of-words matrix — the order of words within documents is completely discarded.
- **Polysemy**: *bank* (financial) and *bank* (river) collapse into a single average vector. SVD cannot separate different senses of the same word.
- **Non-linearity**: SVD finds the best *linear* low-rank approximation; the true semantic manifold of language is highly non-linear.
- **Scalability**: the SVD of a $10^6 \times 10^6$ matrix is computationally expensive.

LSA vs. word2vec at the exam

LSA builds a word–document matrix and applies SVD. Word2vec builds a word–context matrix implicitly through a neural prediction task. Both produce dense word vectors, but word2vec captures local syntactic patterns (it uses a fixed-size context window) while LSA captures global document-level co-occurrence. LSA has no nonlinearity and cannot learn compositional features.

Latent Dirichlet Allocation (LDA). **Latent Dirichlet Allocation** (Blei, Ng & Jordan, 2003) is a fully *generative probabilistic* model of text, not a matrix factorisation. It assumes documents are mixtures of latent **topics**, and each topic is a probability distribution over words.

Generative story.

1. For each topic $k = 1, \ldots, K$: draw a word distribution $\boldsymbol{\phi}_k \sim \text{Dir}(\boldsymbol{\beta})$ (a distribution over $|V|$ words).
2. For each document d: draw a topic mixture $\boldsymbol{\theta}_d \sim \text{Dir}(\boldsymbol{\alpha})$ (a K-dimensional distribution over topics).
3. For each word position n in document d:
   - Draw a topic $z_{d,n} \sim \text{Multinomial}(\boldsymbol{\theta}_d)$.
   - Draw a word $w_{d,n} \sim \text{Multinomial}(\boldsymbol{\phi}_{z_{d,n}})$.

The Dirichlet prior encourages sparse topic mixtures (a document is about a few topics) and sparse word distributions (a topic uses a few distinctive words), which yields interpretable topics.

Inference. Given a corpus, the goal is to infer the posterior $P(\boldsymbol{\theta}, \mathbf{z}, \boldsymbol{\phi} \mid \mathbf{w})$. This is intractable in closed form and approximated via variational inference or collapsed Gibbs sampling.

Word representation from LDA. Once trained, the topic-word matrix $\mathbf{\Phi} \in \mathbb{R}^{K \times |V|}$ gives a K-dimensional representation for each word (its weight across topics). Alternatively, a word's representation can be taken as the expected topic assignment $P(z \mid w)$.

What LDA captures and what it misses

What LDA captures: document-level thematic structure. Topics are interpretable clusters of words that co-occur across documents (e.g. a *sports* topic might have high probability for *goal*, *player*, *team*, *match*). LDA is the gold standard for topic modelling and produces human-interpretable topics.

What LDA misses:

- **Word order**: same bag-of-words assumption as LSA.
- **Fine-grained syntax**: topics are coarse, document-level concepts. LDA cannot represent that *run* as a verb ("she runs") differs from *run* as a noun ("a run of bad luck").
- **Word analogies**: LDA produces topic distributions, not geometric vectors; you cannot perform analogy arithmetic (king − man + woman ≈ queen).
- **Scalability**: inference is slow on very large corpora compared to word2vec.

Distributed Representations (Neural Word Embeddings). **Distributed representations** — the approach that word2vec and GloVe implement — differ conceptually from both LSA and LDA in a fundamental way: rather than decomposing an explicit co-occurrence matrix, they learn word vectors *as a side effect of training a neural prediction task on raw text.*

Distributed Representation

A representation is **distributed** if the meaning of a concept is encoded across *many* dimensions of a vector simultaneously, and each dimension participates in the encoding of *many* concepts. This is in contrast to a **local** (one-hot) representation, where a single unit encodes a single concept and all other units are zero.

The key properties that distinguish distributed representations from LSA and LDA:

1. **Learned via prediction, not factorisation.** Word2vec trains a shallow neural network to predict a word from its context (CBOW) or a context from a word (Skip-gram). The embedding matrix is not computed analytically — it is found by gradient descent on a prediction loss. This means the vectors encode the information that is *predictively useful* for the task, which turns out to capture both semantic and syntactic regularities.
2. **Non-linear.** Even though word2vec itself has no hidden layer (it is essentially a bilinear model), the *composition* of embeddings with softmax is nonlinear, and deeper models (e.g. NNLM with tanh hidden layers) apply explicit nonlinearities. LDA and LSA are both fundamentally linear.
3. **Local context window.** Word2vec uses a fixed-size window (typically 5–10 words) around each word. This captures syntactic patterns (word order, grammatical roles) that bag-of-words approaches miss entirely.
4. **Geometric regularity (the killer feature).** Distributed representations encode semantic *relationships* as constant *differences* between vectors:

$$C(\text{king}) - C(\text{man}) + C(\text{woman}) \approx C(\text{queen}).$$

   This is an emergent property — it was not engineered in, but arises from training on the prediction task. Neither LSA nor LDA vectors exhibit this clean relational geometry.

Comparison: LSA vs. LDA vs. Distributed Representations

Property	**LSA**	**LDA**	**Dist. Repr.**
Method	SVD of TF-IDF	Generative model + VI	Neural prediction
Context	Document-level	Document-level	Local window
Word order	Ignored	Ignored	Captured (partially)
Nonlinearity	No	Yes (Dirichlet)	Yes (activation)
Polysemy	No	Partial	Partial
Analogy arithmetic	Weak	No	Strong
Interpretability	Low	High (topics)	Low
Scalability	Slow (SVD)	Slow (inference)	Fast (SGD)

Why "distributed" is the right word

In a one-hot vector, the word *dog* is represented by a single active unit. In a distributed representation, *dog* is represented by a pattern of activations across 300 dimensions — some dimensions might encode *animacy*, others *size*, others *domesticity*, others *noun-ness*. No single dimension has a clean interpretation, but the *pattern as a whole* is uniquely identifying and semantically meaningful. This is why distributed representations are exponentially more expressive than local ones: n binary units can represent 2^n distinct concepts distributedly, but only n concepts locally.

16.6 The Neural Network Language Model (Bengio et al., 2003)

Bengio, Ducharme, Vincent & Janvin (2003) proposed the first neural network language model (NNLM) that jointly learned word embeddings and a language model. While their primary goal was to improve language modelling perplexity, the embedding matrix $\mathbf{C}$ they obtained as a by-product turned out to be the real breakthrough.

Architecture

The model predicts the next word w_t given a context of $n-1$ preceding words $w_{t-n+1}, \ldots, w_{t-1}$. The training objective is to minimise the negative log-likelihood:

$$E = -\log \hat{P}(w_t \mid w_{t-n+1}, \ldots, w_{t-1}).$$

The network has four distinct layers, illustrated in the diagram below.

NNLM Architecture (4 Layers)

1. **Input layer** $[(n-1)\cdot|V|]$: the $n-1$ context words are each represented as a one-hot vector of length $|V|$.
2. **Projection layer** $[(n-1)\cdot m]$: each one-hot vector is mapped to its m-dimensional embedding via a shared lookup table $\mathbf{C} \in \mathbb{R}^{|V|\times m}$. The $n-1$ embeddings are *concatenated* into a single vector $\mathbf{x} = [C(w_{t-n+1}); \ldots; C(w_{t-1})]$ of dimension $(n-1)\cdot m$.
3. **Hidden layer** $[h,\ \tanh]$: a fully connected layer with $h \in [500, 1000]$ units and tanh

activation: $\mathbf{a} = \tanh(\mathbf{d} + \mathbf{H}\mathbf{x})$, where $\mathbf{H} \in \mathbb{R}^{h \times (n-1)m}$ and $\mathbf{d} \in \mathbb{R}^h$ is a bias.

4. **Output layer** [$|V|$, softmax]: a linear layer followed by softmax gives the predicted probability distribution over the full vocabulary:

$$\hat{P}(w_i = w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{e^{y_{w_i}}}{\sum_{i'=1}^{|V|} e^{y_{w_{i'}}}}, \qquad \mathbf{y} = \mathbf{b} + \mathbf{U}\mathbf{a},$$

where $\mathbf{U} \in \mathbb{R}^{|V| \times h}$ are the hidden-to-output weights and $\mathbf{b} \in \mathbb{R}^{|V|}$ is the output bias.

The **total parameter count** per gradient step is dominated by $\mathbf{C}$, $\mathbf{H}$, and $\mathbf{U}$, giving a per-update complexity of:

$$\mathcal{O}\big(n \cdot m + n \cdot m \cdot h + h \cdot |V|\big).$$

The crucial observation is that the **projection matrix $\mathbf{C}$ is shared** across all $n-1$ context positions. Every word in the vocabulary has a single m-dimensional embedding that is used regardless of where in the context window the word appears. This weight sharing is what gives the model the ability to generalise: if two words appear in similar contexts, their embedding vectors will be pulled towards similar positions, regardless of whether they appeared in the first, second, or third context slot.

Results and Limitations

Bengio et al. evaluated on two corpora:

- **Brown corpus**: 1.2M words, $|V| \approx 16{,}000$, 200K test set; best model used $h = 100$, $n = 5$, $m = 30$. Training took 3 weeks on 40 CPU cores.
- **AP News**: 14M words, $|V|$ reduced to 18K; $h = 60$, $n = 6$, $m = 100$.

Relative improvements in test-set perplexity over the best smoothed n-gram baseline were 24% (Brown) and 8% (AP News) — a meaningful improvement, but at enormous computational cost.

Bengio's Overlooked Contribution

Bengio et al. (2003) considered their main contribution to be the improved language model perplexity and mentioned the word vectors only as a by-product for future work. In hindsight, the embedding matrix $\mathbf{C}$ was the most impactful output of the paper. It was left to Mikolov et al. (2013) — a decade later — to recognise that one could train *only* for high-quality embeddings, radically simplify the model, and scale to vastly larger corpora.

The principal limitation of the NNLM is its **computational cost**: the $h \times |V|$ output layer requires computing $|V|$ inner products at every gradient step. With $|V| \sim 10^6$ this makes the model impractical for large vocabularies. Moreover, the softmax normalisation requires a sum over the entire vocabulary at every step — an $\mathcal{O}(|V|)$ operation that cannot be avoided without approximation.

16.7 Word2vec (Mikolov et al., 2013)

Mikolov, Chen, Corrado & Dean (2013) proposed **word2vec**, a family of models designed with a single explicit goal: obtain the best possible word embeddings as efficiently as possible. The strategy is to *simplify* the NNLM so aggressively that the model can be trained on a billion-word corpus in less than a day.

Word2vec Design Principles

1. **No hidden layer**: eliminating the tanh hidden layer removes the $n \cdot m \cdot h$ cost entirely. The projection directly feeds into the output.
2. **Shared projection layer**: unlike the NNLM, where the projection layer is specific to each context position, word2vec shares a single m-dimensional representation for each word regardless of context position.
3. **Symmetric context**: the context window includes words from both the *past* and the *future* of the target word, not just the preceding words.
4. **Hierarchical softmax or negative sampling**: approximate the softmax output to reduce complexity from $\mathcal{O}(h \cdot |V|)$ to $\mathcal{O}(m \cdot \log |V|)$ (hierarchical softmax) or $\mathcal{O}(m \cdot k)$ (negative sampling with k noise samples).

Word2vec comes in two architectural flavours, both shown in Figure **??**.

Skip-gram

The **skip-gram** model takes a single word w_t as input and tries to predict the $n/2$ words to its left and the $n/2$ words to its right:

$$\text{objective:} \quad \max_{\theta} \sum_{t} \sum_{\substack{-n/2 \le j \le n/2 \\ j \neq 0}} \log P(w_{t+j} \mid w_t; \theta).$$

Architecturally: the input word w_t is looked up in the embedding matrix $\mathbf{C}$ to yield an m-dimensional vector. This *single* vector is projected to n output distributions over $|V|$ words (one for each context position). The mapping is **one-to-many**: from one centre word, predict many surrounding context words.

Skip-gram is particularly good at representing *rare* words: since each training example focuses on predicting the context of one word, even a word that appears rarely in the corpus will have its embedding updated each time it is the centre word.

Continuous Bag-of-Words (CBOW)

The **CBOW** model inverts the direction: it takes the $n-1$ surrounding context words as input and predicts the single target word w_t at the centre.

CBOW Architecture

For each training pair $(\text{context}, w_t)$ where $\text{context} = (w_{t-n/2}, \ldots, w_{t-1}, w_{t+1}, \ldots, w_{t+n/2})$:

1. **Input layer** $[|V|]$: a combined one-hot vector of length $|V|$ with 1s at the positions of all n context words.
2. **Projection layer** $[m]$: the shared embedding matrix $\mathbf{C} \in \mathbb{R}^{|V| \times m}$ is applied; the resulting context word vectors are *averaged* (not concatenated as in NNLM):

$$\mathbf{p} = \frac{1}{n} \sum_{j \in \text{context}} C(w_j).$$

3. **Output layer** $[|V|,$ hierarchical softmax$]$: the averaged projection $\mathbf{p}$ is multiplied by an output embedding matrix $\mathbf{C}' \in \mathbb{R}^{m \times |V|}$ to produce a score per vocabulary word; hierarchical softmax gives the probability $\hat{P}(w_t \mid \text{context})$.

Objective: minimise $E = -\log \hat{P}(w_t \mid \text{context})$.

Gradient update intuition. The key quantity is the prediction error at the output. For each $\langle \text{context}, w_t \rangle$ pair:

- If $\hat{P}(w_i = w_t \mid \text{context})$ is **overestimated** (the model is too confident about the wrong word), a portion of the output embedding $C'(w_i)$ is *subtracted* from the context word vectors in $\mathbf{C}$, pushing those context embeddings away from $C'(w_i)$.
- If $\hat{P}(w_i = w_t \mid \text{context})$ is **underestimated**, a portion of $C'(w_i)$ is *added* to the context word vectors, pulling them closer to $C'(w_i)$.

Note that only the context word vectors (not the target word embedding) are updated during the backward pass for CBOW. After training, the learned input embedding matrix $\mathbf{C}$ is the primary word embedding used downstream; $\mathbf{C}'$ is discarded.

Complexity and Empirical Performance

The per-update complexity of word2vec is:

$$\mathcal{O}(n \cdot m + m \cdot \log |V|),$$

a dramatic reduction from the NNLM's $\mathcal{O}(n \cdot m \cdot h + h \cdot |V|)$.

On the Google News corpus (6 billion words, $|V| \sim 10^6$):

- **CBOW** with $m = 1000$: 2 days on 140 CPU cores.
- **Skip-gram** with $m = 1000$: 2.5 days on 125 CPU cores.
- **NNLM** (Bengio et al.) with $m = 100$ only: 14 days on 180 CPU cores — and with a smaller embedding!

Training speed reached 10^5–5×10^6 words per second. In terms of word analogy accuracy (see Section 16.8): the best NNLM achieved 12.3% overall accuracy while word2vec with skip-gram reached **53.3%** — more than a four-fold improvement. The ability to train on orders of magnitude more data was the decisive factor.

16.8 Geometric Regularities in the Embedding Space

The most striking property of word2vec embeddings is that **semantic and syntactic relationships are encoded as linear vector offsets**. This is not merely an aesthetic curiosity — it means that arithmetic on embedding vectors corresponds to reasoning about meaning.

Constant Difference Vectors

When the embedding vectors for a set of related word pairs are plotted (e.g. using PCA to project to 2D), the difference vector $C(w_{\text{capital}}) - C(w_{\text{country}})$ is approximately *constant* across all country–capital pairs:

$$C(\text{Beijing}) - C(\text{China}) \;\approx\; C(\text{Paris}) - C(\text{France}) \;\approx\; C(\text{Rome}) - C(\text{Italy}) \;\approx\; \cdots$$

Similarly, the male–female gender difference vector is approximately constant:

$$C(\text{queen}) - C(\text{king}) \;\approx\; C(\text{woman}) - C(\text{man}) \;\approx\; C(\text{actress}) - C(\text{actor}) \;\approx\; \cdots$$

Analogy Completion by Vector Arithmetic

These constant-offset properties enable **analogy completion**: given an analogy of the form "a is to b as c is to ?", the answer is found by the vector operation:

$$w^* = \arg\min_{w \in V} \big\| C(w) - \big(C(b) - C(a) + C(c)\big) \big\|,$$

i.e. find the word whose embedding is closest to the vector $C(b) - C(a) + C(c)$. Examples:

Analogy	**Vector expression**	**Result**
king : man → queen : woman	$C(\text{king}) - C(\text{man}) + C(\text{woman})$	$\approx C(\text{queen})$
Paris : France → Rome : Italy	$C(\text{Paris}) - C(\text{France}) + C(\text{Italy})$	$\approx C(\text{Rome})$
Windows : Microsoft → Android : Google	$C(\text{Windows}) - C(\text{Microsoft}) + C(\text{Google})$	$\approx C(\text{Android})$
Einstein : scientist → Picasso : painter	$C(\text{Einstein}) - C(\text{scientist}) + C(\text{painter})$	$\approx C(\text{Picasso})$
his : he → her : she	$C(\text{his}) - C(\text{he}) + C(\text{she})$	$\approx C(\text{her})$
Cu : copper → Au : gold	$C(\text{Cu}) - C(\text{copper}) + C(\text{gold})$	$\approx C(\text{Au})$

Why Do Linear Regularities Emerge?

The linear structure arises because word2vec's training objective effectively decomposes the *pointwise mutual information* (PMI) matrix of word co-occurrences. PMI is itself additive in log-space: $\text{PMI}(w, c) = \log P(c \mid w) - \log P(c)$. The embedding dot product $C(w)^\top C'(c)$ approximates PMI, and the constant offset between related pairs reflects the log-ratio of their respective conditional probabilities being approximately constant across the relation. GloVe (Section 16.10) makes this connection explicit.

16.9 Applications of Word Embeddings

Word embeddings have become the default input representation for essentially all NLP tasks. We illustrate three canonical applications.

Information Retrieval

Classical keyword-matching search cannot handle synonymy or paraphrase. Word embeddings provide a principled solution: represent a query and each document as an *aggregated embedding vector* (e.g. the sum or average of their word vectors), then retrieve documents ranked by cosine similarity to the query vector.

Example query: *"restaurants in mountain view that are not very good"*. After phrase detection, the query decomposes as: *restaurants* + *in* + *mountain view* + *that are* + *not very good*. Each phrase is embedded and summed to form a query vector, which can retrieve relevant results even if the retrieved documents use different vocabulary (e.g. *"dining"* instead of *"restaurants"*, *"poor quality"* instead of *"not very good"*). Note: this simple additive approach works well for short queries but degrades for long documents.

Document Classification and Similarity

With bag-of-words, two documents are judged as similar if they share many *identical* words. Under word embeddings, two documents can be judged similar even if they use different but semantically related vocabularies. For instance:

- D_0: *"a cat sat on the mat"*
- D_1: *"a cat sat on the mat"* (identical to D_0)
- D_2: *"a dog lay on the rug"*

Under BoW, D_0 is equally similar to both D_1 and D_2. Under word embeddings, D_2 is correctly identified as semantically similar to D_0, since *cat*≈*dog* and *mat*≈*rug* in the embedding space.

Sentiment Analysis

Rather than training an explicit classifier, word embeddings enable *geometry-based* sentiment analysis. Words with positive sentiment (*excellent*, *amazing*, *fantastic*) cluster together in the embedding space, as do words with negative sentiment (*terrible*, *awful*, *horrible*). A document's sentiment can be estimated by measuring the cosine distance of its aggregate embedding vector to the centroids of the positive and negative clusters — no labelled sentiment data required.

16.10 GloVe: Global Vectors for Word Representation

GloVe (Global Vectors for Word Representation), introduced by Pennington, Socher & Manning (2014), revisits the foundations of word embedding from a count-based perspective and makes explicit a connection that word2vec exploits only implicitly.

Core Idea: Co-occurrence Ratios Encode Meaning

GloVe's key insight is that the *ratio* of co-occurrence probabilities, rather than the probabilities themselves, encodes semantic relationships.

Let X_{ij} be the number of times word j appears in the context of word i, and let $P(k \mid i) = X_{ik}/\sum_j X_{ij}$ be the probability that word k appears in the context of word i.

Consider the words *ice* and *steam*. The table below shows co-occurrence probabilities with selected probe words:

Probability / Ratio	$k = solid$	$k = gas$	$k = water$	$k = fashion$
$P(k \mid ice)$	1.9×10^{-4}	6.6×10^{-5}	3.0×10^{-3}	1.7×10^{-5}
$P(k \mid steam)$	2.2×10^{-5}	7.8×10^{-4}	2.2×10^{-3}	1.8×10^{-5}
$P(k \mid ice)/P(k \mid steam)$	8.9	8.5×10^{-2}	≈ 1	≈ 1

The ratio is large for *solid* (related to ice but not steam), small for *gas* (related to steam but not ice), and close to 1 for *water* (related to both) and *fashion* (related to neither). The *ratio* thus discriminates related from unrelated words far more cleanly than raw probabilities — and GloVe's model is explicitly designed to predict these log-ratios.

The GloVe Loss Function

GloVe trains two sets of vectors $\mathbf{w}_i, \tilde{\mathbf{w}}_j \in \mathbb{R}^m$ (centre-word and context-word embeddings) plus scalar biases $b_i, \tilde{b}_j$ by minimising the following **weighted least-squares objective**:

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \Big(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij} \Big)^2 ,$$

where $f(X_{ij})$ is a weighting function that:

- Sets $f(0) = 0$ so that zero-count pairs contribute nothing (using the convention $0 \log 0 = 0$).
- Grows sub-linearly for large co-occurrence counts, preventing very frequent word pairs from dominating:

$$f(x) = \begin{cases} (x/x_{\max})^\alpha & \text{if } x < x_{\max}, \\ 1 & \text{otherwise,} \end{cases}$$

  with $x_{\max} = 100$ and $\alpha = 3/4$ as typical hyperparameters.

The objective says: the dot product $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j$ should approximate $\log X_{ij}$ (up to the bias terms). Since $\log X_{ij} \approx \text{PMI}(i, j) + \text{const}$, GloVe explicitly regresses on the PMI matrix — making precise what word2vec achieves only implicitly through its training dynamics.

After training, GloVe uses the sum $\mathbf{w}_i + \tilde{\mathbf{w}}_i$ as the final embedding for word i, averaging the information captured by the two roles (centre and context) the word plays.

Geometric Properties of GloVe

GloVe embeddings exhibit the same linear regularity as word2vec: the constant-difference-vector property holds for morphological transformations (*slow/slower/slowest*, *short/shorter/shortest*), gender relations (*man/woman*, *king/queen*), and analogical reasoning. The 2D PCA projections of GloVe vectors show clearly parallel trajectories for word families related by the same transformation (e.g. comparatives and superlatives, or male/female nominal pairs).

Nearest Neighbours in GloVe Space

A concrete qualitative evaluation: the nearest neighbours of the word *frog* in a GloVe embedding trained on Wikipedia are:

1. frog (self), frogs, toad, litoria, leptodactylidae, rana, lizard, eleutherodactylus.

These are all amphibians or closely related taxonomic families — the embedding has recovered a biologically meaningful neighbourhood with no explicit biological supervision.

Word2vec vs. GloVe

Property	**Word2vec**	**GloVe**
Training signal	Local context window	Global co-occurrence matrix
Objective	Predictive (log-likelihood)	Reconstruction (weighted LS)
PMI connection	Implicit	Explicit
Scalability	Online, very fast	Requires full co-occurrence matrix
Rare word handling	Good (skip-gram)	Moderate ($f(0) = 0$ helps)
Empirical performance	Similar	Similar

In practice, both methods produce embeddings of comparable quality on standard benchmarks. The choice is often made on engineering grounds: word2vec is more straightforward to train incrementally on a stream of data, while GloVe requires computing the full co-occurrence matrix first but then trains very quickly.

16.11 Recap: Word Embeddings

Word Embedding Lecture — Summary (Exam Checklist)

1. **Autoencoder**: network trained to reproduce its input; bottleneck hidden layer $J \ll I$ forces compressed representation; loss $= \|\hat{\mathbf{x}} - \mathbf{x}\|^2 + \lambda$ sparsity; applications include compression, dim. reduction, anomaly detection, VAEs, and word embedding.
2. **One-hot / BoW problems**: dimension $|V| \sim 10^5$–10^6 (curse of dimensionality); all vectors orthogonal (semantic blindness); no generalisation across related words.
3. **Distributional hypothesis (Firth, 1957)**: words that appear in similar contexts have similar meanings — the basis of all embedding methods.
4. **Word embedding**: mapping $C : V \to \mathbb{R}^m$ with $m \ll |V|$; dense, continuous, semantically smooth; similar words cluster together.
5. **N-gram LM**: approximation $P(w_i \mid w_1, \ldots, w_{i-1}) \approx P(w_i \mid w_{i-n+1}, \ldots, w_{i-1})$; estimated from counts; intractable for large n and $|V|$; short context misses long-range dependencies.
6. **NNLM (Bengio 2003)**: shared lookup table $\mathbf{C} \in \mathbb{R}^{|V| \times m}$; context embeddings concatenated $\to$ tanh hidden layer $\to$ softmax over $|V|$; complexity $\mathcal{O}(nm + nmh + h|V|)$; embeddings are a by-product; too slow for large data.
7. **Word2vec (Mikolov 2013)**: no hidden layer; shared projection; symmetric context; complexity $\mathcal{O}(nm + m \log |V|)$; two architectures: **Skip-gram** (centre word $\to$ context words, better for rare words) and **CBOW** (context words $\to$ centre word, faster for frequent words).
8. **CBOW update**: only context word vectors in $\mathbf{C}$ are updated per training step; overestimated probability $\Rightarrow$ subtract $C'(w_i)$ from context vectors; underestimated $\Rightarrow$ add.
9. **Word2vec vs. NNLM**: $\sim 1000\times$ speed-up; word analogy accuracy 53.3% (word2vec skip-gram) vs. 12.3% (NNLM).
10. **Geometric regularities**: semantic/syntactic relations as constant difference vectors; analogy $a : b :: c :?$ solved by $C(b) - C(a) + C(c)$; examples: king−man+woman≈queen; Paris−France+Italy≈Rome.
11. **Applications**: IR (query/doc as summed embeddings, cosine similarity); document similarity (semantics beyond lexical overlap); sentiment analysis (cosine distance to sentiment cluster centroids).
12. **GloVe (Pennington 2014)**: makes PMI structure explicit; loss $J = \sum_{i,j} f(X_{ij})(\mathbf{w}_i^\top \tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij})^2$; $f(x)$ down-weights very frequent pairs; final embedding $= \mathbf{w}_i + \tilde{\mathbf{w}}_i$; same linear regularities as word2vec; global co-occurrence vs. local window training.

Index